

Week 1 Labs

- 1. RTL simulation and Waveform**
- 2. Synthesis**
- 3. Netlist creation**
- 4. GLS simulation and Waveform**

Week 1 Labs

1. mkdir and git clone - SETUP!

- mkdir vsd -> cd vsd -> mkdir VLSI
- git clone "sky130".
 - Contains all the Verilog files, and libraries required for our lab experiments.

```
w0rblee@w0rblee-VirtualBox: ~/vsd/VLSI/sky130RTLDesignAndSynthesisWorkshop
928663 -rw-rw-r-- 1 w0rblee w0rblee 0 Sep 19 17:37 .magic_tkcon_hst
917515 drwx----- 5 w0rblee w0rblee 4096 Sep 19 18:00 snap/
917880 drwxrwxr-x 17 w0rblee w0rblee 4096 Sep 19 18:36 yosys/
917552 drwx----- 12 w0rblee w0rblee 4096 Sep 19 18:37 .config/
917573 -rw----- 1 w0rblee w0rblee 1458 Sep 19 19:27 .bash_history
917854 drwxrwxr-x 2 w0rblee w0rblee 4096 Sep 21 11:45 vsd/
917506 drwxr-x-- 18 w0rblee w0rblee 4096 Sep 21 11:45 ./

w0rblee@w0rblee-VirtualBox:~$ cd vsd/
w0rblee@w0rblee-VirtualBox:~/vsd$ mkdir L1
w0rblee@w0rblee-VirtualBox:~/vsd$ cd L1/
w0rblee@w0rblee-VirtualBox:~/vsd/L1$ cd ..
w0rblee@w0rblee-VirtualBox:~/vsd$ mkdir VLSI
w0rblee@w0rblee-VirtualBox:~/vsd$ cd VLSI/
w0rblee@w0rblee-VirtualBox:~/vsd/VLSI$ mkdir vsdflow
w0rblee@w0rblee-VirtualBox:~/vsd/VLSI$ git clone https://github.com/kunalg123/sky130RTLDesignAndSynthesisWorkshop.git
Cloning into 'sky130RTLDesignAndSynthesisWorkshop'...
remote: Enumerating objects: 417, done.
remote: Counting objects: 100% (69/69), done.
remote: Compressing objects: 100% (52/52), done.
remote: Total 417 (delta 19), reused 47 (delta 12), pack-reused 348 (from 1)
Receiving objects: 100% (417/417), 7.79 MiB | 8.68 MiB/s, done.
Resolving deltas: 100% (242/242), done.
w0rblee@w0rblee-VirtualBox:~/vsd/VLSI$ cd
sky130RTLDesignAndSynthesisWorkshop/ vsdflow/
w0rblee@w0rblee-VirtualBox:~/vsd/VLSI$ cd
sky130RTLDesignAndSynthesisWorkshop/ vsdflow/
w0rblee@w0rblee-VirtualBox:~/vsd/VLSI$ cd
sky130RTLDesignAndSynthesisWorkshop/ vsdflow/
w0rblee@w0rblee-VirtualBox:~/vsd/VLSI$ cd sky130RTLDesignAndSynthesisWorkshop/
w0rblee@w0rblee-VirtualBox:~/vsd/VLSI/sky130RTLDesignAndSynthesisWorkshop$ pwd
/home/w0rblee/vsd/VLSI/sky130RTLDesignAndSynthesisWorkshop
w0rblee@w0rblee-VirtualBox:~/vsd/VLSI/sky130RTLDesignAndSynthesisWorkshop$
```

2. Working with “iverilog and gtk-wave” - RUNNING!

- Navigate to “verilog_files” in the above cloned repo.
- Loading design and testbench in simulator.
 - cmd - “iverilog “design_name” “tb_name” ”.
- It generates an “a.out” file.
- Execute this “a.out” file.
 - cmd “./a.out”.
 - It will dump a “.vcd” file - waveform!
- Load this file in gtk-wave.
 - cmd “gtkwave “tb_name.vcd” ”

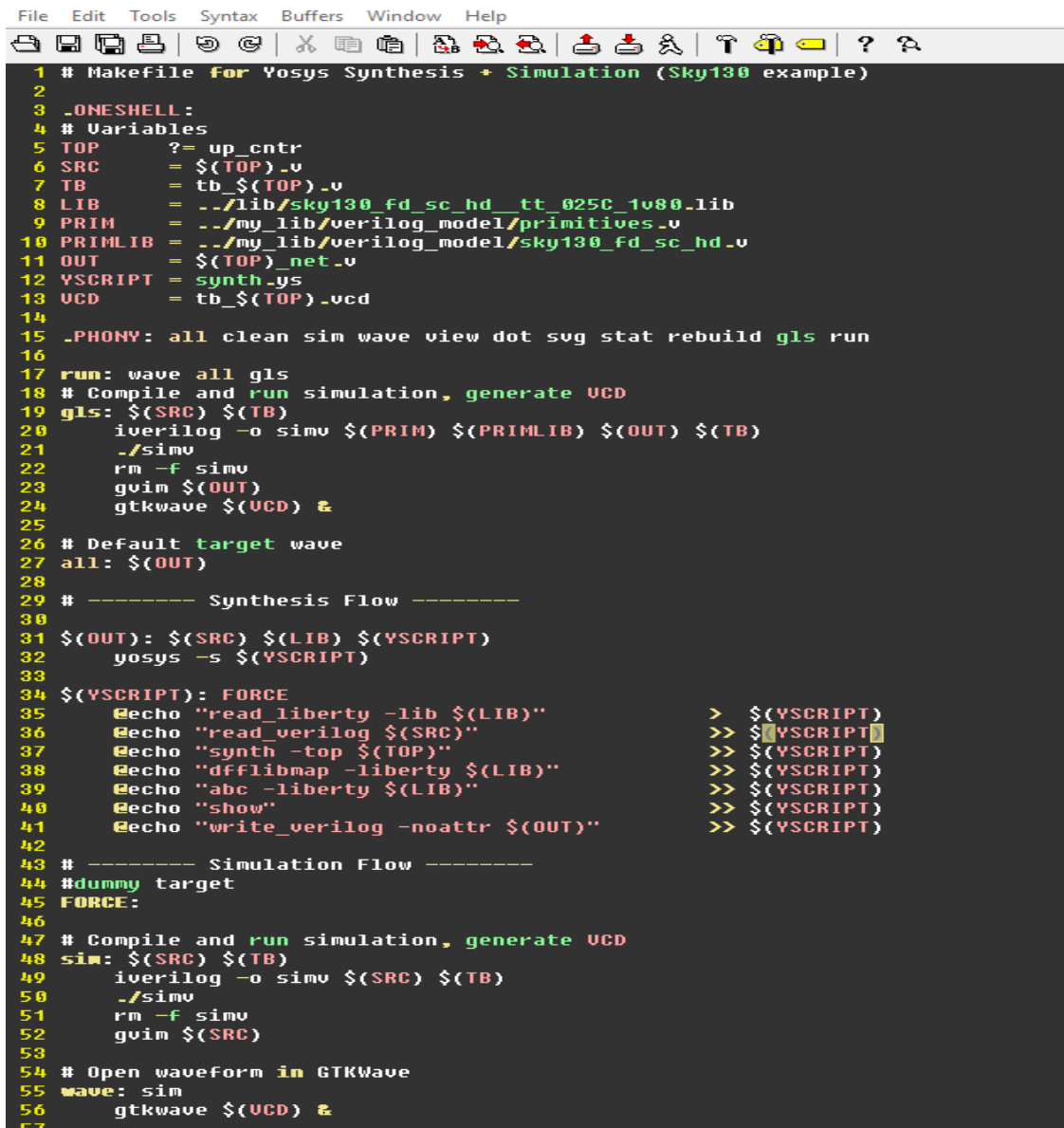
```
w0rblee@w0rblee-VirtualBox: ~/vsd/VLSI/sky130RTLDesignAndSynthesisWorkshop/verilog_files
Cloning into 'sky130RTLDesignAndSynthesisWorkshop'...
remote: Enumerating objects: 417, done.
remote: Counting objects: 100% (69/69), done.
remote: Compressing objects: 100% (52/52), done.
remote: Total 417 (delta 19), reused 47 (delta 12), pack-reused 348 (from 1)
Receiving objects: 100% (417/417), 7.79 MiB | 8.68 MiB/s, done.
Resolving deltas: 100% (242/242), done.
w0rblee@w0rblee-VirtualBox:~/vsd/VLSI$ cd
sky130RTLDesignAndSynthesisWorkshop/ vsdflow/
w0rblee@w0rblee-VirtualBox:~/vsd/VLSI$ cd
sky130RTLDesignAndSynthesisWorkshop/ vsdflow/
w0rblee@w0rblee-VirtualBox:~/vsd/VLSI$ cd
sky130RTLDesignAndSynthesisWorkshop/ vsdflow/
w0rblee@w0rblee-VirtualBox:~/vsd/VLSI$ cd sky130RTLDesignAndSynthesisWorkshop/
w0rblee@w0rblee-VirtualBox:~/vsd/VLSI/sky130RTLDesignAndSynthesisWorkshop$ pwd
/home/w0rblee/vsd/VLSI/sky130RTLDesignAndSynthesisWorkshop
w0rblee@w0rblee-VirtualBox:~/vsd/VLSI/sky130RTLDesignAndSynthesisWorkshop$ cd verilog_files/
w0rblee@w0rblee-VirtualBox:~/vsd/VLSI/sky130RTLDesignAndSynthesisWorkshop/verilog_files$ iverilog good_mux.v tb_good_
tb_good_counter.v tb_good_latch.v tb_good_mux.v tb_good_shift_reg.v
w0rblee@w0rblee-VirtualBox:~/vsd/VLSI/sky130RTLDesignAndSynthesisWorkshop/verilog_files$ iverilog good_mux.v tb_good_mux
.v
w0rblee@w0rblee-VirtualBox:~/vsd/VLSI/sky130RTLDesignAndSynthesisWorkshop/verilog_files$ ./a.out
VCD info: dumpfile tb_good_mux.vcd opened for output.
tb_good_mux.v:23: $finish called at 300000 (1ps)
w0rblee@w0rblee-VirtualBox:~/vsd/VLSI/sky130RTLDesignAndSynthesisWorkshop/verilog_files$ gtkwave tb_good_mux.vcd

GTKWave Analyzer v3.3.116 (w)1999-2023 BSI

[0] start time.
[300000] end time.
Exiting.
w0rblee@w0rblee-VirtualBox:~/vsd/VLSI/sky130RTLDesignAndSynthesisWorkshop/verilog_files$
```

Week 1 Labs

3. Simulating all the designs present in “verilog_files”.
 - a. “Verilog_files have all the design and corresponding Testbenches”.
 - b. same commands and flow to be used stated above in point “2”.
4. For ease of life, I have developed a makefile to run all the required commands!
 - a. Flow will be like this
 - i. Design name
 - ii. Design code
 - iii. RTL level simulation and waveform
 - iv. Synthesis Diagram
 - v. Netlist generation
 - vi. Netlist code
 - vii. GLS simulation and waveform
 - b. makefile - automated run
 - i. cmd - make run TOP=”design_name”.

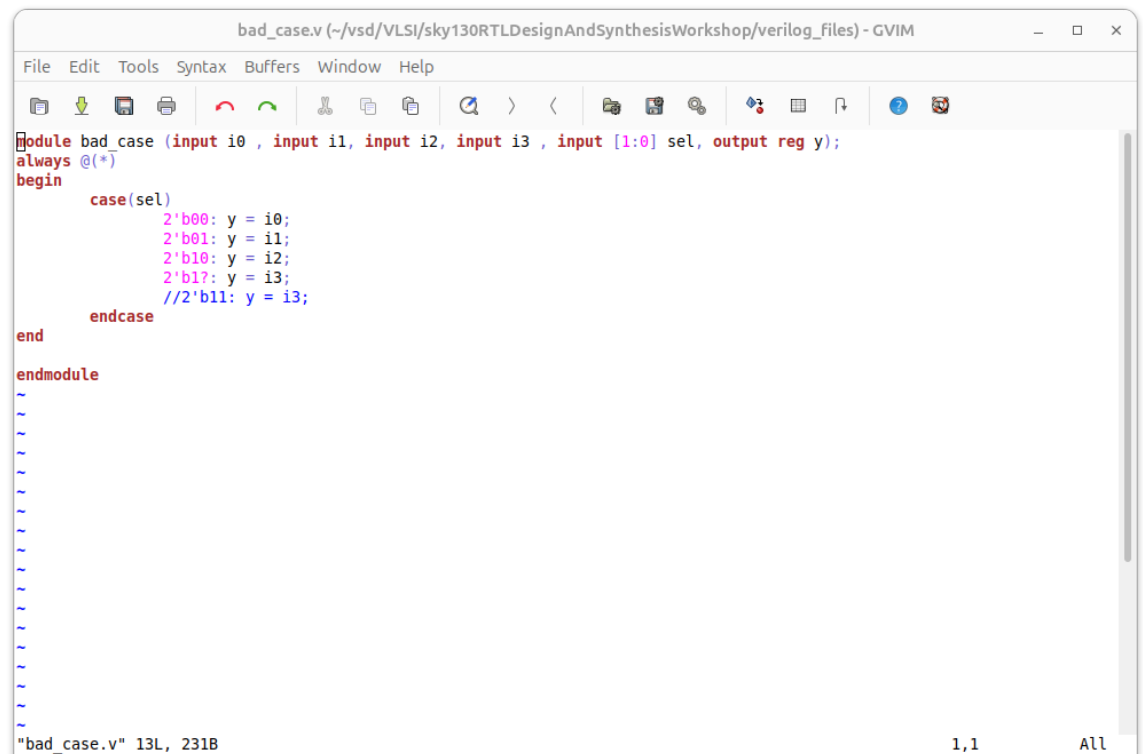


```
1 # Makefile for Yosys Synthesis + Simulation (Sky130 example)
2
3 _ONESHELL:
4 # Variables
5 TOP      ?= up_ctr
6 SRC      = $(TOP).v
7 TB       = tb_$(TOP).v
8 LIB      = ../lib/sky130_fd_sc_hd_tt_025C_1v80.lib
9 PRIM     = ../my_lib/verilog_model/primitives.v
10 PRIMLIB  = ../my_lib/verilog_model/sky130_fd_sc_hd.v
11 OUT      = $(TOP).net.v
12 VSCRIPT  = synth.ys
13 UCD      = tb_$(TOP).vcd
14
15 _PHONY: all clean sim wave view dot svg stat rebuild gls run
16
17 run: wave all gls
18 # Compile and run simulation, generate UCD
19 gls: $(SRC) $(TB)
20     iverilog -o simv $(PRIM) $(PRIMLIB) $(OUT) $(TB)
21     ./simv
22     rm -f simv
23     gvim $(OUT)
24     gtkwave $(UCD) &
25
26 # Default target wave
27 all: $(OUT)
28
29 # ----- Synthesis Flow -----
30
31 $(OUT): $(SRC) $(LIB) $(VSCRIPT)
32     yosys -s $(VSCRIPT)
33
34 $(VSCRIPT): FORCE
35     @echo "read_liberty -lib $(LIB)"           > $(VSCRIPT)
36     @echo "read_verilog $(SRC)"              >> $(VSCRIPT)
37     @echo "synth -top $(TOP)"                 >> $(VSCRIPT)
38     @echo "dfflibmap -liberty $(LIB)"         >> $(VSCRIPT)
39     @echo "abc -liberty $(LIB)"               >> $(VSCRIPT)
40     @echo "show"                             >> $(VSCRIPT)
41     @echo "write_verilog -noattr $(OUT)"      >> $(VSCRIPT)
42
43 # ----- Simulation Flow -----
44 #dummy target
45 FORCE:
46
47 # Compile and run simulation, generate UCD
48 sim: $(SRC) $(TB)
49     iverilog -o simv $(SRC) $(TB)
50     ./simv
51     rm -f simv
52     gvim $(SRC)
53
54 # Open waveform in GTKWave
55 wave: sim
56     gtkwave $(UCD) &
57
```

Week 1 Labs

5. Design Name - bad_case

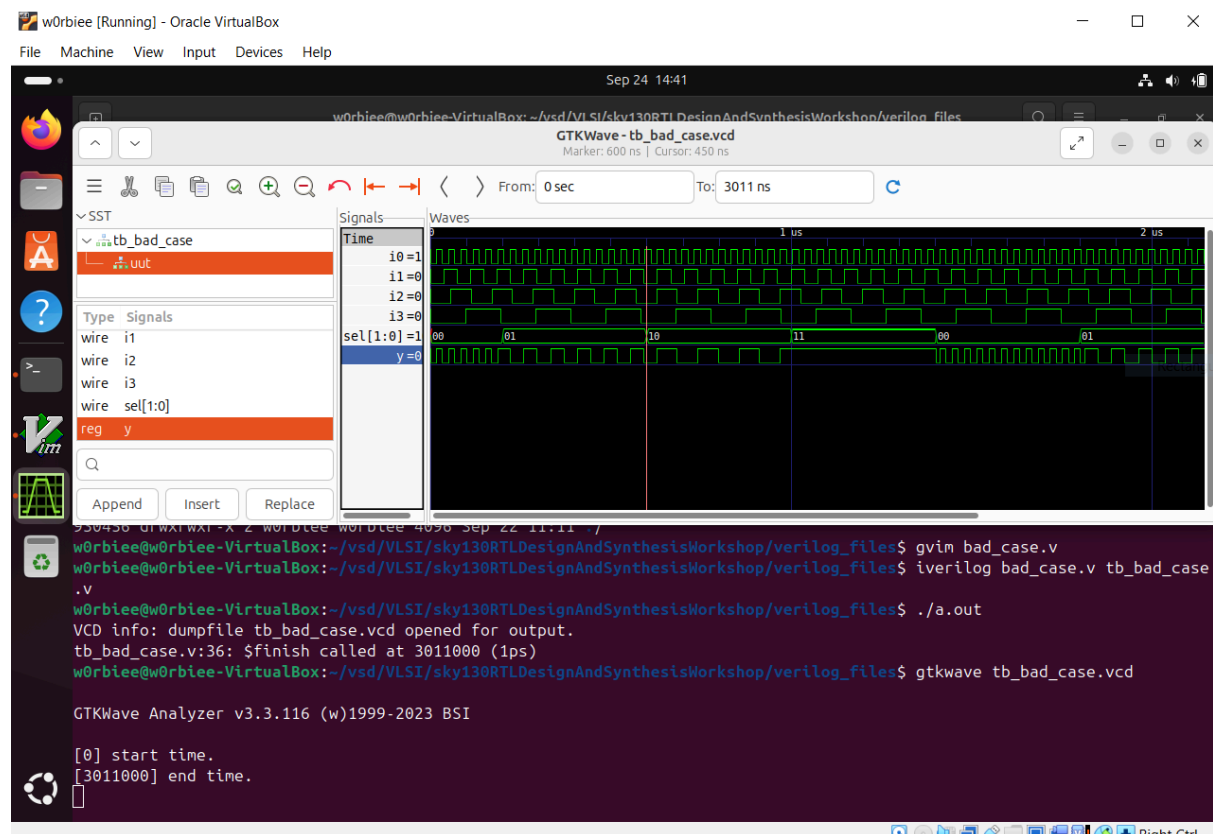
a. Design code



The screenshot shows a Verilog code editor window titled "bad_case.v (~/.vsd/VLSI/sky130RTLDesignAndSynthesisWorkshop/verilog_files) - GVIM". The code defines a module named "bad_case" with inputs "i0", "i1", "i2", "i3", and a 2-bit select signal "sel[1:0]", and a 1-bit output "y". The module contains an "always" block triggered by any change in the inputs, which implements a 4-to-1 multiplexer using a "case" statement. The output "y" is assigned the value of "i0" for "sel" = 00, "i1" for "sel" = 01, "i2" for "sel" = 10, and "i3" for "sel" = 11. The status bar at the bottom indicates the file is "bad_case.v" with 13 lines and 231 bytes.

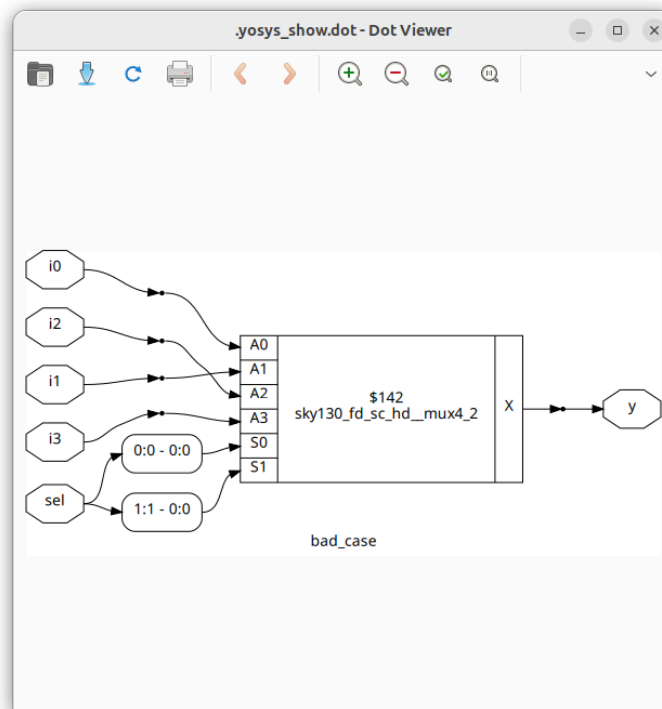
```
module bad_case (input i0 , input i1, input i2, input i3 , input [1:0] sel, output reg y);
always @(*)
begin
    case(sel)
        2'b00: y = i0;
        2'b01: y = i1;
        2'b10: y = i2;
        2'b11: y = i3;
        //2'b11: y = i3;
    endcase
end
endmodule
```

b. RTL level simulation and waveform



Week 1 Labs

c. Synthesis Diagram



d. Netlist generation - Netlist code

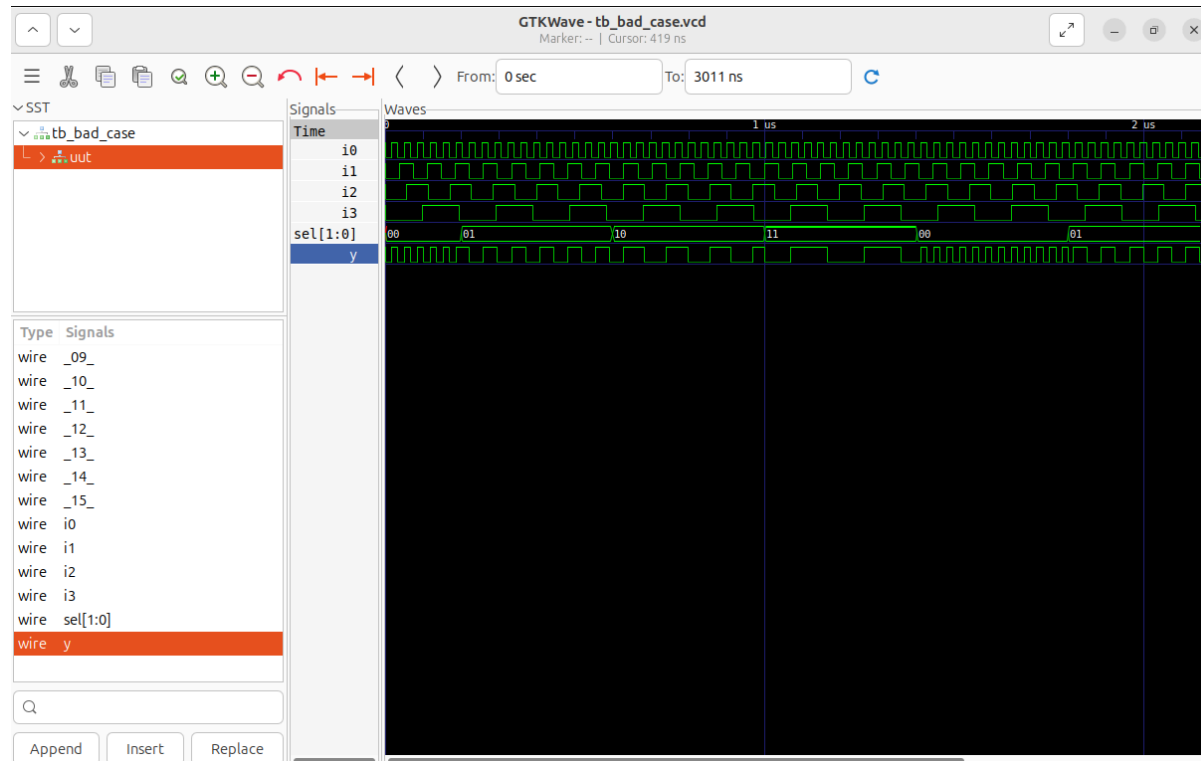
```
bad_case_net.v (-/vsd/VLSI/sky130RTLDesignAndSynthesisWorkshop/verilog_files) - GVIM1
File Edit Tools Syntax Buffers Window Help

module bad_case(i0, i1, i2, i3, sel, y);
    input i0;
    wire i0;
    input i1;
    wire i1;
    input i2;
    wire i2;
    input i3;
    wire i3;
    input [1:0] sel;
    wire [1:0] sel;
    output y;
    wire y;
    wire _00;
    wire _01;
    wire _02;
    wire _03;
    wire _04;
    wire _05;
    wire _06;
    wire _07;
    wire _08;
    wire _09;
    wire _10;
    wire _11;
    wire _12;
    wire _13;
    wire _14;
    wire _15;
    sky130_fd_sc_hd_mux4_2 _16 (
        .A0(_09),
        .A1(_10),
        .A2(_11),
        .A3(_12),
        .S0(_13),
        .S1(_14),
        .X(_15)
    );
    assign _13 = sel[0];
    assign _14 = sel[1];
    assign _11 = i2;
    assign _10 = i1;
    assign _12 = i0;
    assign y = _15;
endmodule

bad_case_net.v 32,1 8% bad_case_net.v 32,1 Bot
```

Week 1 Labs

e. GLS simulation and waveform



f. Observations

i. Pre-synthesis

1. Output was not following “i3” correctly during $sel == 2'b11$. Making it a bad case.

ii. Post-synthesis

1. Output is correctly following i3 when $sel == 2'b11$, making it a good case! Optimized the logic because the cell library replaced the “bad_case” with a pre-made correct 4x1 mux cell.
2. Classic case of RTL simulation $\neq$ Synthesis Simulation[GLS].

- iii. During RTL simulation, the case statement in the design exhibited overlapping conditions. Specifically, the literal $2'b1?$ expanded to both $2'b10$ and $2'b11$, causing an overlap with the explicitly defined $2'b10$ branch. In Verilog simulation semantics, the first matching branch is taken, so $sel = 2'b10$ always mapped to i2, and the subsequent match (i3) was ignored. This made the RTL case construct ambiguous or “bad.”

- iv. After synthesis, however, the tool interpreted $2'b1?$ as a don't-care pattern (casez expansion). This resolved the overlap and synthesized into a clean 4-to-1 multiplexer, where $sel = 00 \rightarrow i0$, $01 \rightarrow i1$, $10 \rightarrow i2$, and $11 \rightarrow i3$. Thus, a “bad” case at RTL transformed into a

Week 1 Labs

“good” case in the gate-level netlist. This highlights a potential mismatch between simulation and synthesized behavior when wildcards are used in case statements, and underscores the importance of writing non-overlapping, fully specified case constructs in RTL for consistent results.

Week 1 Labs

6. Design Name - bad_counter

a. Design code

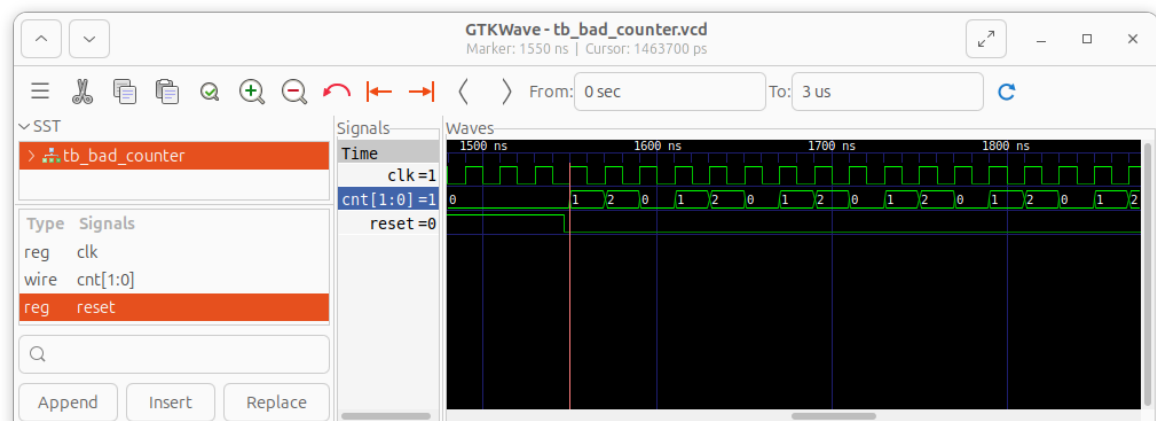
bad_counter.v (~\vsd\VLSI\sky130RTLDesignAndSynthesisWorkshop\verilog_files) - GVIM

File Edit Tools Syntax Buffers Window Help

```
1 module bad_counter (input clk , input reset , output reg [1:0] cnt);
2 wire res_int;
3
4 assign res_int = (cnt == 2'b11) | reset;
5
6 always @(posedge clk , posedge res_int)
7 begin
8     if(res_int)
9         cnt <= 2'b00;
10    else
11        cnt <= cnt+1;
12 end
13
14
15
16 endmodule
```

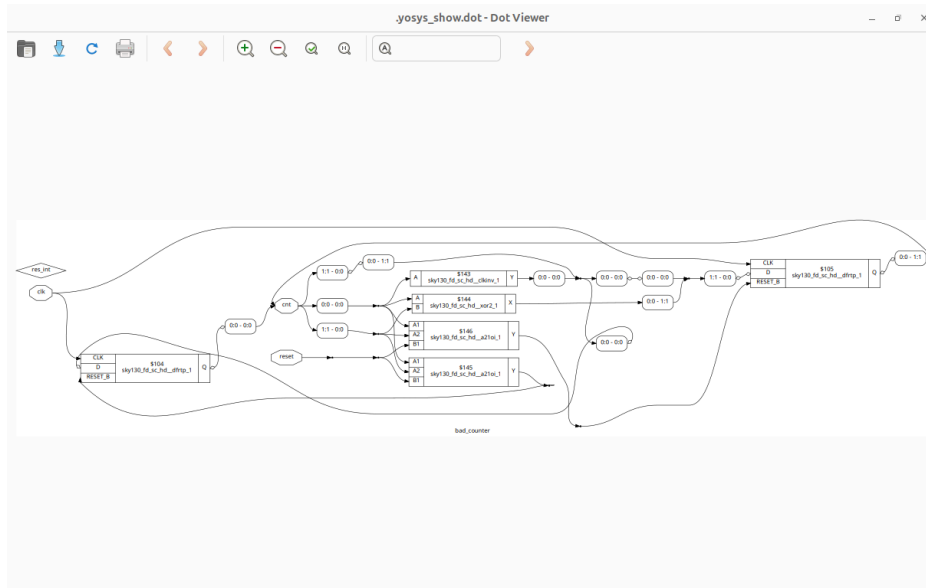
1,1 Top

b. RTL level simulation and waveform



Week 1 Labs

c. Synthesis Diagram



d. Netlist generation Netlist code

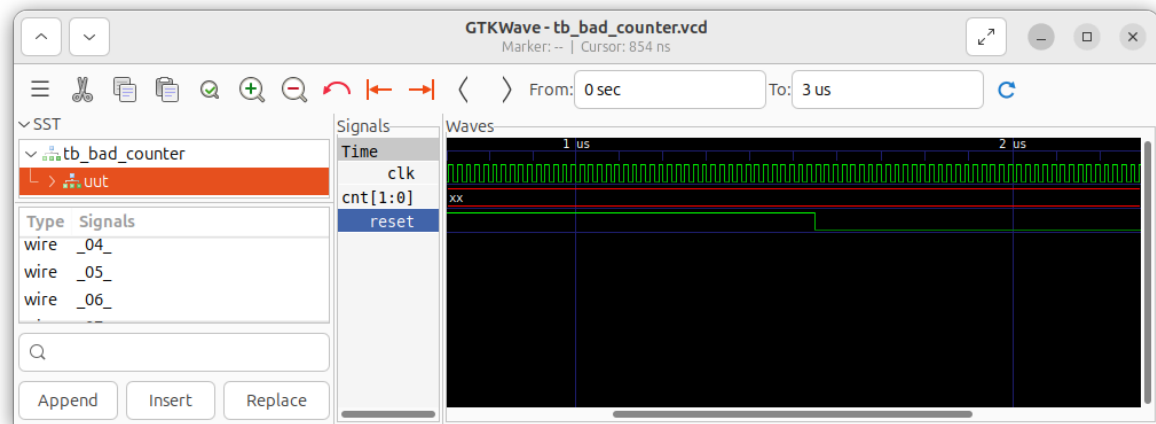
```
bad_counter_net.v (~vsd/VLSI/sky130RTLDesignAndSynthesisWorkshop/verilog_files) - GVIM1
File Edit Tools Syntax Buffers Window Help
[Icons]

2 32 sky130_fd_sc_hd_a2io_1_14_ ( 36 .Y(_03_)
3 module bad_counter(clk, reset, cnt) 37 );
; 38 sky130_fd_sc_hd_a2io_1_15_ (
4 input clk; 39 .A1(_05_),
5 wire clk; 40 .A2(_06_),
6 input reset; 41 .B1(_07_),
7 wire reset; 42 .Y(_03_)
8 output [1:0] cnt; 37 );
9 wire [1:0] cnt; 38 sky130_fd_sc_hd_a2io_1_15_ (
10 wire _00_; 39 .A1(_05_),
11 wire _01_; 40 .A2(_06_),
12 wire _02_; 41 .B1(_07_),
13 wire _03_; 42 .Y(_04_)
14 wire _04_; 43 );
15 wire _05_; 44 sky130_fd_sc_hd_dfrtp_1_16_ (
16 wire _06_; 45 .CLK(clk),
17 wire _07_; 46 .D(_08_[0]),
18 wire [1:0] _08_; 47 .Q(cnt[0]),
19 wire [1:0] _09_; 48 .RESET_B(_10_)
20 wire _10_; 49 );
21 wire _11_; 50 sky130_fd_sc_hd_dfrtp_1_17_ (
22 wire res_int; 51 .CLK(clk),
23 sky130_fd_sc_hd_clkinv_1_12_ ( 52 .D(_09_[1]),
24 .A(_05_), 53 .Q(cnt[1]),
25 .Y(_01_) 54 .RESET_B(_11_)
26 ); 55 );
27 sky130_fd_sc_hd_xor2_1_13_ ( 56 assign _08_[1] = cnt[1];
28 .A(_05_), 57 assign _09_[0] = _08_[0];
29 .B(_06_), 58 assign _05 = cnt[0];
30 .X(_02_) 59 assign _08[0] = _01_;
60 assign _06 = cnt[1];
61 assign _09[1] = _02_;
65 endmodule

bad_counter_net.v 22,1 2% bad_counter_net.v 45,14 88% bad_counter_net.v 53,15 Bot
```

Week 1 Labs

e. GLS simulation and waveform



f. Observations

- i. In RTL simulation, the counter design appeared to function correctly, cycling through $00 \rightarrow 01 \rightarrow 10 \rightarrow 00$ as intended. However, after synthesis, gate-level simulation produced an unknown (X) value on the counter output. This discrepancy arises because the design used a combinational expression of the counter output (`cnt == 2'b11`) as part of the asynchronous reset logic. While simulation tolerated this construct, synthesis introduced a feedback path from the counter output into its own asynchronous reset input. Such usage can cause glitches, races, and X-propagation, leading to unpredictable results in GLS. This highlights that asynchronous resets should only be driven by external signals, and terminal count detection should be handled synchronously within the clock domain to ensure consistent behavior across RTL and synthesized implementations.

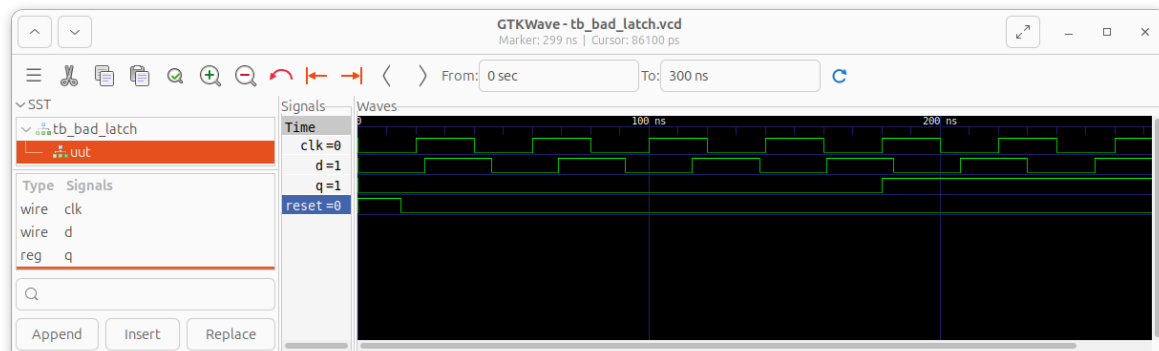
Week 1 Labs

7. Design Name - bad_latch

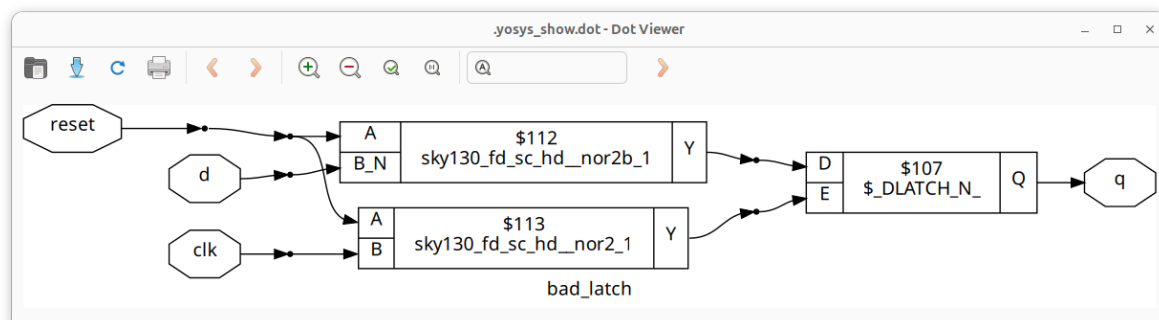
a. Design code

```
bad_latch.v (~vsd/VLSI/sky130RTLDesignAndSynthesisWorkshop/verilog_files) - GVIM
File Edit Tools Syntax Buffers Window Help
1
2 module bad_latch (input clk , input reset , input d , output reg q);
3 always @ (clk,reset)
4 begin
5     if(reset)
6         q <= 1'b0;
7     else if(clk)
8         q <= d;
9 end
10 endmodule
10 lines yanked into "+ 1,0-1 All
```

b. RTL level simulation and waveform

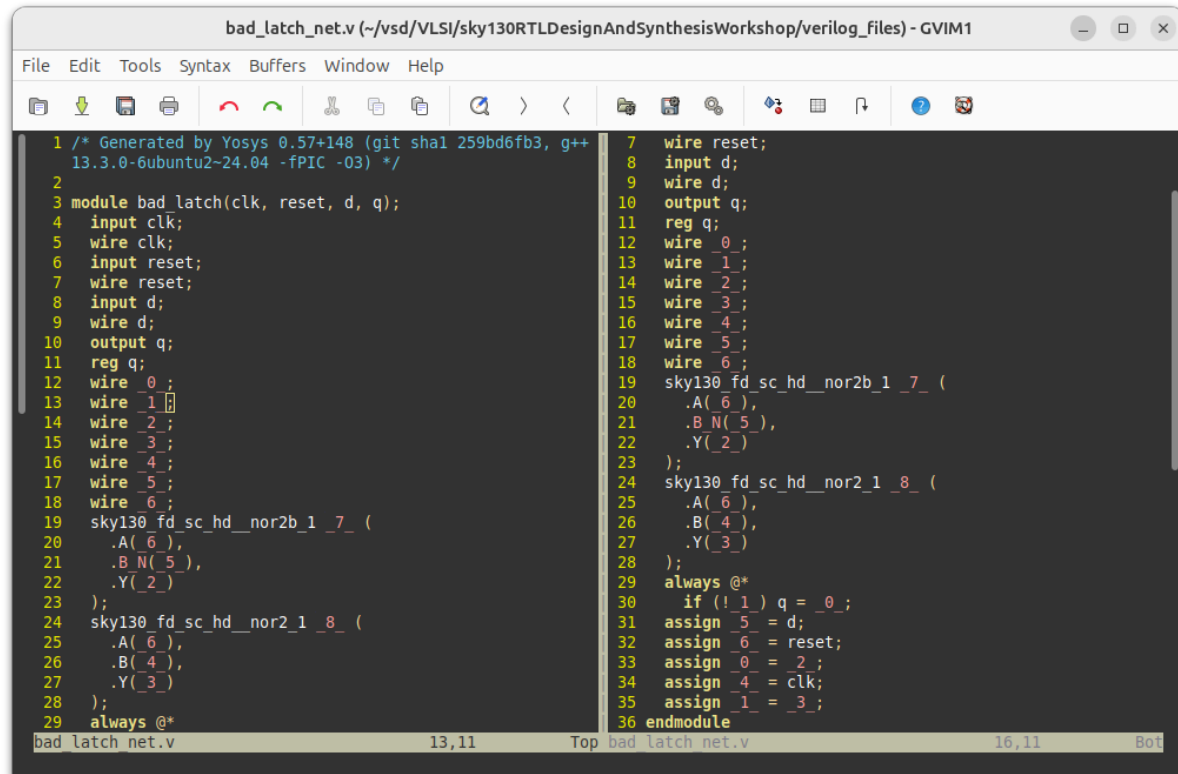


c. Synthesis Diagram



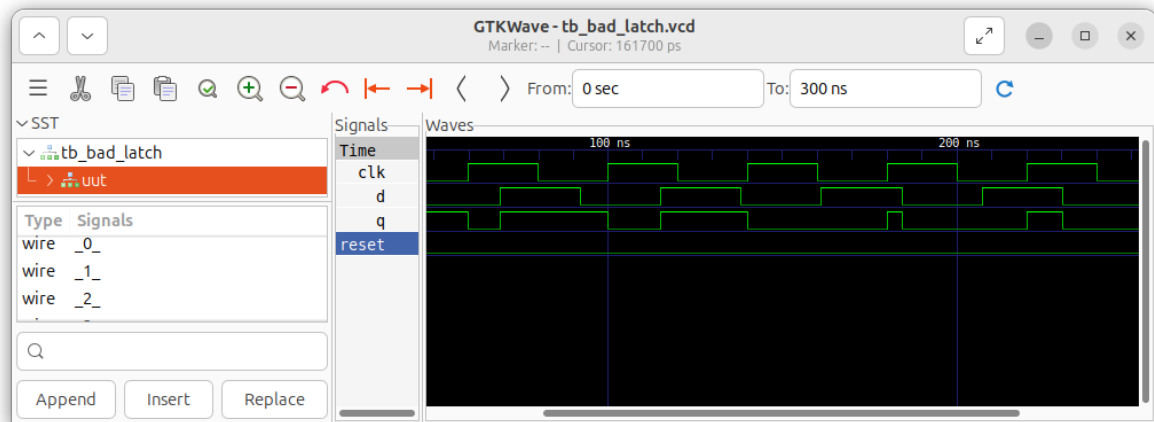
d. Netlist generation Netlist code

Week 1 Labs



```
1 /* Generated by Yosys 0.57+148 (git sha1 259bd6fb3, g++  
13.3.0-6ubuntu2~24.04 -fPIC -O3) */  
2  
3 module bad_latch(clk, reset, d, q);  
4   input clk;  
5   wire clk;  
6   input reset;  
7   wire reset;  
8   input d;  
9   wire d;  
10  output q;  
11  reg q;  
12  wire _0_;  
13  wire _1_;  
14  wire _2_;  
15  wire _3_;  
16  wire _4_;  
17  wire _5_;  
18  wire _6_;  
19  sky130_fd_sc_hd_nor2b_1_7_ (  
20    .A(_6_),  
21    .B_N(_5_),  
22    .Y(_2_)  
23  );  
24  sky130_fd_sc_hd_nor2_1_8_ (  
25    .A(_6_),  
26    .B(_4_),  
27    .Y(_3_)  
28  );  
29  always @*  
30    if (!_1_) q = _0_;  
31    assign _5_ = d;  
32    assign _6_ = reset;  
33    assign _0_ = _2_;  
34    assign _4_ = clk;  
35    assign _1_ = _3_;  
36 endmodule
```

e. GLS simulation and waveform



f. Observations

i. Pre - Synthesis

1. In pre-synthesis simulation, **q** only updates when **clk** = 1. When **clk** = 0, **q** holds its previous value. This behavior gives the impression that **d** is sampled conditionally based on **clk**, and the latch behavior is not clearly visible at this abstraction level.

ii. Post - Synthesis

1. After synthesis, the synthesizer infers a level-sensitive latch because the always block is sensitive to the level of **clk**. As a

Week 1 Labs

result, `q` continuously follows `d` while `clk = 1`, revealing the actual hardware behavior of the latch. This post-synthesis waveform shows more transparency than what RTL simulation suggests.

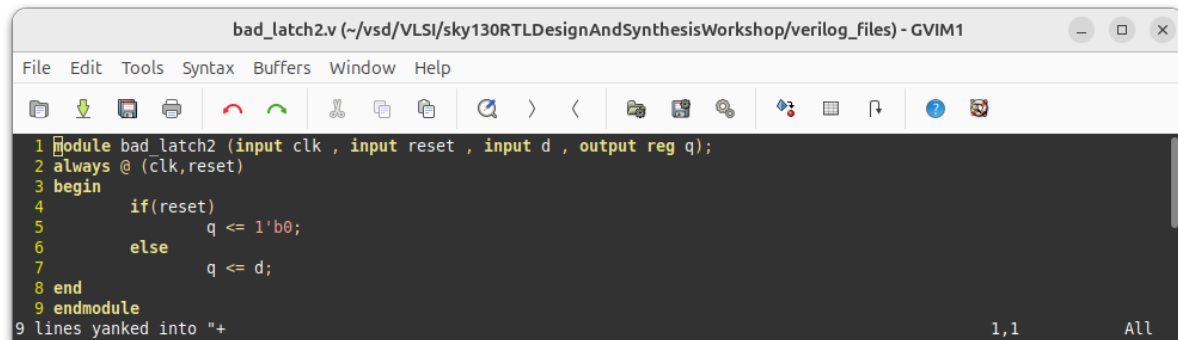
iii. Conclusion

1. This observation highlights that using a signal like `clk` as a level in the sensitivity list can lead to unintended latch inference and continuous transparency. To ensure predictable and correct post-synthesis behavior, proper D flip-flop behavior should be implemented using `always @(posedge clk)` along with an optional asynchronous reset.

Week 1 Labs

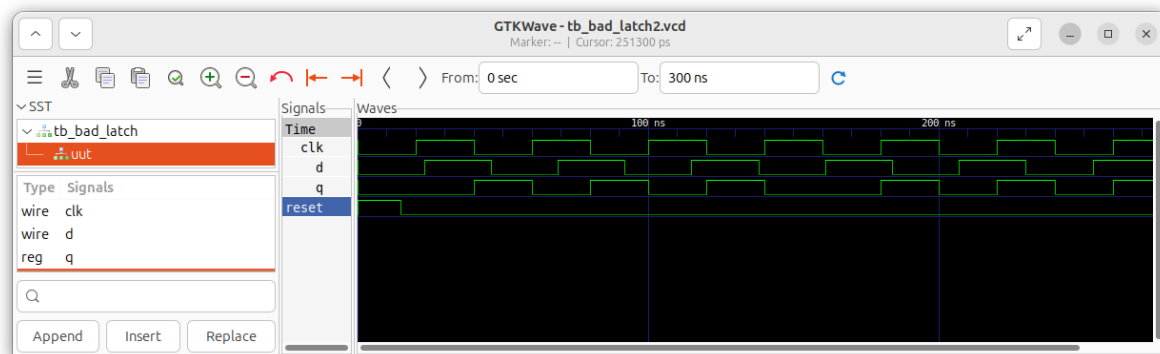
8. Design Name - bad_latch_2

a. Design code

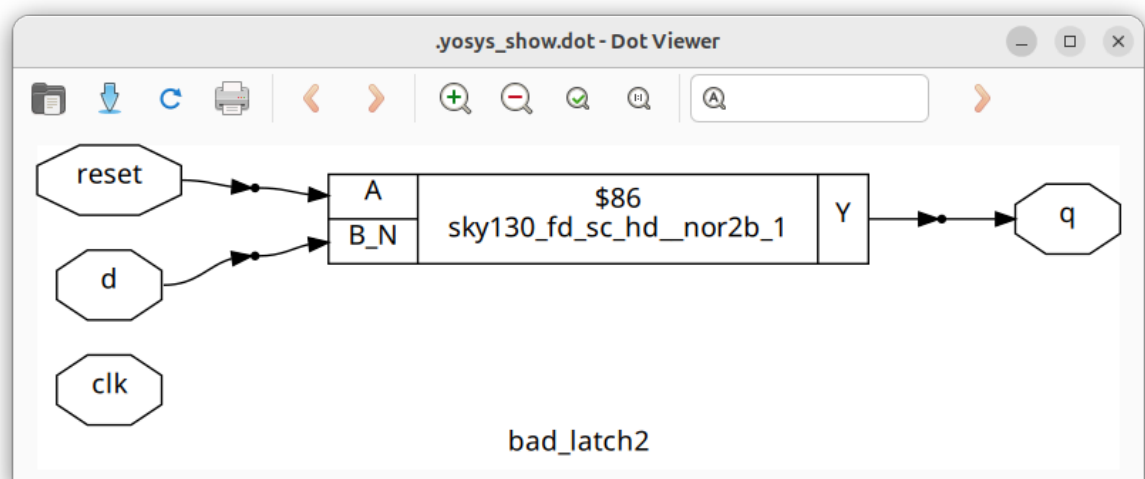


```
1 module bad_latch2 (input clk , input reset , input d , output reg q);
2 always @ (clk,reset)
3 begin
4     if(reset)
5         q <= 1'b0;
6     else
7         q <= d;
8 end
9 endmodule
9 lines yanked into "+
```

b. RTL level simulation and waveform

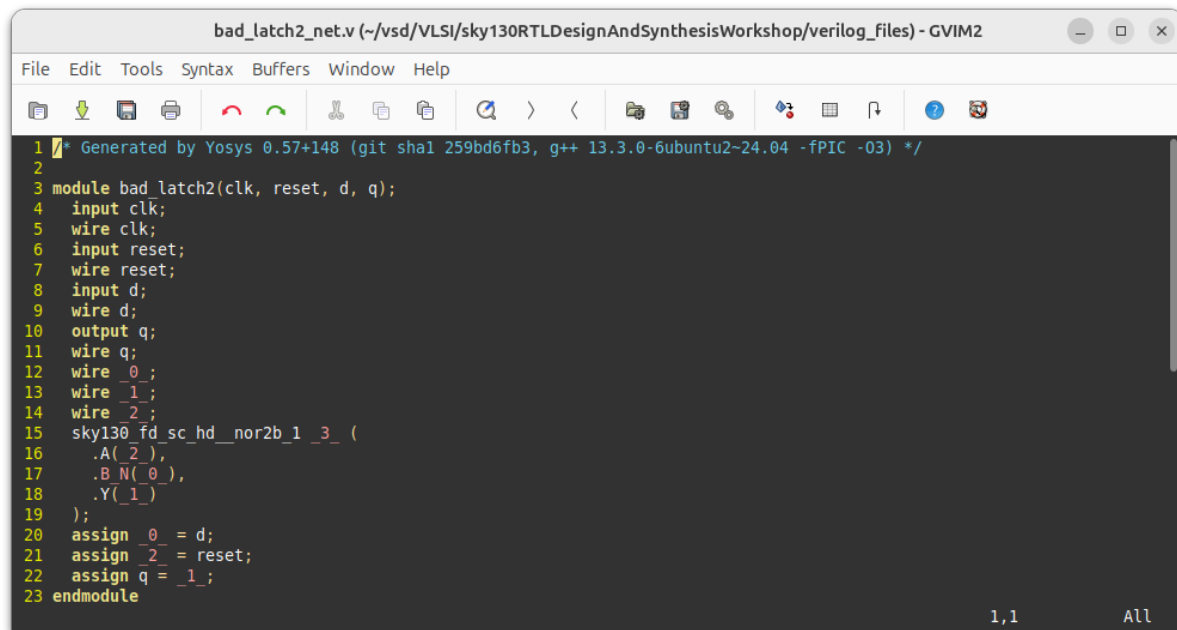


c. Synthesis Diagram



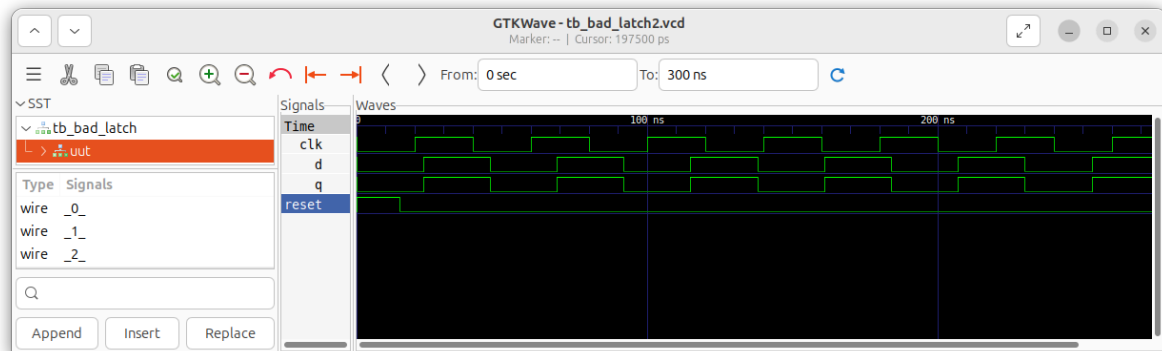
Week 1 Labs

d. Netlist generation Netlist code



```
1 Generated by Yosys 0.57+148 (git sha1 259bd6fb3, g++ 13.3.0-6ubuntu2~24.04 -fPIC -O3) */
2
3 module bad_latch2(clk, reset, d, q);
4   input clk;
5   wire clk;
6   input reset;
7   wire reset;
8   input d;
9   wire d;
10  output q;
11  wire q;
12  wire _0;
13  wire _1;
14  wire _2;
15  sky130_fd_sc_hd_nor2b_1_3_ (
16    .A(_2),
17    .B N(_0),
18    .Y(_1)
19  );
20  assign _0 = d;
21  assign _2 = reset;
22  assign q = _1;
23 endmodule
```

e. GLS simulation and waveform



f. Observations

i. Pre - Synthesis

1. In RTL simulation, the `bad_latch2` module appeared to behave like an edge-sensitive D flip-flop because `q` updated when `clk` toggled. However, the sensitivity list used (`clk`, `reset`) without specifying edges.

ii. Post - Synthesis

1. After synthesis, the synthesizer inferred a level-sensitive latch instead of a flip-flop. This is because `clk` in the sensitivity list is treated as a regular signal in hardware, not as an edge. Consequently, the relationship between `q` and `d` became transparent whenever `clk` is high, producing a level-sensitive latch, not an edge-triggered flip-flop.

iii. Conclusion

1. This highlights a key RTL vs. synthesis mismatch: simulation may hide improper sensitivity, while synthesis generates the

Week 1 Labs

actual hardware behavior, potentially causing timing hazards or unintended transparent latches.

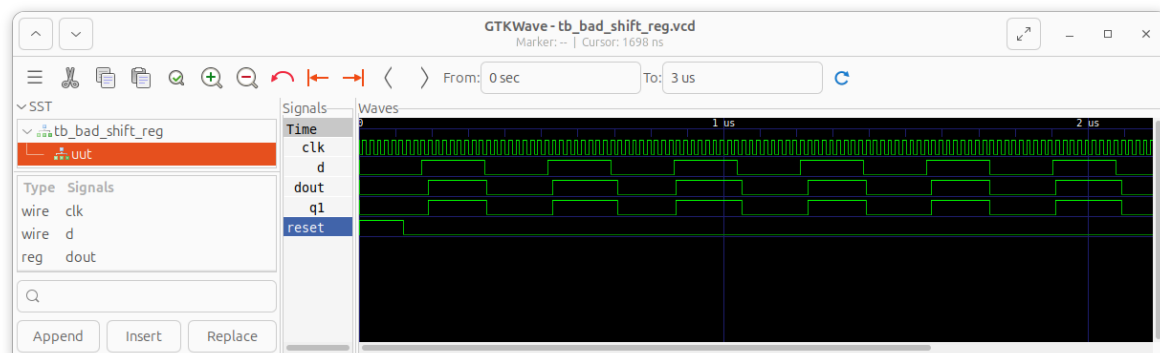
Week 1 Labs

9. Design Name - bad_shift_reg

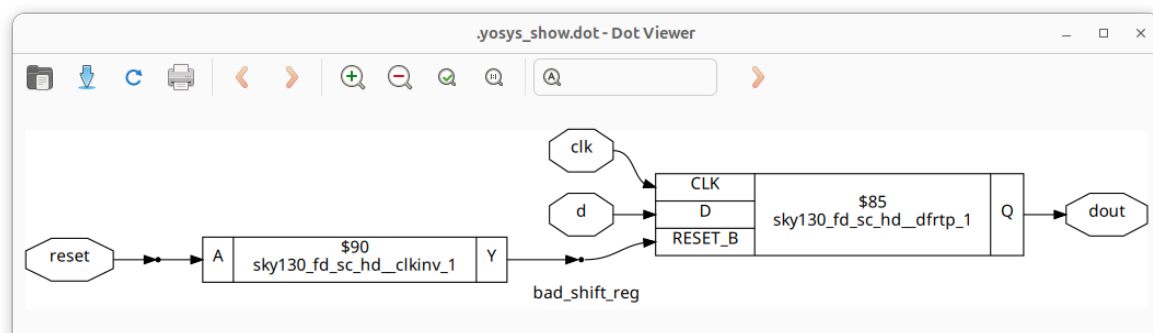
a. Design code

```
bad_shift_reg.v (~\vsd\VLSI/sky130RTLDesignAndSynthesisWorkshop/verilog_files) - GVIM
File Edit Tools Syntax Buffers Window Help
1 []
2 module bad_shift_reg (input clk , input reset , input d , output reg dout);
3 reg q1;
4 always @ (posedge clk , posedge reset)
5 begin
6     if(reset)
7     begin
8         q1 = 1'b0;
9         dout = 1'b0;
10    end
11    else
12    begin
13        q1 = d;
14        dout = q1;
15    end
16 end
17 endmodule
```

b. RTL level simulation and waveform

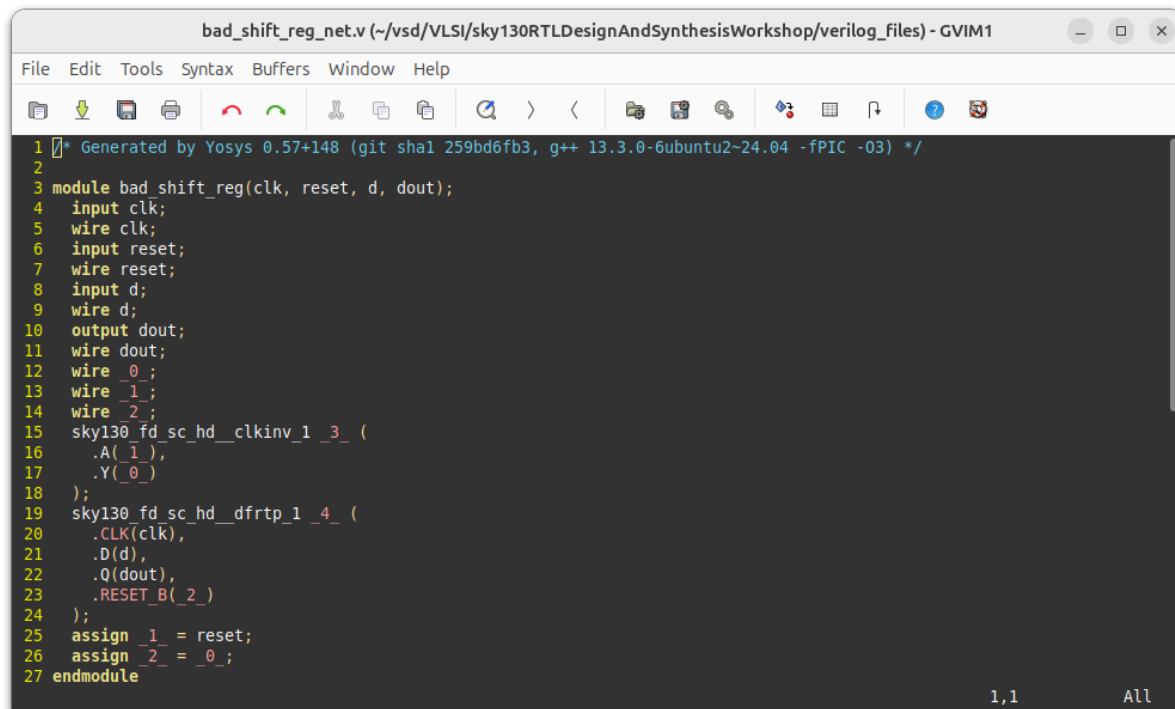


c. Synthesis Diagram



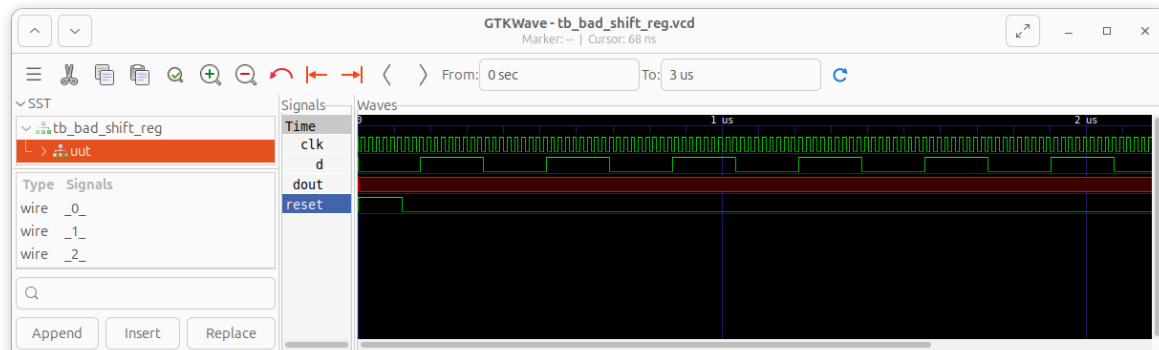
Week 1 Labs

d. Netlist generation Netlist code



```
1 Generated by Yosys 0.57+148 (git sha1 259bd6fb3, g++ 13.3.0-6ubuntu2~24.04 -fPIC -O3) */
2
3 module bad_shift_reg(clk, reset, d, dout);
4   input clk;
5   wire clk;
6   input reset;
7   wire reset;
8   input d;
9   wire d;
10  output dout;
11  wire dout;
12  wire _0_;
13  wire _1_;
14  wire _2_;
15  sky130_fd_sc_hd_clkinv_1_3_ (
16    .A(_1_),
17    .Y(_0_)
18  );
19  sky130_fd_sc_hd_dfrtp_1_4_ (
20    .CLK(clk),
21    .D(d),
22    .Q(dout),
23    .RESET_B(_2_)
24  );
25  assign _1_ = reset;
26  assign _2_ = _0_;
27 endmodule
```

e. GLS simulation and waveform



f. Observations

i. Pre - Synth

1. During RTL simulation, the `bad_shift_reg` module showed `dout` correctly following `q1` and `d`.

ii. Post - Synth

1. However, after synthesis, `dout` became unknown (X). This occurred because the RTL used blocking assignments (`=`) in a clocked always block. In simulation, blocking assignments execute sequentially, masking timing issues, but in real hardware all flip-flops update simultaneously at the clock edge.

iii. Conclusion

1. As a result, `dout` attempted to sample `q1` at the same edge it was being updated, leading to undefined behavior. Using non-blocking assignments (`<=`) for all sequential elements

Week 1 Labs

resolves this, ensuring proper register timing and matching RTL and gate-level simulation.

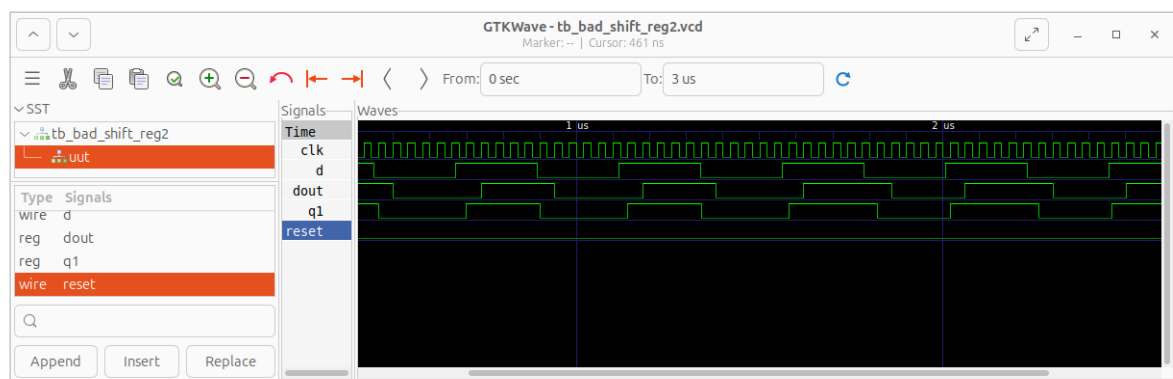
Week 1 Labs

10. Design Name - bad_shift_reg2

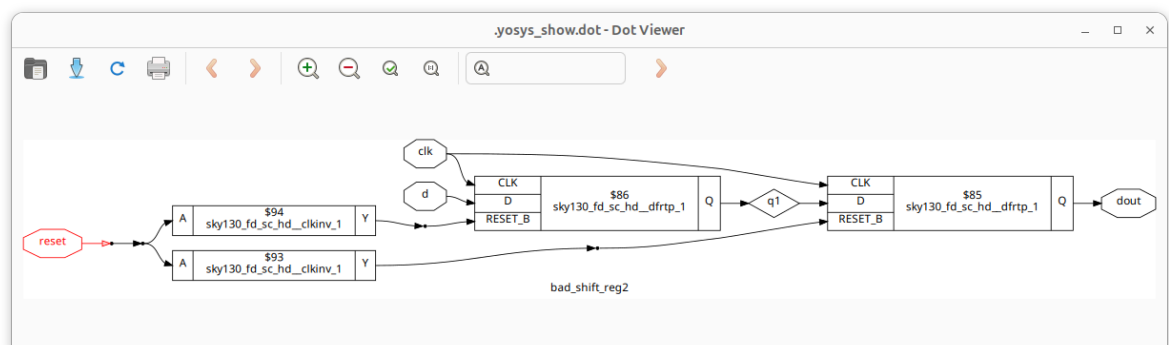
a. Design code

```
bad_shift_reg2.v (~vsd/VLSI/sky130RTLDesignAndSynthesisWorkshop/verilog_files) - GVIM
File Edit Tools Syntax Buffers Window Help
1 []
2 module bad_shift_reg2 (input clk , input reset , input d , output reg dout);
3 reg q1;
4 always @ (posedge clk , posedge reset)
5 begin
6     if(reset)
7     begin
8         q1 = 1'b0;
9         dout = 1'b0;
10    end
11    else
12    begin
13        dout = q1;
14        q1 = d;
15    end
16 end
17 endmodule
17 lines yanked into "+ 1,0-1 All
```

b. RTL level simulation and waveform

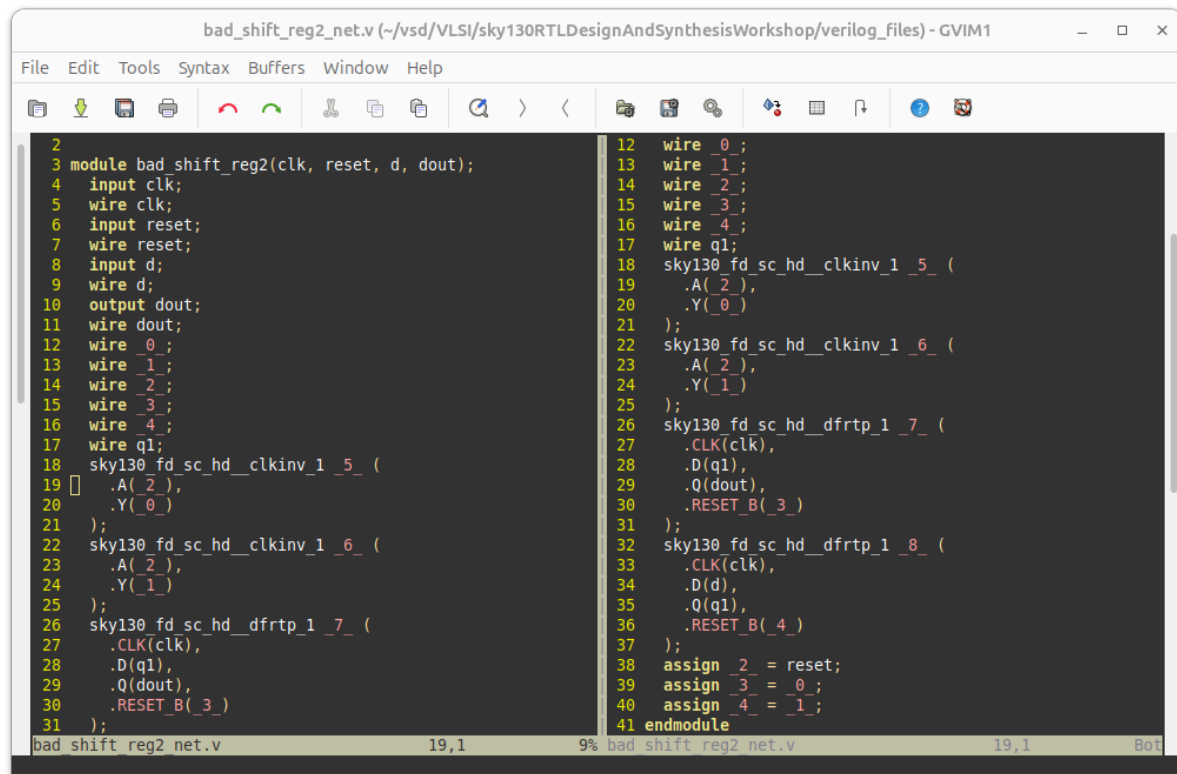


c. Synthesis Diagram



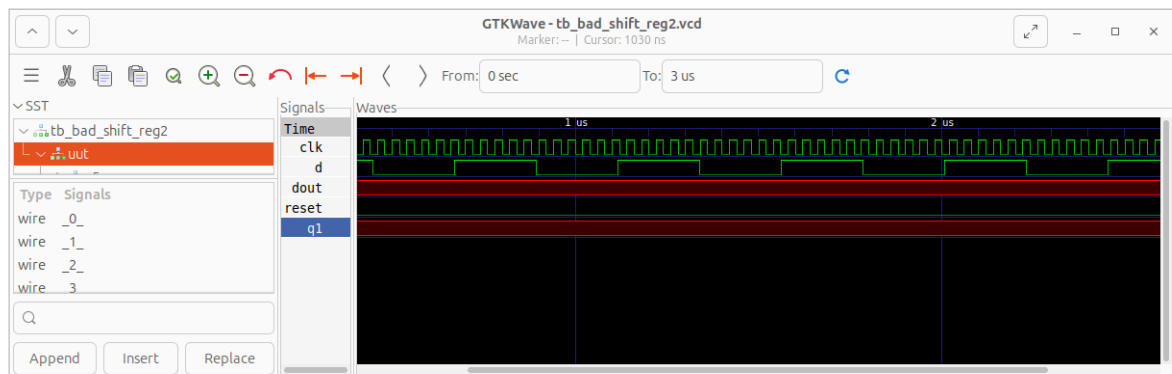
Week 1 Labs

d. Netlist generation Netlist code



```
2
3 module bad_shift_reg2(clk, reset, d, dout);
4   input clk;
5   wire clk;
6   input reset;
7   wire reset;
8   input d;
9   wire d;
10  output dout;
11  wire dout;
12  wire _0_;
13  wire _1_;
14  wire _2_;
15  wire _3_;
16  wire _4_;
17  wire q1;
18  sky130 fd_sc_hd_clkinv_1_5_ (
19    .A(_2_),
20    .Y(_0_)
21  );
22  sky130 fd_sc_hd_clkinv_1_6_ (
23    .A(_2_),
24    .Y(_1_)
25  );
26  sky130 fd_sc_hd_dfrtp_1_7_ (
27    .CLK(clk),
28    .D(q1),
29    .Q(dout),
30    .RESET_B(_3_)
31  );
32  sky130 fd_sc_hd_dfrtp_1_8_ (
33    .CLK(clk),
34    .D(d),
35    .Q(q1),
36    .RESET_B(_4_)
37  );
38  assign _2_ = reset;
39  assign _3_ = 0;
40  assign _4_ = 1;
41 endmodule
```

e. GLS simulation and waveform



f. Observations

- During RTL simulation, the shift register using blocking assignments (=) behaved as expected: **q1** followed **d** and **dout** followed the previous value of **q1**.
- However, after synthesis, both **q1** and **dout** became unknown (X). This occurs because blocking assignments in clocked always blocks create a combinational loop in hardware, which cannot be implemented correctly by the synthesizer. Non-blocking assignments (<=) ensure all sequential registers update in parallel, producing predictable, synthesizable behavior.
- Lesson: Always use non-blocking assignments for sequential logic to ensure consistency between simulation and synthesized hardware.

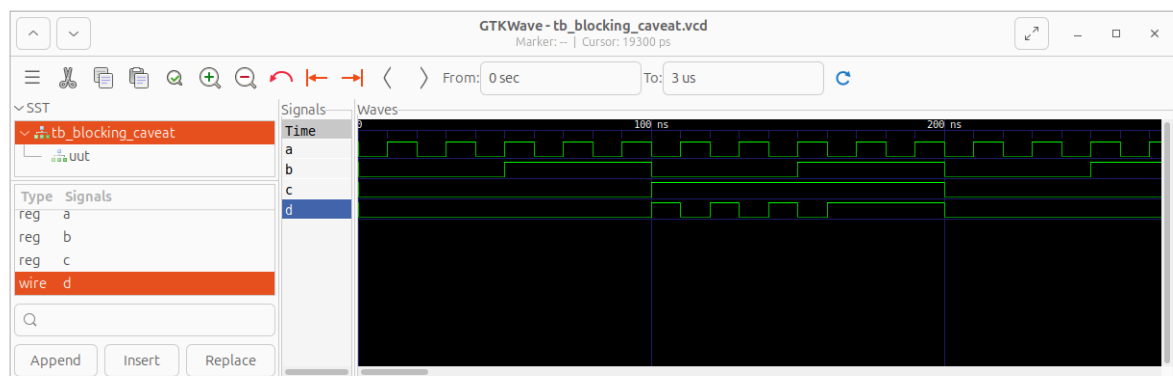
Week 1 Labs

11. Design Name - blocking_caveat

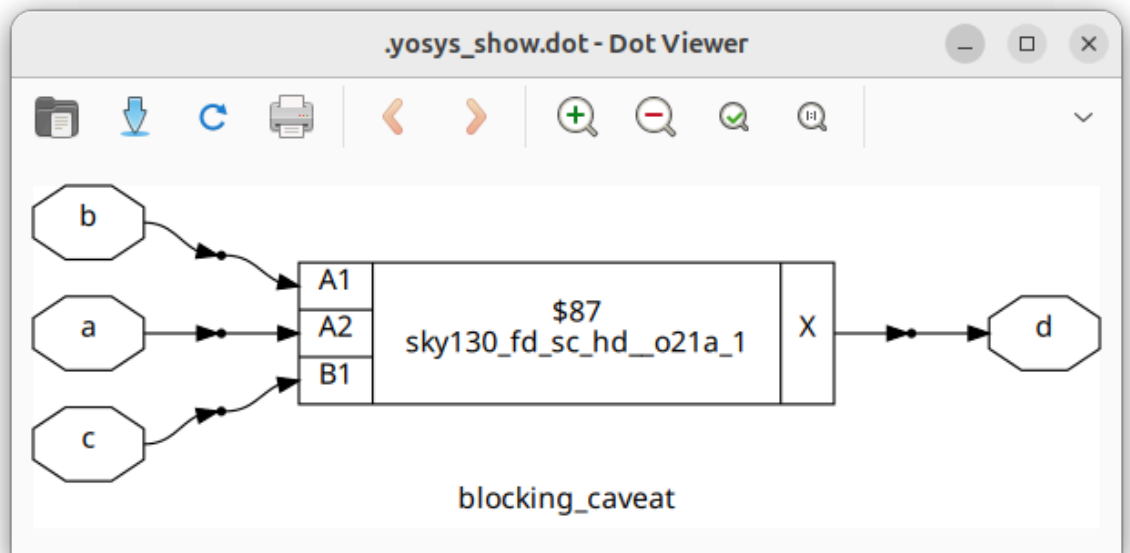
a. Design code

```
blocking_caveat.v (~vsd/VLSI/sky130RTLDesignAndSynthesisWorkshop/verilog_files) - GVIM
File Edit Tools Syntax Buffers Window Help
1 module blocking_caveat (input a , input b , input c , output reg d);
2 reg x;
3 always @ (*)
4 begin
5     d = x & c;
6     x = a | b;
7 end
8 endmodule
8 lines yanked into "+
1,1 All
```

b. RTL level simulation and waveform

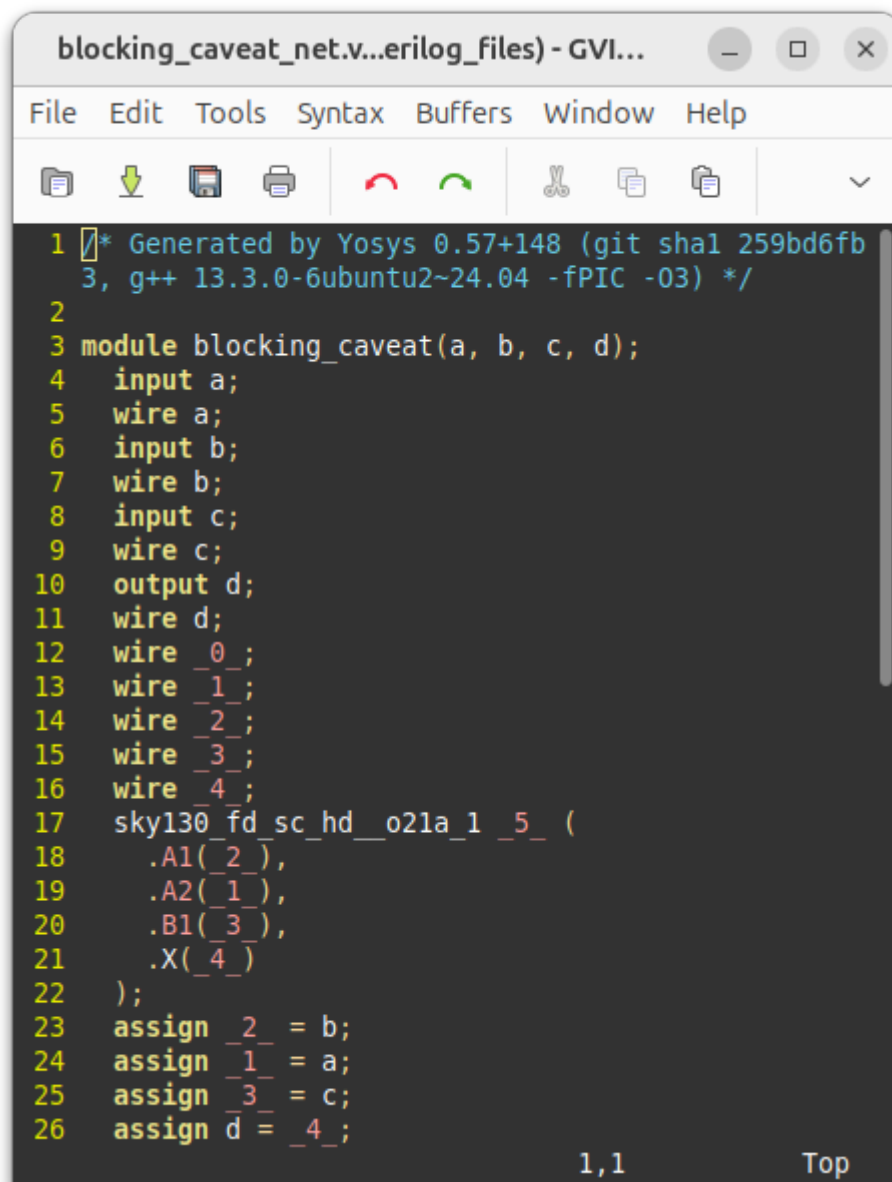


c. Synthesis Diagram



Week 1 Labs

d. Netlist generation Netlist code

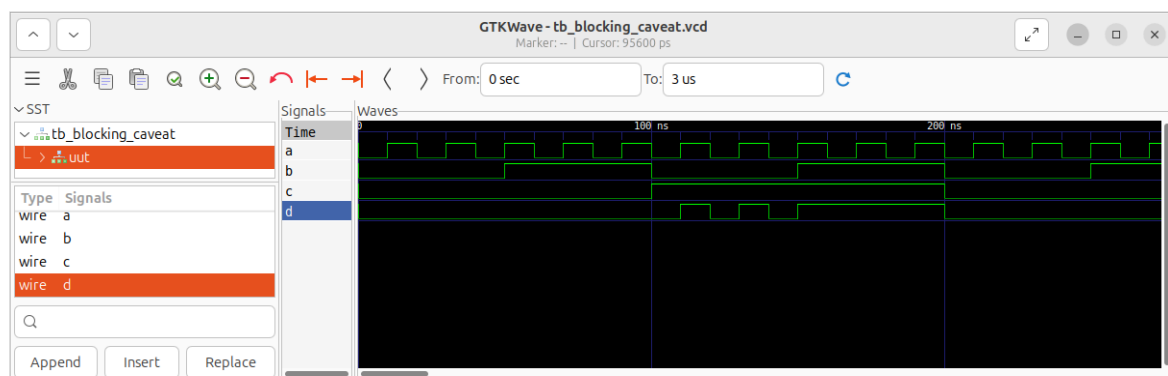


The screenshot shows a text editor window titled "blocking_caveat_net.v...erilog_files) - GVI...". The window contains Verilog code for a module named "blocking_caveat". The code defines four inputs (a, b, c, d) and four wires (_0_, _1_, _2_, _3_, _4_). It uses a "sky130" component to generate a signal. The code is as follows:

```
1 /* Generated by Yosys 0.57+148 (git sha1 259bd6fb
2 3, g++ 13.3.0-6ubuntu2~24.04 -fPIC -O3) */
3 module blocking_caveat(a, b, c, d);
4   input a;
5   wire a;
6   input b;
7   wire b;
8   input c;
9   wire c;
10  output d;
11  wire d;
12  wire _0_;
13  wire _1_;
14  wire _2_;
15  wire _3_;
16  wire _4_;
17  sky130 fd_sc_hd_o21a_1_5_ (
18    .A1(_2_),
19    .A2(_1_),
20    .B1(_3_),
21    .X(_4_)
22  );
23  assign _2_ = b;
24  assign _1_ = a;
25  assign _3_ = c;
26  assign d = _4_;
```

The bottom right corner of the window shows "1,1" and "Top".

e. GLS simulation and waveform



Week 1 Labs

f. Observations

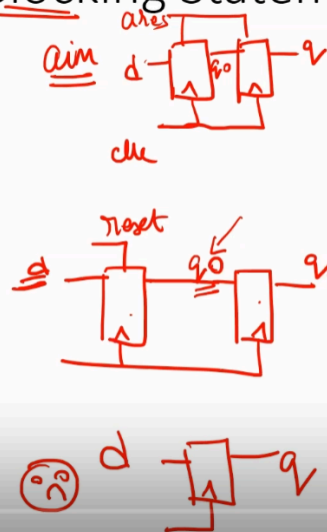
- Simulation and Synthesis Mismatch
- In the combinational block, the use of blocking assignments (=) caused a simulation vs synthesis mismatch. Specifically, d was assigned as $d = x \& c$; before $x = a \mid b$; was updated. In RTL simulation, blocking assignments execute sequentially, so d used the previous value of x , resulting in a one-step lag. In synthesis, however, all right-hand-side signals are evaluated concurrently, so d depends on the new value of x , producing a different result.
- Lesson: When using blocking assignments in combinational logic, always assign intermediate variables before using them, or avoid intermediate variables entirely. This ensures consistency between simulation and synthesized hardware.

Caveats with Blocking Statements

```

module code (input clk, input reset,
input d,
output reg q);
reg q0;
always @(posedge clk, posedge reset)
begin
  If (reset)
  begin
    q0 = 1'b0;
    q = 1'b0;
  end
  else
  begin
    q = q0;
    q0 = d;
  end
end
endmodule

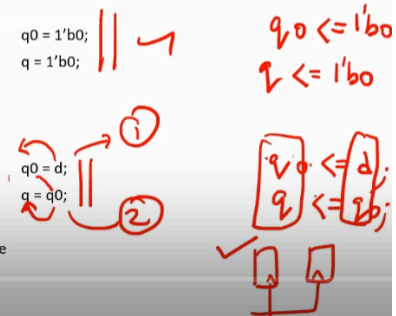
```



```

module code (input clk, input reset,
input d,
output reg q);
reg q0;
always @(posedge clk, posedge reset)
begin
  If (reset)
  begin
    q0 = 1'b0;
    q = 1'b0;
  end
  else
  begin
    q0 = d;
    q = q0;
  end
end
endmodule

```



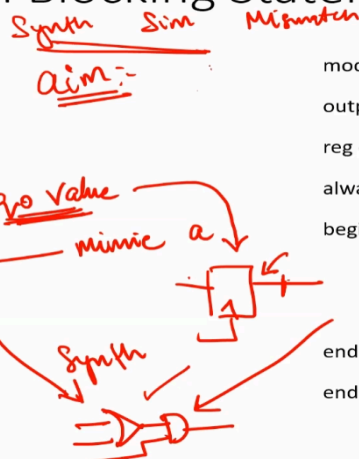
Use non blocking
for writing
seq. ckt

Caveats with Blocking Statements

```

module code (input a,b,c
output reg y);
reg q0;
always @(*)
begin
  y = q0 & c;
  q0 = a | b;
end
endmodule

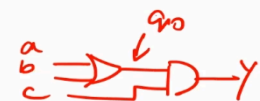
```



```

module code (input a,b,c
output reg y);
reg q0;
always @(*)
begin
  q0 = a | b;
  y = q0 & c;
end
endmodule

```



Latent value of q0
is used in Sim

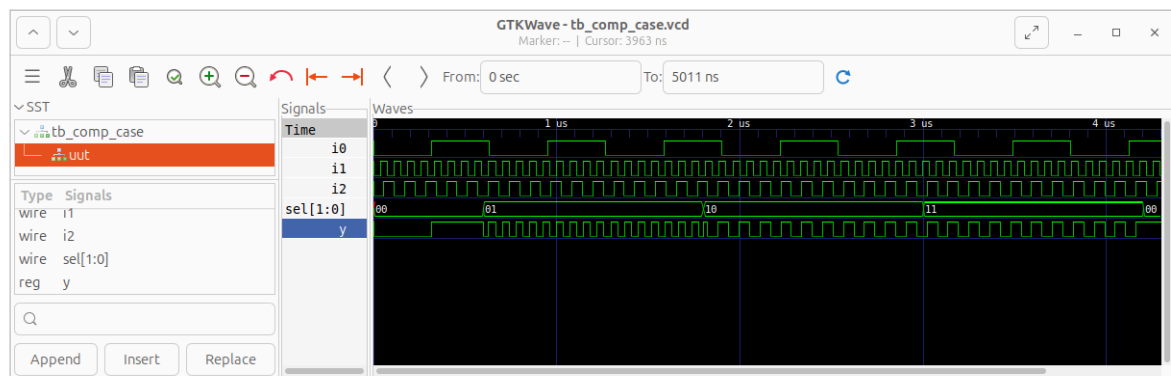
Week 1 Labs

12. Design Name - comp_case

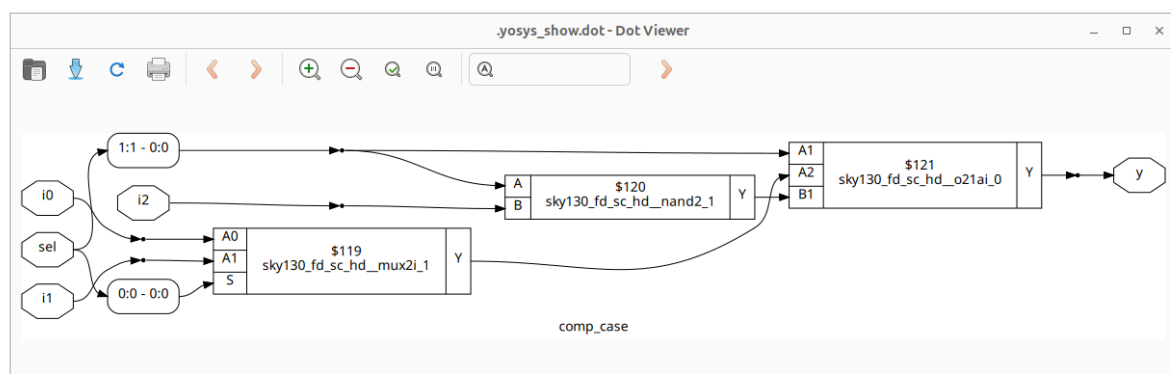
a. Design code

```
comp_case.v (~vsd/VLSI/sky130RTLDesignAndSynthesisWorkshop/verilog_files) - GVIM
File Edit Tools Syntax Buffers Window Help
1
2
3 module comp_case (input i0 , input i1 , input i2 , input [1:0] sel, output reg y);
4 always @ (*)
5 begin
6     case(sel)
7         2'b00 : y = i0;
8         2'b01 : y = i1;
9         default : y = i2;
10    endcase
11 end
12 endmodule
12 lines yanked into "+ 1,0-1 All
```

b. RTL level simulation and waveform

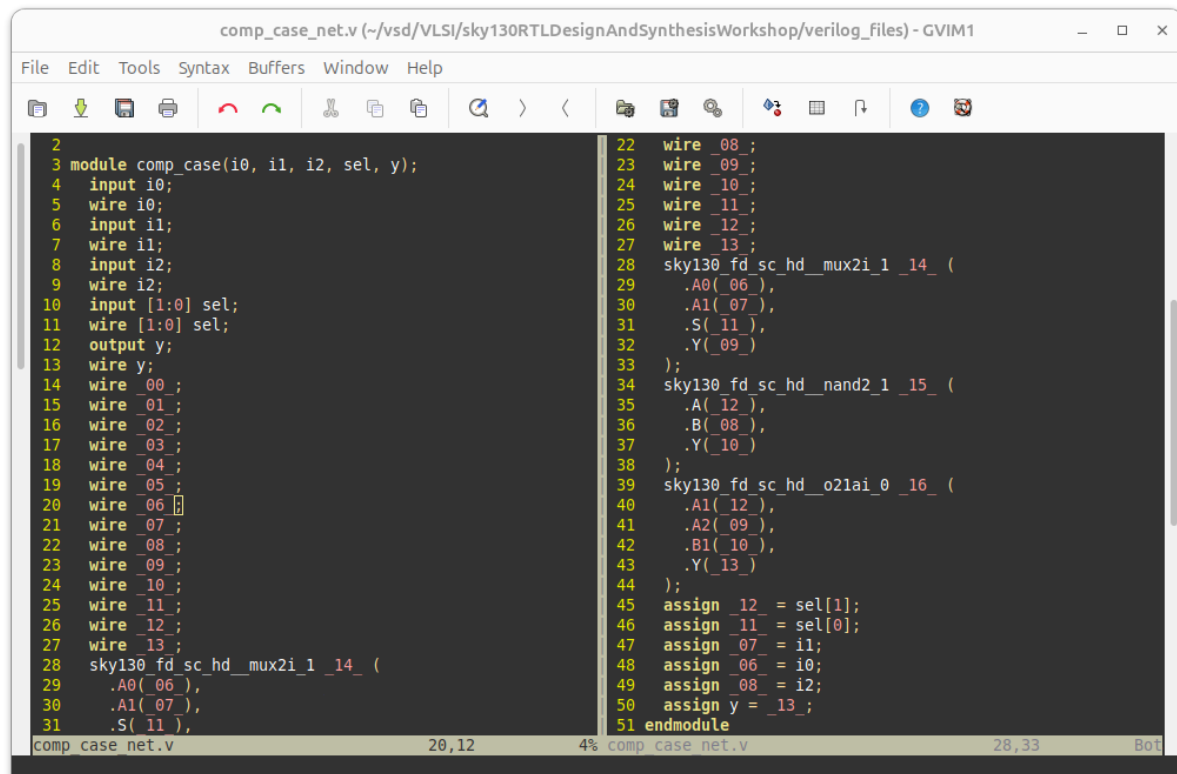


c. Synthesis Diagram



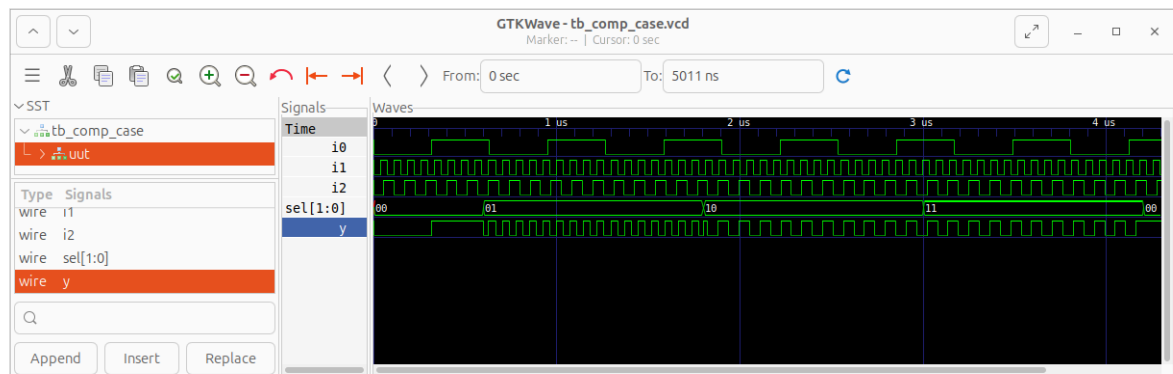
Week 1 Labs

d. Netlist generation Netlist code



```
2 module comp_case(i0, i1, i2, sel, y);
3   input i0;
4   wire i0;
5   input i1;
6   wire i1;
7   input i2;
8   wire i2;
9   input [1:0] sel;
10  wire [1:0] sel;
11  output y;
12  wire y;
13  wire _00;
14  wire _01;
15  wire _02;
16  wire _03;
17  wire _04;
18  wire _05;
19  wire _06;
20  wire _07;
21  wire _08;
22  wire _09;
23  wire _10;
24  wire _11;
25  wire _12;
26  wire _13;
27  sky130_fd_sc_hd_mux2i_1 _14_ (
28    .A0(_06),
29    .A1(_07),
30    .S(_11),
31    .Y(_09)
32  );
33  sky130_fd_sc_hd_nand2_1 _15_ (
34    .A(_12),
35    .B(_08),
36    .Y(_10)
37  );
38  sky130_fd_sc_hd_o2iai_0 _16_ (
39    .A1(_12),
40    .A2(_09),
41    .B1(_10),
42    .Y(_13)
43  );
44  assign _12 = sel[1];
45  assign _11 = sel[0];
46  assign _07 = i1;
47  assign _06 = i0;
48  assign _08 = i2;
49  assign y = _13;
50 endmodule
```

e. GLS simulation and waveform



f. Observations

- In the `comp_case` module, the 2-bit selector `sel` is fully covered using explicit case items (`2'b00`, `2'b01`) and a `default` branch for all remaining values. This ensures that every possible input combination assigns `y`, preventing inferred latches. As a result, simulation and synthesized hardware produce identical behavior, with no RTL vs synthesis mismatch.
- Lesson: Using a `default` branch (or fully specifying all cases) in combinational case statements guarantees predictable and synthesizable logic.

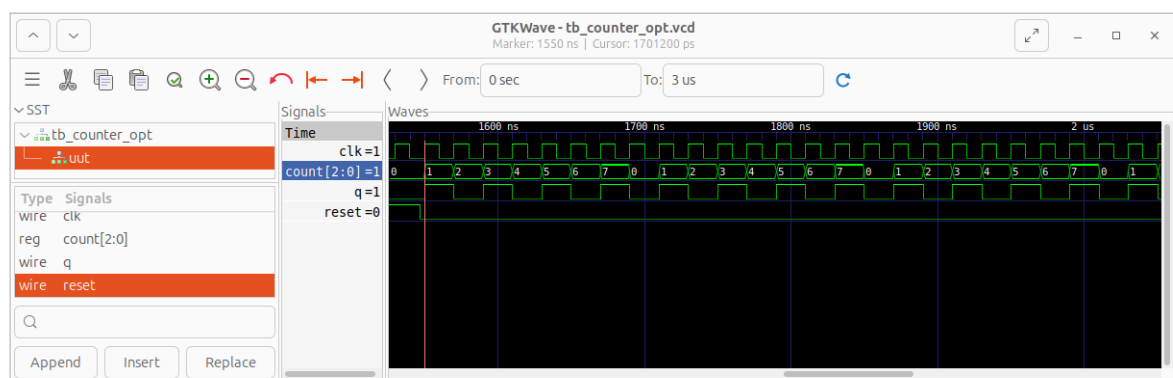
Week 1 Labs

13. Design Name - counter_opt

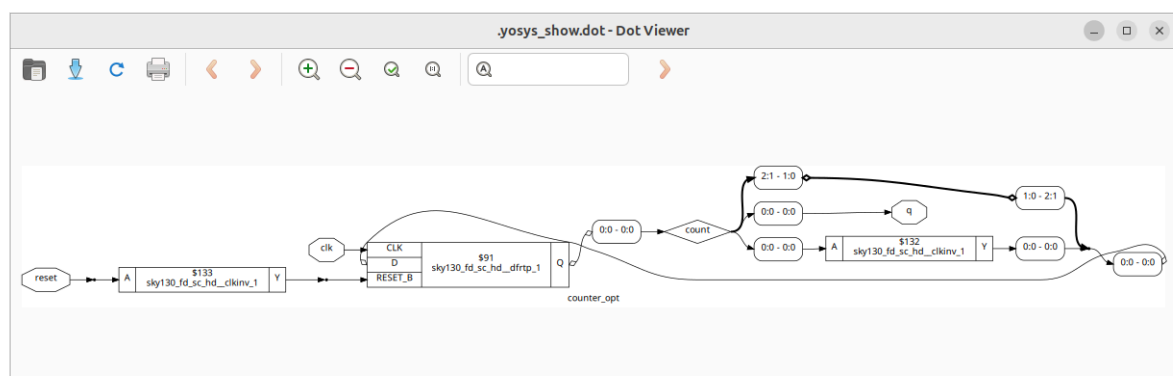
a. Design code

```
counter_opt.v (~/vsd/VLSI/sky130RTLDesignAndSynthesisWorkshop/verilog_files) - GVIM
File Edit Tools Syntax Buffers Window Help
1 module counter_opt (input clk , input reset , output q);
2 reg [2:0] count;
3 assign q = count[0];
4
5 always @(posedge clk ,posedge reset)
6 begin
7     if(reset)
8         count <= 3'b000;
9     else
10        count <= count + 1;
11 end
12
13 endmodule
13 lines yanked into "+ 1,1 All
```

b. RTL level simulation and waveform



c. Synthesis Diagram

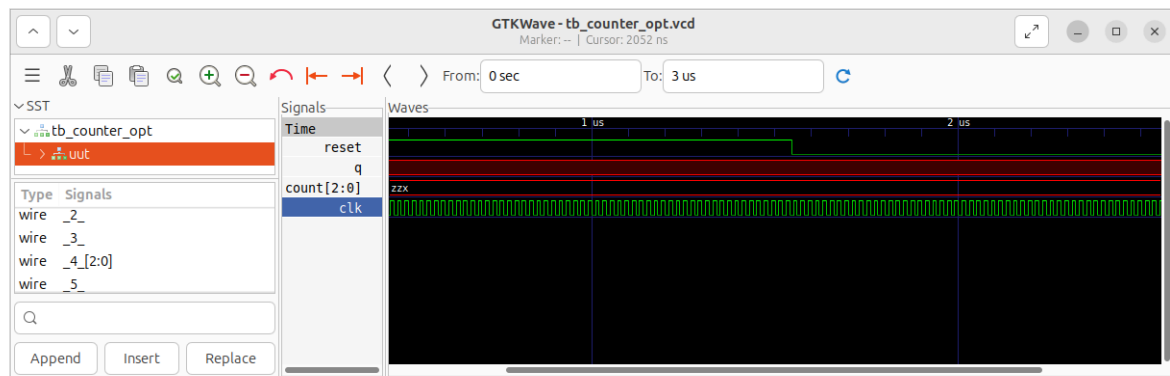


Week 1 Labs

d. Netlist generation Netlist code

```
counter_opt_net.v (~vstd/VLSI/sky130RTLDesignAndSynthesisWorkshop/verilog_files) - GVIM1
File Edit Tools Syntax Buffers Window Help
1 /* Generated by Yosys 0.57+148 (git sha1 259bd6fb3, g++ 13.3.0-6ubuntu2~24.04 -fPIC -O3) */
2
3 module counter_opt(clk, reset, q);
4   input clk;
5   wire clk;
6   input reset;
7   wire reset;
8   output q;
9   wire q;
10  wire _0_;
11  wire _1_;
12  wire _2_;
13  wire _3_;
14  wire [2:0] _4_;
15  wire _5_;
16  wire [2:0] count;
17  sky130_fd_sc_hd_clkinv_1_6 (
18    .A(_2_),
19    .Y(_0_)
20  );
21  sky130_fd_sc_hd_clkinv_1_7 (
22    .A(_3_),
23    .Y(_1_)
24  );
25  sky130_fd_sc_hd_dfrtp_1_8 (
26    .CLK(clk),
27    .D(_4_[0]),
28    .Q(count[0]),
29    .RESET_B(_5_)
30  );
31  assign _4_[2:1] = count[2:1];
32  assign q = count[0];
33  assign _2_ = count[0];
34  assign _4_[0] = _0_;
35  assign _3_ = reset;
36  assign _5_ = _1_;
37 endmodule
```

e. GLS simulation and waveform



f. Observations

- In the `counter_opt` module, the output `q` is assigned as the least significant bit of a 3-bit counter (`q = count[0]`). During RTL simulation, `q` toggles every clock cycle as expected. However, after synthesis, `q` may behave differently. This mismatch occurs because synthesis tools can optimize the multi-bit counter when only the LSB is used, and asynchronous reset propagation or internal carry logic may not match simulation timing.
- Lesson: When only part of a multi-bit counter is used, either implement the output directly as a T flip-flop or ensure full initialization and proper reset handling to guarantee RTL-synthesized consistency.

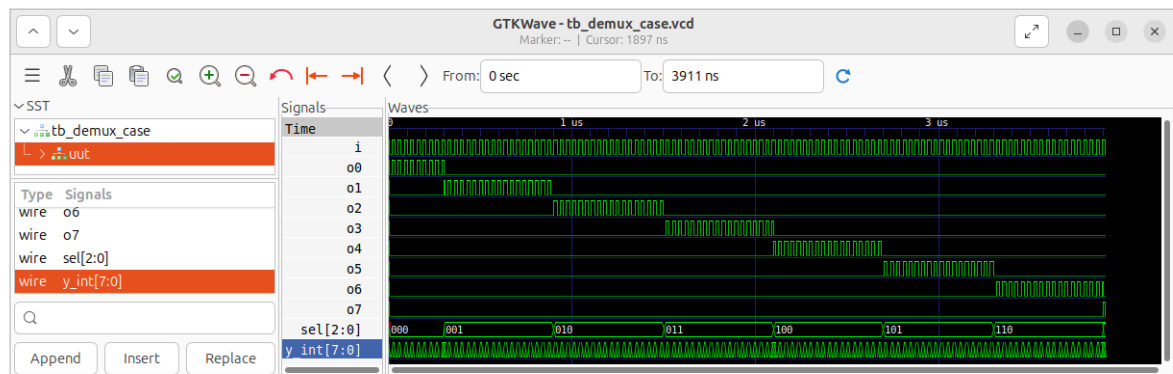
Week 1 Labs

14. Design Name - demux_case

a. Design code

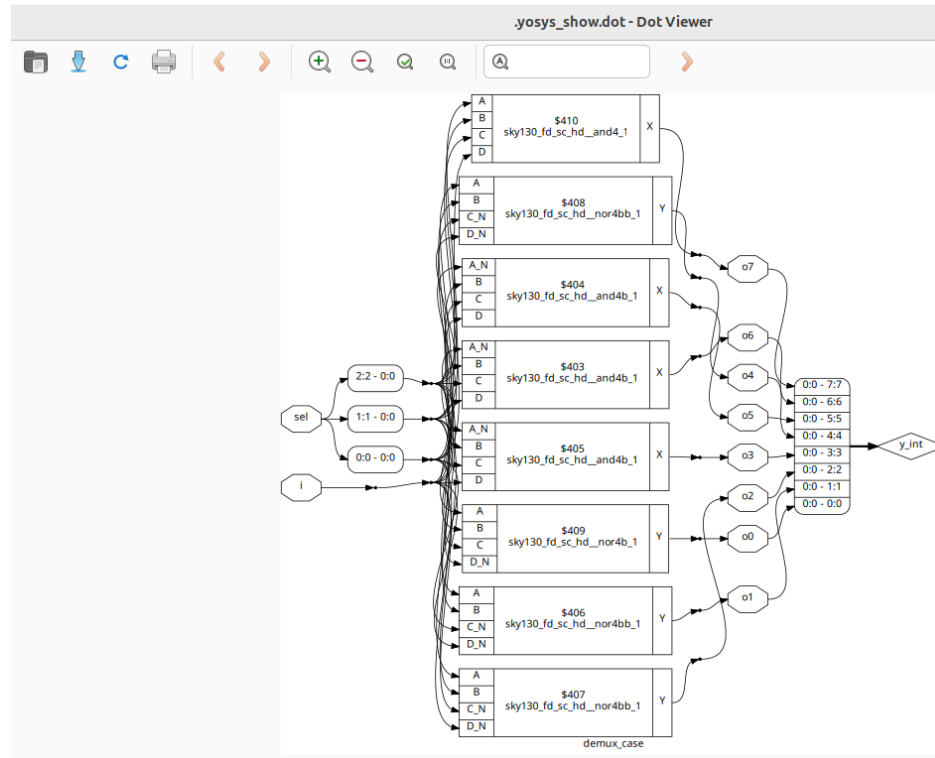
```
demux_case.v (~vsd/VLSI/sky130RTLDesignAndSynthesisWorkshop/verilog_files) - GVIM
File Edit Tools Syntax Buffers Window Help
[Icons]
1 module demux_case (output o0 , output o1, output o2 , output o3, output o4, output o5, output o6 , output o7 , input
  [2:0] sel , input i);
2 reg [7:0] y_int;
3 assign {o7,o6,o5,o4,o3,o2,o1,o0} = y_int;
4 integer k;
5 always @ (*)
6 begin
7   y_int = 8'b0;
8   case(sel)
9     3'b000 : y_int[0] = i;
10    3'b001 : y_int[1] = i;
11    3'b010 : y_int[2] = i;
12    3'b011 : y_int[3] = i;
13    3'b100 : y_int[4] = i;
14    3'b101 : y_int[5] = i;
15    3'b110 : y_int[6] = i;
16    3'b111 : y_int[7] = i;
17   endcase
18 end
19 end
20 endmodule
21
22
22 lines yanked into "+ 1,1 All
```

b. RTL level simulation and waveform



Week 1 Labs

c. Synthesis Diagram



d. Netlist generation Netlist code

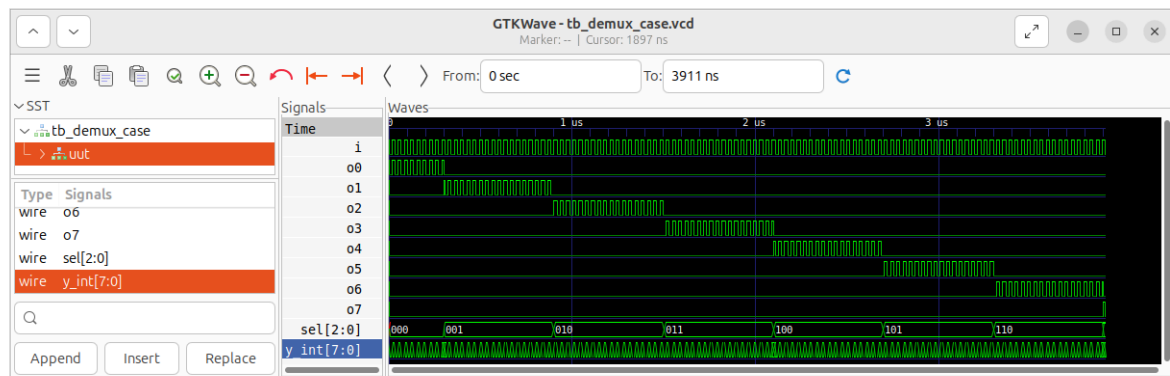
```
demux_case_net.v (~vsd/VLSI/sky130RTLDesignAndSynthesisWorkshop/verilog_files) - GVIM1
File Edit Tools Syntax Buffers Window Help
[Icons]

2 module demux_case(o0, o1, o2, o3, o4, o5, o6, o7, sel, i);
3   , o4, o5, o6, o7, sel, i);
4   output o0;
5   wire o0;
6   output o1;
7   wire o1;
8   output o2;
9   wire o2;
10  output o3;
11  wire o3;
12  output o4;
13  wire o4;
14  output o5;
15  wire o5;
16  output o6;
17  wire o6;
18  output o7;
19  wire o7;
20  input [2:0] sel;
21  wire [2:0] sel;
22  input i;
23  wire i;
24  wire o0;
25  wire o1;
26  wire o2;
27  wire o3;
28  wire o4;
29  wire o5;
30  wire o6;
31  wire o7;
32  wire o8;
33  wire o9;
34  wire o10;
35  wire o11;
36  wire o12;
37  wire 14;
38  wire 15;
39  wire 16;
40  wire 17;
41  wire 18;
42  wire 19;
43  wire 20;
44  wire 21;
45  wire 22;
46  wire 23;
47  wire 24;
48  wire 25;
49  wire 26;
50  wire 27;
51  wire 28;
52  wire 29;
53  wire 30;
54  wire 31;
55  wire 32;
56  wire 33;
57  wire 34;
58  wire 35;
59  wire 36;
60  wire 37;
61  wire 38;
62  wire 39;
63  wire 40;
64  wire 41;
65  wire 42;
66  wire 43;
67  wire 44;
68  wire 45;
69  wire 46;
70  wire 47;
71  wire [7:0] y_int;
72  sky130_fd_sc_hd_and4b_1_47_
73  (
74    .A( 45 ),
75    .B( 44 ),
76    .C( 35 ),
77    .D( 35 ),
78    .X( 37 )
79  );
80  sky130_fd_sc_hd_and4b_1_48_
81  (
82    .A( 45 ),
83    .B( 44 ),
84    .C( 35 ),
85    .D( 35 ),
86    .X( 41 )
87  );
88  sky130_fd_sc_hd_and4b_1_49_
89  (
90    .A( 46 ),
91    .B( 45 ),
92    .C( 44 ),
93    .D( 35 ),
94    .X( 39 )
95  );
96  sky130_fd_sc_hd_nor4bb_1_50_
97  (
98    .A( 46 ),
99    .B( 45 ),
100   .C( 44 ),
101   .D( 35 ),
102   .Y( 37 )
103  );
104  sky130_fd_sc_hd_nor4bb_1_51_
105  (
106    .A( 46 ),
107    .B( 44 ),
108    .C( 35 ),
109    .D( 45 ),
110    .Y( 38 )
111  );
112  sky130_fd_sc_hd_nor4bb_1_52_
113  (
114    .A( 45 ),
115    .B( 44 ),
116    .C( 35 ),
117    .D( 46 ),
118    .Y( 40 )
119  );
120  sky130_fd_sc_hd_nor4bb_1_53_
121  (
122    .A( 46 ),
123    .B( 45 ),
124    .C( 44 ),
125    .D( 35 ),
126    .Y( 36 )
127  );
128  assign y_int = { o7, o6, o5, o4, o3, o2, o1, o0 };
129  assign _46 = sel[2];
130  assign _45 = sel[1];
131  assign _44 = sel[0];
132  assign _35 = i;
133  assign o6 = 42;
134  assign o5 = 41;
135  assign o3 = 39;
136  assign o1 = 37;
137  assign o2 = 38;
138  assign o4 = 40;
139  assign o8 = 36;
140  assign o7 = 43;
141  endmodule
142

demux_case_net.v 19,1 0% demux_case_net.v 73,18 34% demux_case_net.v 188,13 70% demux_case_net.v 141,9 Bot
```

Week 1 Labs

e. GLS simulation and waveform



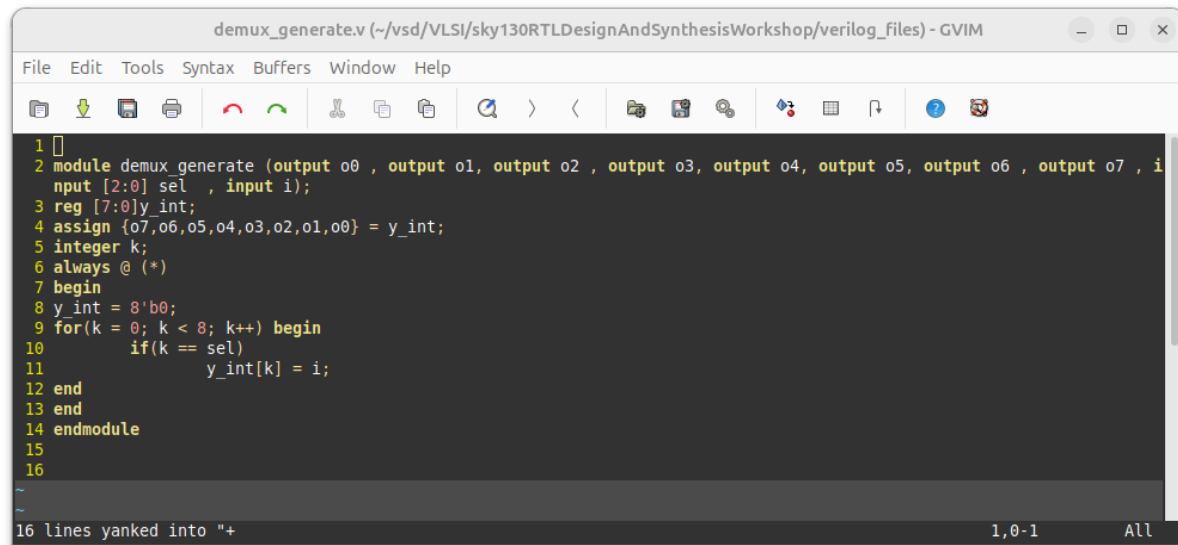
f. Observations

- i. The `demux_case` module implements a 3-to-8 demultiplexer using a combinational case statement. An 8-bit internal register `y_int` is cleared at the start of the always block, and only the selected bit corresponding to `sel` is assigned the input `i`. Outputs are driven from `y_int`. This ensures fully defined combinational logic with no inferred latches. RTL simulation matches synthesized hardware exactly, producing predictable one-hot outputs for all selector values.

Week 1 Labs

15. Design Name - demux_generator

a. Design code

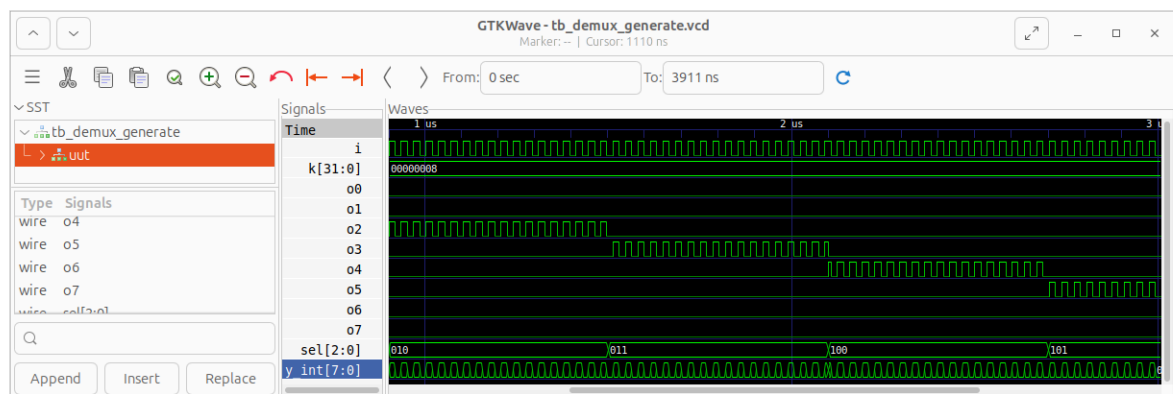


The screenshot shows a Verilog code editor window titled "demux_generate.v (~/.vsd/VLSI/sky130RTLDesignAndSynthesisWorkshop/verilog_files) - GVIM". The code defines a module "demux_generate" with inputs "o0", "o1", "o2", "o3", "o4", "o5", "o6", "o7", and "i". It includes a register "y_int" of size 8, an integer "k", and an "always" block that updates "y_int" based on the input "i" and a selection signal "sel". The code is as follows:

```
1 []
2 module demux_generate (output o0 , output o1, output o2 , output o3, output o4, output o5, output o6 , output o7 , i
3   nput [2:0] sel , input i);
4   reg [7:0]y_int;
5   assign {o7,o6,o5,o4,o3,o2,o1,o0} = y_int;
6   integer k;
7   always @ (*)
8   begin
9     y_int = 8'b0;
10    for(k = 0; k < 8; k++) begin
11      if(k == sel)
12        y_int[k] = i;
13    end
14  end
15 endmodule
16
```

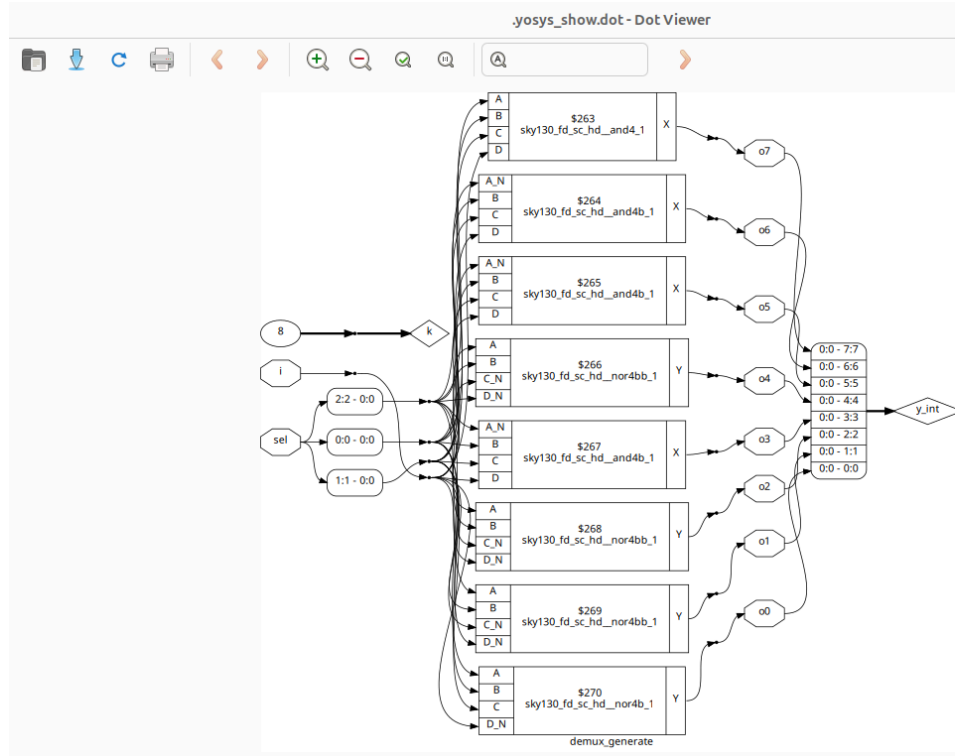
The status bar at the bottom indicates "16 lines yanked into "+ and "1,0-1 All".

b. RTL level simulation and waveform



Week 1 Labs

c. Synthesis Diagram



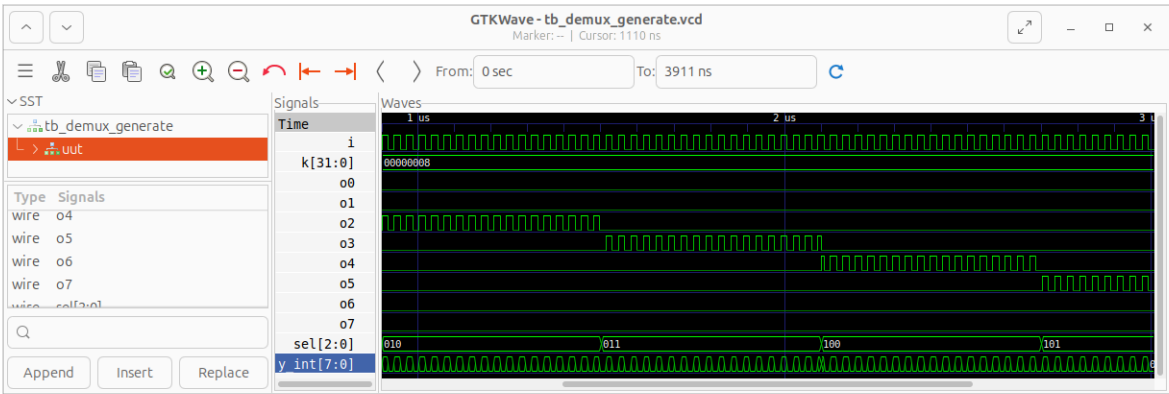
d. Netlist generation Netlist code

Diagram showing the netlist generation code for the `demux_generate` module. The code is displayed in the `demux_generate_net.v` file, which is part of the `sky130RTLDesignAndSynthesisWorkshop` project. The code defines the module `demux_generate` and its internal logic, including the generation of the netlist.

```
1 /* Generated by Yosys 0.57+148 (36 wire 12; 76 .D N( 24 ), 89 .C N( 23 ),
2 git sha1 259bd6fb3, g++ 13.3.0-6 37 wire 13; 77 .Y( 18 ) 90 .D N( 13 ),
3 ubuntu2-24.04 -fPIC -O3) */ 38 wire 14; 78 ); 91 .Y( 16 )
4 module demux_generate(o0, o1, o2 39 wire 15; 79 sky130_fd_sc_hd_and4b_1 29_ 92 );
5 , o3, o4, o5, o6, o7, sel, i); 40 wire 16; 80 .A N( 24 ), 93 sky130_fd_sc_hd_nor4bb_1 31_
6 output o0; 41 wire 17; 81 .B( 22 ), 94 .A( 24 ),
7 wire o1; 42 wire 18; 82 .C( 23 ), 95 .B( 23 ),
8 output o2; 43 wire 19; 83 .D( 13 ), 96 .C( 23 ),
9 wire o3; 44 wire 20; 84 .X( 17 ) 97 .D N( 22 ),
10 output o4; 45 wire 21; 85 ); 98 .Y( 15 )
11 wire o5; 46 wire 22; 86 sky130_fd_sc_hd_nor4bb_1 30_ 99 );
12 output o6; 47 wire 23; 87 .A( 24 ), 100 sky130_fd_sc_hd_nor4b_1 32_
13 wire o7; 48 wire 24; 88 .B( 22 ), 101 .A( 24 ),
14 input [2:0] sel; 49 wire [31:0] k; 89 .C N( 23 ), 102 .B( 22 ),
15 wire [2:0] sel; 50 wire [7:0] y_int; 90 .D N( 13 ), 103 .C( 23 ),
16 wire i; 51 sky130_fd_sc_hd_and4_1 25_ ( 91 .Y( 16 ) 104 .D N( 13 ),
17 wire 0; 52 .A( 24 ), 92 ); 105 .Y( 14 )
18 wire 1; 53 .B( 22 ), 93 sky130_fd_sc_hd_nor4bb_1 31_ 106 );
19 wire 2; 54 .C( 23 ), 94 .A( 24 ), 107 assign k = 32'd8;
20 wire 3; 55 .D( 13 ), 95 .B( 23 ), 108 assign y_int = { o7, o6, o5, o
21 wire 4; 56 .X( 21 ) 96 .C N( 13 ), 109 assign _24 = sel[2];
22 wire 5; 57 ); 97 .D N( 22 ), 110 assign _22 = sel[0];
23 wire 6; 58 sky130_fd_sc_hd_and4b_1 26_ 98 .Y( 15 ) 111 assign _23 = sel[1];
24 wire 7; 59 .A N( 22 ), 99 ); 112 assign _13 = i;
25 wire 8; 60 .B( 23 ), 100 sky130_fd_sc_hd_nor4b_1 32_ 113 assign o7 = 21;
26 wire 9; 61 .C( 13 ), 101 .A( 24 ), 114 assign o6 = 20;
27 wire 10; 62 .D( 24 ), 102 .B( 22 ), 115 assign o5 = 19;
28 wire 11; 63 .X( 20 ) 103 .C( 23 ), 116 assign o4 = 18;
29 wire 12; 64 ); 104 .D N( 13 ), 117 assign o3 = 17;
30 wire 13; 65 sky130_fd_sc_hd_and4b_1 27_ 105 .Y( 14 ) 118 assign o2 = 16;
31 wire 14; 66 .A N( 23 ), 106 ); 119 assign o1 = 15;
32 wire 15; 67 .B( 13 ), 107 assign k = 32'd8; 120 assign o0 = 14;
33 wire 16; 68 .C( 24 ), 121 endmodule
34 wire 17; 69 .D( 22 ), 122
```

Week 1 Labs

e. GLS simulation and waveform



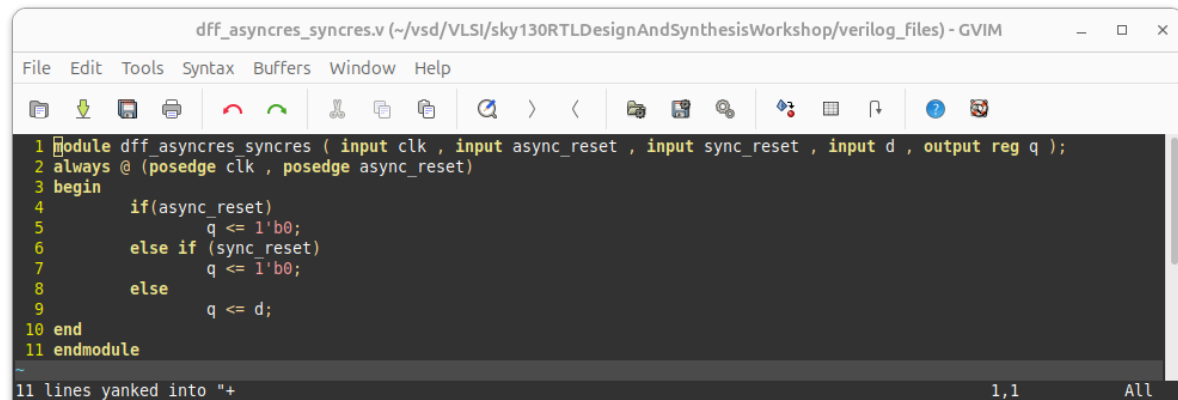
f. Observations

i.

Week 1 Labs

16. Design Name - dff_asyncres_syncres

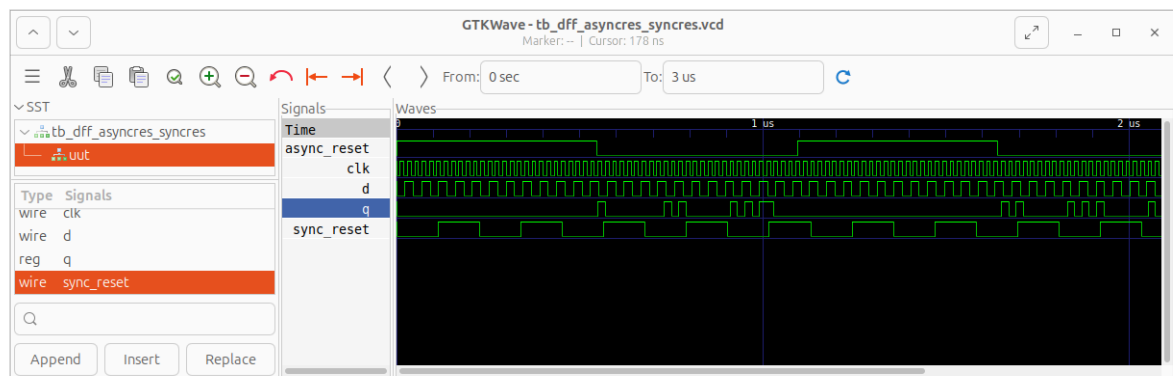
a. Design code



```
1 module dff_asyncres_syncres ( input clk , input async_reset , input sync_reset , input d , output reg q );
2 always @ (posedge clk , posedge async_reset)
3 begin
4     if(async_reset)
5         q <= 1'b0;
6     else if (sync_reset)
7         q <= 1'b0;
8     else
9         q <= d;
10 end
11 endmodule
```

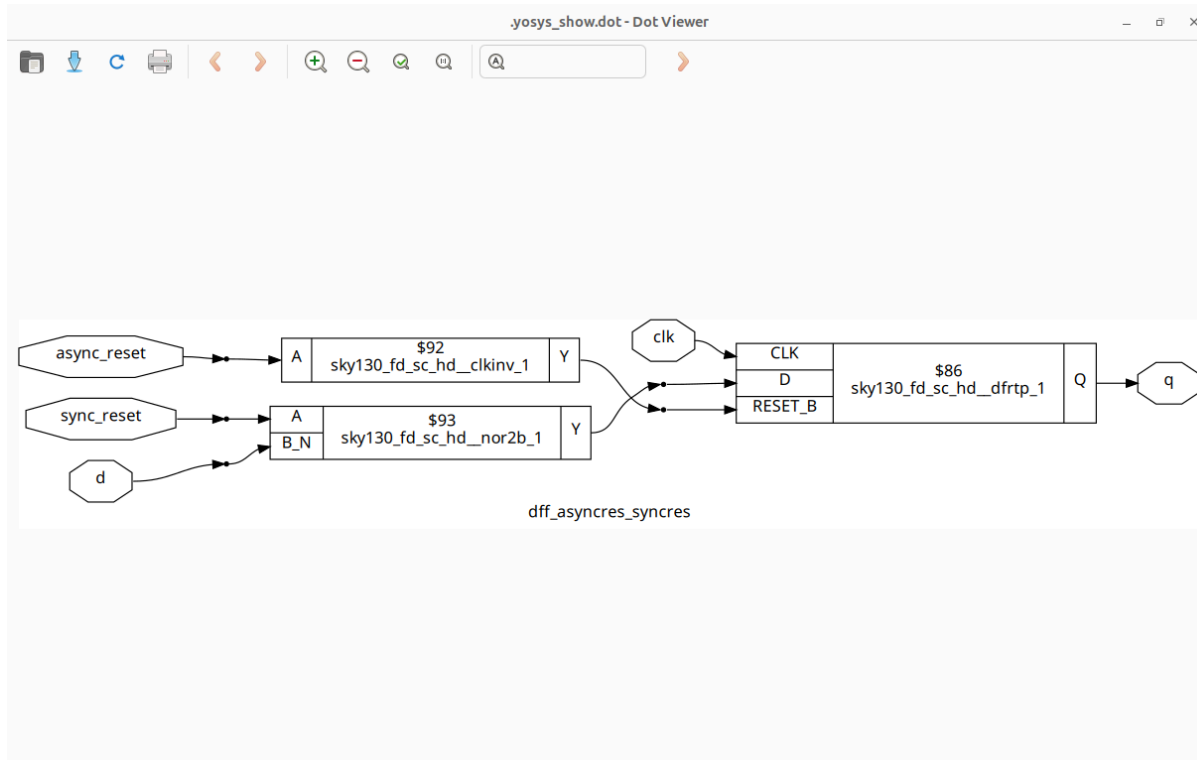
11 lines yanked into "+ 1,1 All

b. RTL level simulation and waveform



Week 1 Labs

c. Synthesis Diagram



d. Netlist generation

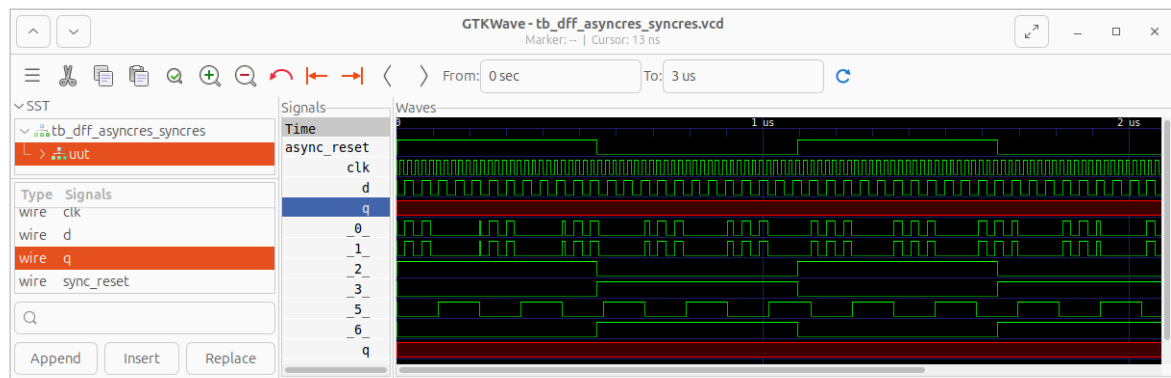
dff_asyncres_syncres_net.v (~vsd/VLSI/sky130RTLDesignAndSynthesisWorkshop/verilog_files) - GVIM1

```
1 /* Generated by Yosys 0.57+148 (git sha1 259bd6fb3, g++ 13.3.0-6ubuntu
2 ~24.04 -fPIC -O3) */
3 module dff_asyncres_syncres(clk, async_reset, sync_reset, d, q);
4   input clk;
5   wire clk;
6   input async_reset;
7   wire async_reset;
8   input sync_reset;
9   wire sync_reset;
10  input d;
11  wire d;
12  output q;
13  wire q;
14  wire _0_;
15  wire _1_;
16  wire _2_;
17  wire _3_;
18  wire _4_;
19  wire _5_;
20  wire _6_;
21  sky130_fd_sc_hd_clkinv_1 _7_ (
22    .A(_2_),
23    .Y(_3_)
24  );
25  sky130_fd_sc_hd_nor2b_1 _8_ (
26    .A(_5_),
27    .B_N(_4_),
28    .Y(_1_)
29  );
30  sky130_fd_sc_hd_dfrtp_1 _9_ (
31    .CLK(clk),
32    .D(_0_),
33    .Q(q),
34    .RESET_B(_6_)
35  );
36  assign _4_ = d;
37  assign _5_ = sync_reset;
38  assign _0_ = _1_;
39  assign _2_ = async_reset;
40  assign _6_ = _3_;
41 endmodule
```

dff_asyncres_syncres_net.v 1,1 Top dff_asyncres_syncres_net.v 30,18 Bot

Week 1 Labs

e. GLS simulation and waveform



f. Debugging this!

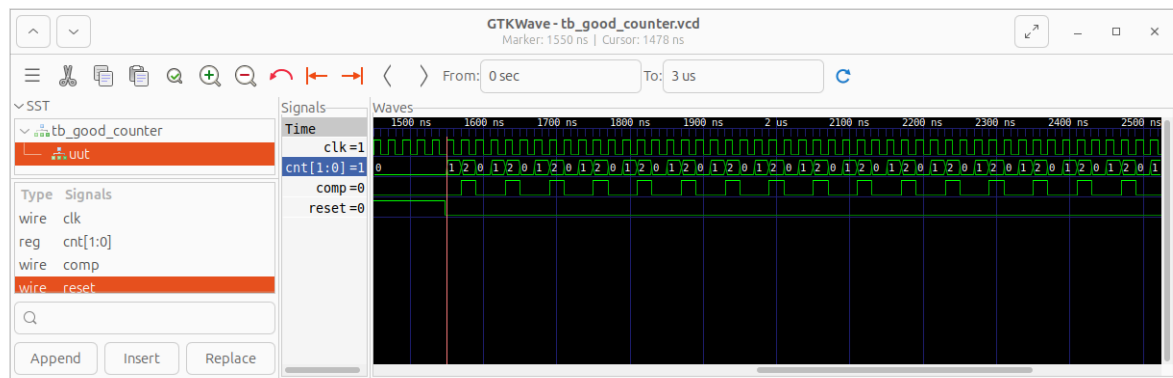
Week 1 Labs

17. Design Name - good_counter

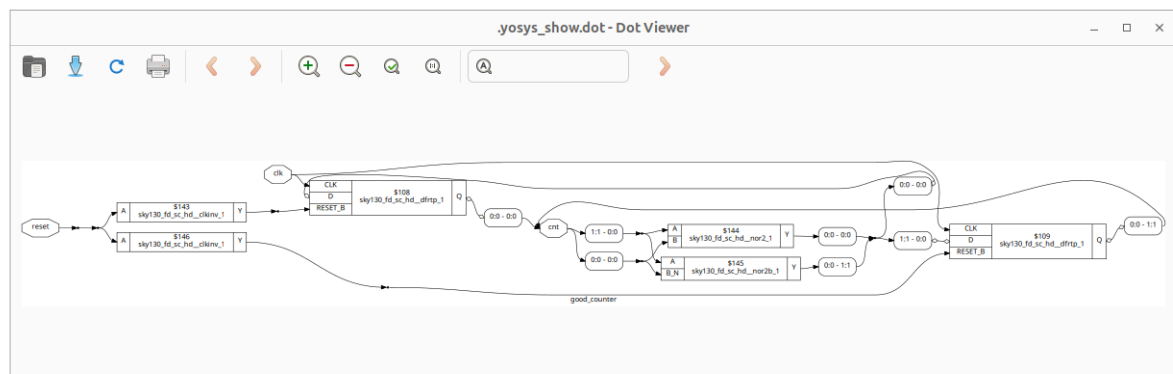
a. Design code

```
good_counter.v (~vsd/VLSI/sky130RTLDesignAndSynthesisWorkshop/verilog_files) - GVIM
File Edit Tools Syntax Buffers Window Help
1 module good_counter (input clk , input reset , output reg [1:0] cnt);
2 wire comp;
3
4 assign comp = (cnt == 2'b10);
5
6 always @(posedge clk , posedge reset)
7 begin
8     if(reset)
9         cnt <= 2'b00;
10    else if(comp)
11        cnt <= 2'b00;
12    else
13        cnt <= cnt+1;
14 end
15
16
17
18 endmodule
18 lines yanked into "+
```

b. RTL level simulation and waveform

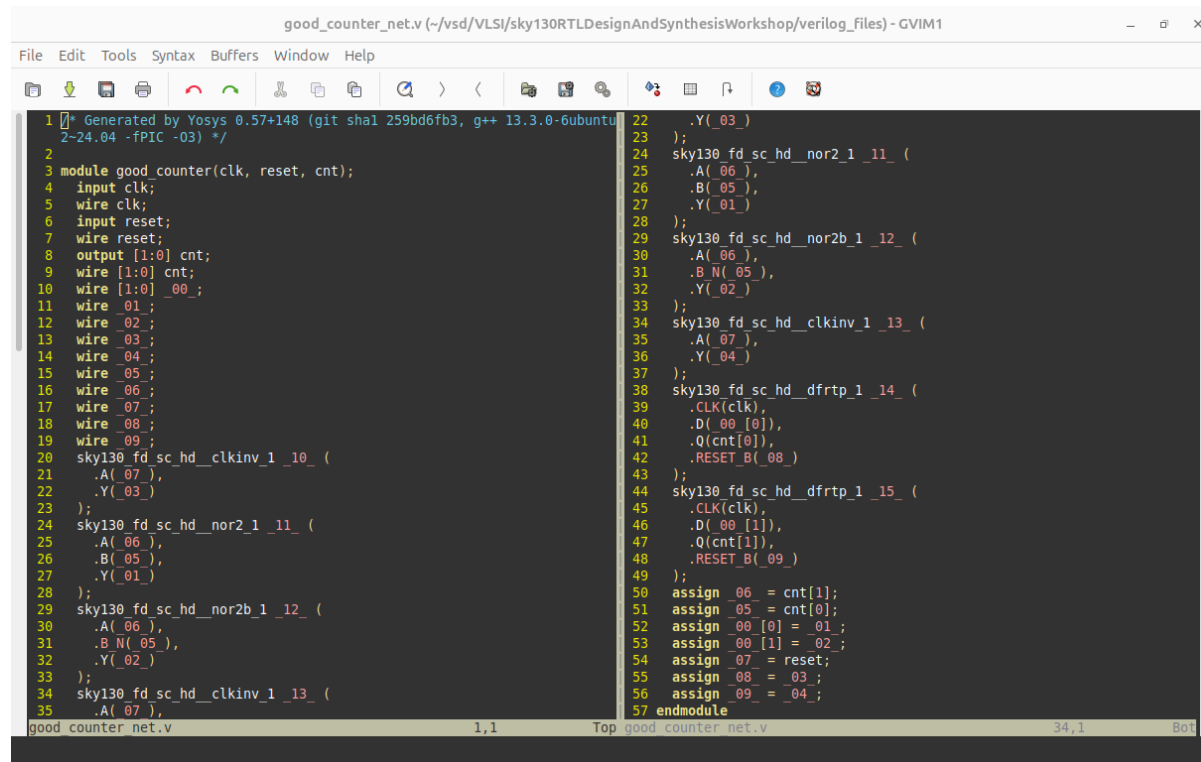


c. Synthesis Diagram



Week 1 Labs

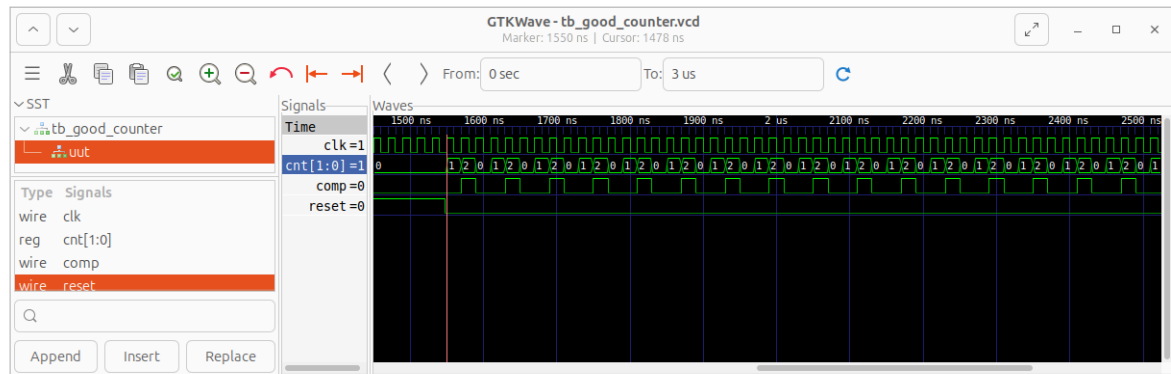
d. Netlist generation



The screenshot shows a Verilog netlist file named 'good_counter_net.v' in a text editor. The code is generated by Yosys 0.57+148 and uses a 13.3.0-6ubuntu2-24.04-fPIC compiler. The netlist defines a module 'good_counter' with inputs 'clk', 'reset', and 'cnt', and outputs 'cnt'. It uses various logic blocks like 'sky130_fd_sc_hd_nor2_1_11', 'sky130_fd_sc_hd_nor2b_1_12', 'sky130_fd_sc_hd_clkinv_1_13', and 'sky130_fd_sc_hd_dfrtp_1_14'. The code includes assignments for 'cnt[1:0]' and 'cnt[0]'.

```
1 /* Generated by Yosys 0.57+148 (git sha1 259bd6fb3, g++ 13.3.0-6ubuntu2-24.04-fPIC -O3) */
2
3 module good_counter(clk, reset, cnt);
4   input clk;
5   wire clk;
6   input reset;
7   wire reset;
8   output [1:0] cnt;
9   wire [1:0] cnt;
10  wire [1:0] _00_;
11  wire _01_;
12  wire _02_;
13  wire _03_;
14  wire _04_;
15  wire _05_;
16  wire _06_;
17  wire _07_;
18  wire _08_;
19  wire _09_;
20  sky130_fd_sc_hd_clkinv_1_10_ (
21    .A(_07_),
22    .Y(_03_)
23  );
24  sky130_fd_sc_hd_nor2_1_11_ (
25    .A(_06_),
26    .B(_05_),
27    .Y(_01_)
28  );
29  sky130_fd_sc_hd_nor2b_1_12_ (
30    .A(_06_),
31    .B_N(_05_),
32    .Y(_02_)
33  );
34  sky130_fd_sc_hd_clkinv_1_13_ (
35    .A(_07_),
36    .Y(_04_)
37  );
38  sky130_fd_sc_hd_dfrtp_1_14_ (
39    .CLK(clk),
40    .D(_00_[0]),
41    .Q(cnt[0]),
42    .RESET_B(_08_)
43  );
44  sky130_fd_sc_hd_dfrtp_1_15_ (
45    .CLK(clk),
46    .D(_00_[1]),
47    .Q(cnt[1]),
48    .RESET_B(_09_)
49  );
50  assign _06_ = cnt[1];
51  assign _05_ = cnt[0];
52  assign _00_[0] = _01_;
53  assign _00_[1] = _02_;
54  assign _07_ = reset;
55  assign _08_ = _03_;
56  assign _09_ = _04_;
57 endmodule
```

e. GLS simulation and waveform



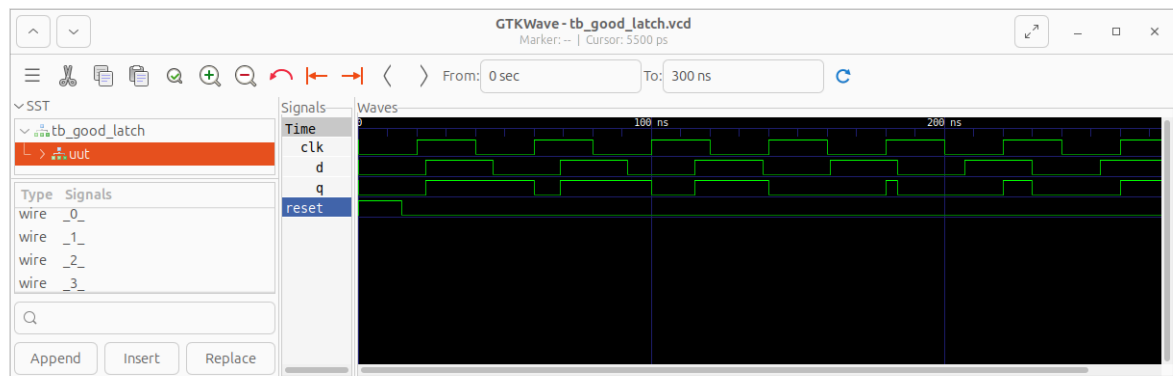
Week 1 Labs

18. Design Name good_latch

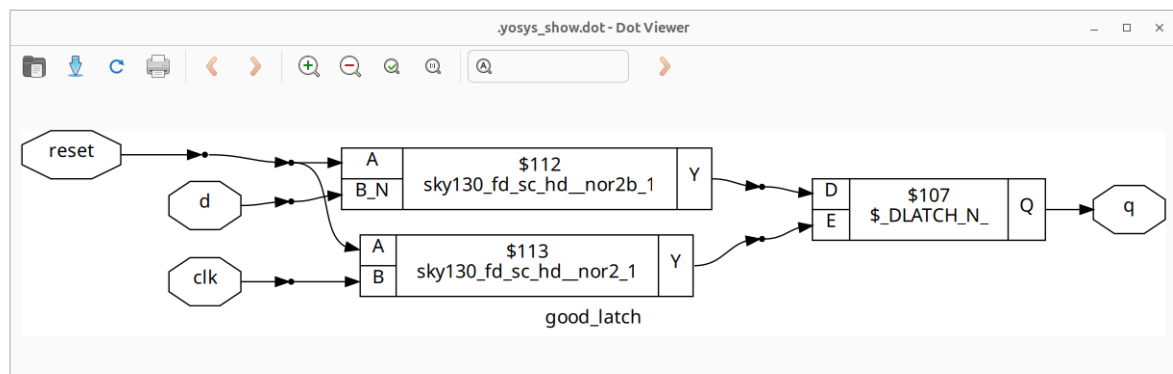
a. Design code

```
good_latch.v (~vsd/VLSI/sky130RTLDesignAndSynthesisWorkshop/verilog_files) - GVIM
File Edit Tools Syntax Buffers Window Help
1 module good_latch (input clk , input reset , input d , output reg q);
2 always @ (clk,reset,d)
3 begin
4     if(reset)
5         q <= 1'b0;
6     else if(clk)
7         q <= d;
8 end
9 endmodule
```

b. RTL level simulation and waveform

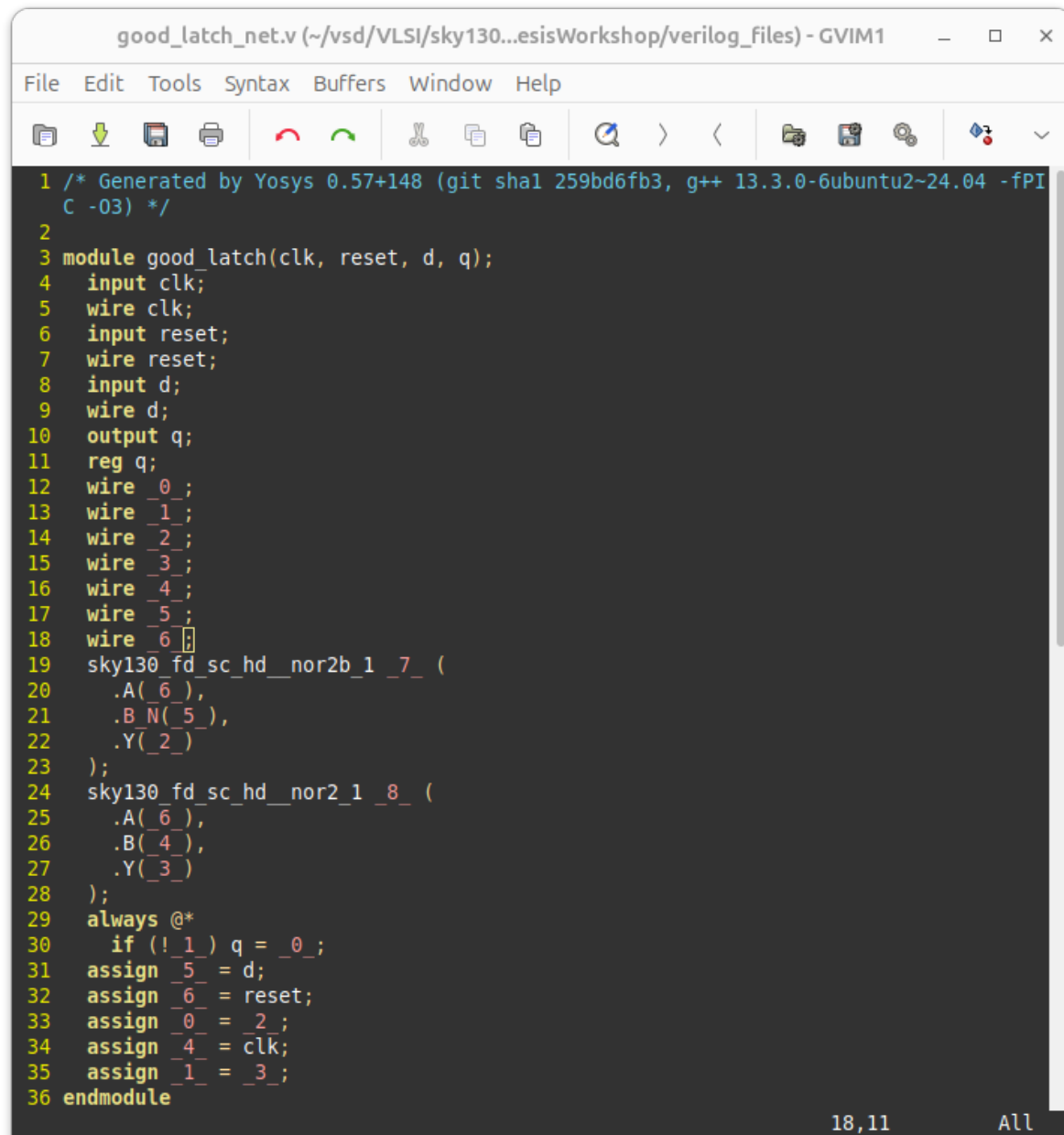


c. Synthesis Diagram



Week 1 Labs

d. Netlist generation

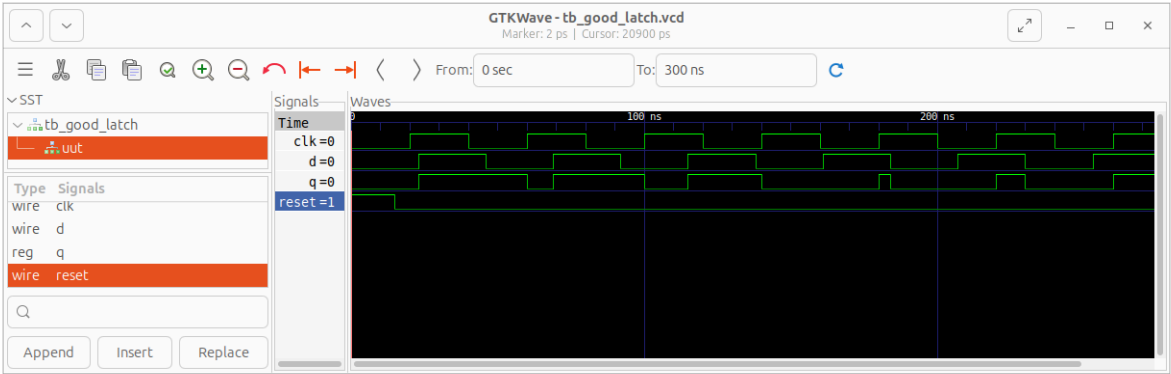


```
1 /* Generated by Yosys 0.57+148 (git sha1 259bd6fb3, g++ 13.3.0-6ubuntu2~24.04 -fPI
   C -03) */
2
3 module good_latch(clk, reset, d, q);
4   input clk;
5   wire clk;
6   input reset;
7   wire reset;
8   input d;
9   wire d;
10  output q;
11  reg q;
12  wire _0_;
13  wire _1_;
14  wire _2_;
15  wire _3_;
16  wire _4_;
17  wire _5_;
18  wire _6_;
19  sky130_fd_sc_hd__nor2_1_7_ (
20    .A(_6_),
21    .B_N(_5_),
22    .Y(_2_)
23  );
24  sky130_fd_sc_hd__nor2_1_8_ (
25    .A(_6_),
26    .B(_4_),
27    .Y(_3_)
28  );
29  always @*
30    if (!_1_) q = _0_;
31  assign _5_ = d;
32  assign _6_ = reset;
33  assign _0_ = _2_;
34  assign _4_ = clk;
35  assign _1_ = _3_;
36 endmodule
```

18,11 All

Week 1 Labs

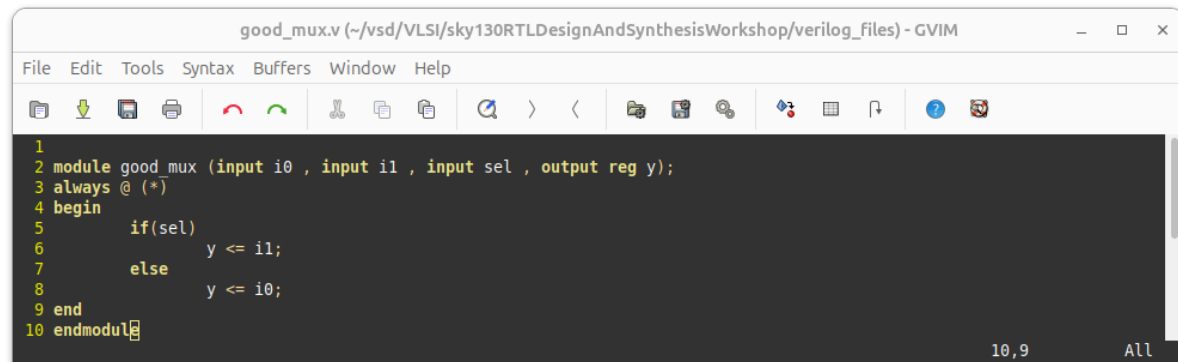
e. GLS simulation and waveform



Week 1 Labs

19. Design Name - good_mux

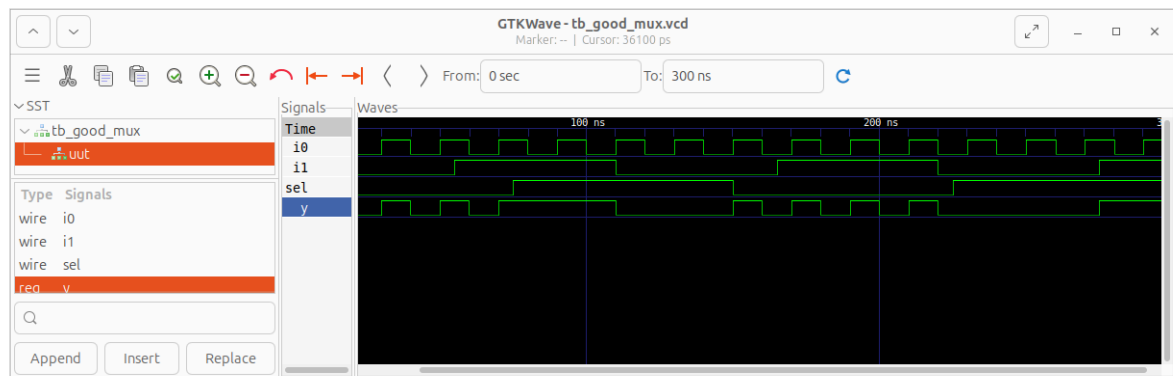
a. Design code



The screenshot shows a Verilog code editor window titled 'good_mux.v (~/\vds/VLSI/sky130RTLDesignAndSynthesisWorkshop/verilog_files) - GVIM'. The code defines a module 'good_mux' with three inputs: 'i0', 'i1', and 'sel', and one output: 'y' (a reg). The logic is as follows: if 'sel' is high, 'y' is assigned the value of 'i1'; otherwise, 'y' is assigned the value of 'i0'. The code is as follows:

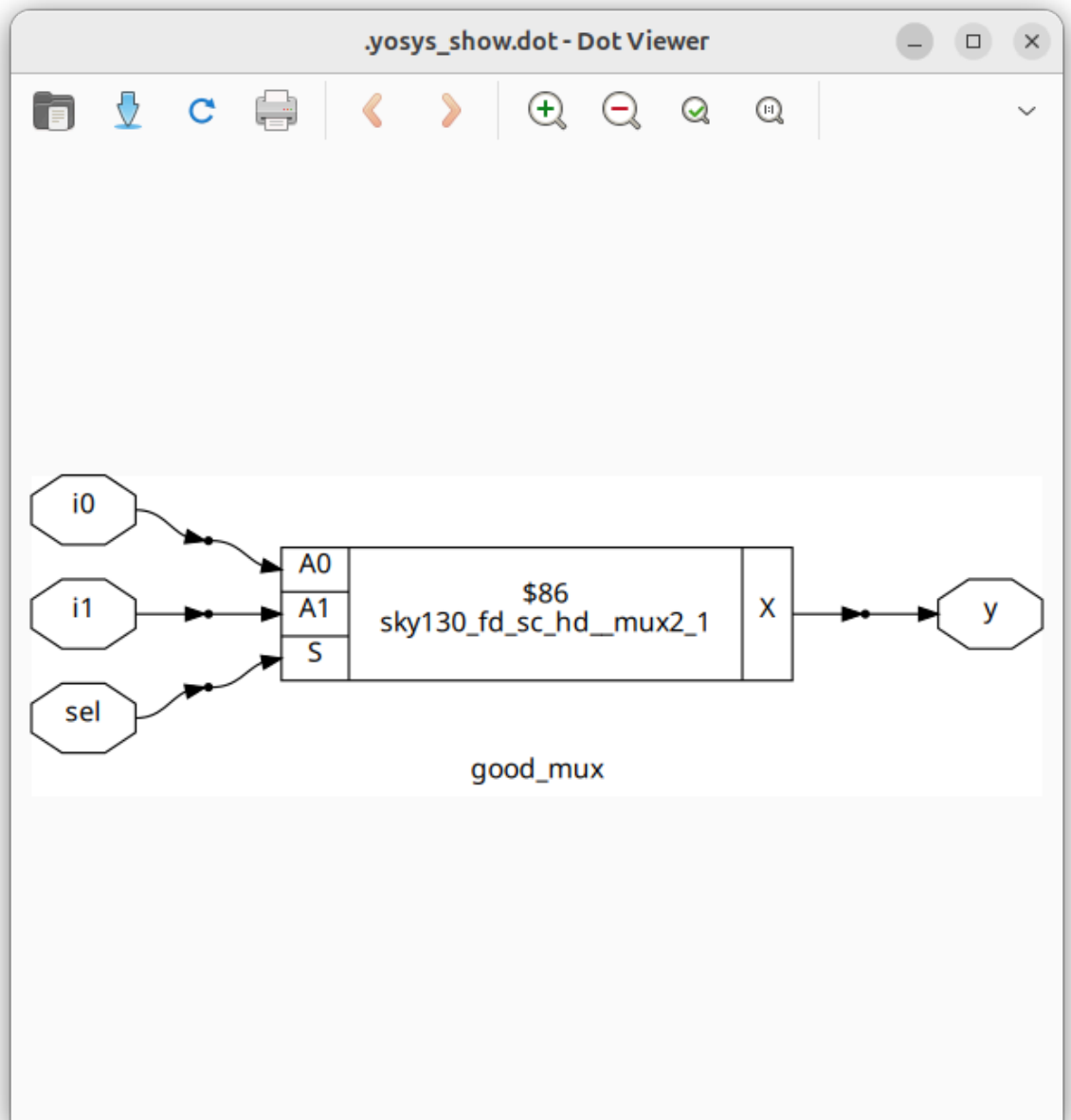
```
1 module good_mux (input i0 , input i1 , input sel , output reg y);
2 always @ (*)
3 begin
4     if(sel)
5         y <= i1;
6     else
7         y <= i0;
8 end
9 endmodule
```

b. RTL level simulation and waveform



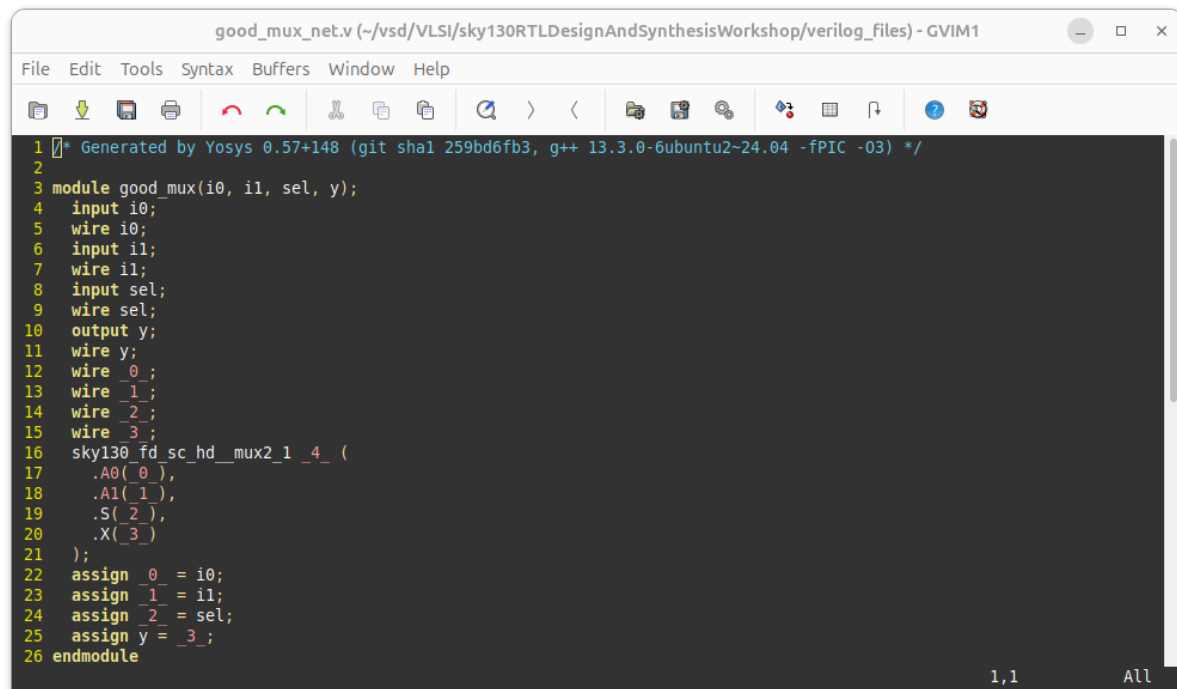
Week 1 Labs

c. Synthesis Diagram



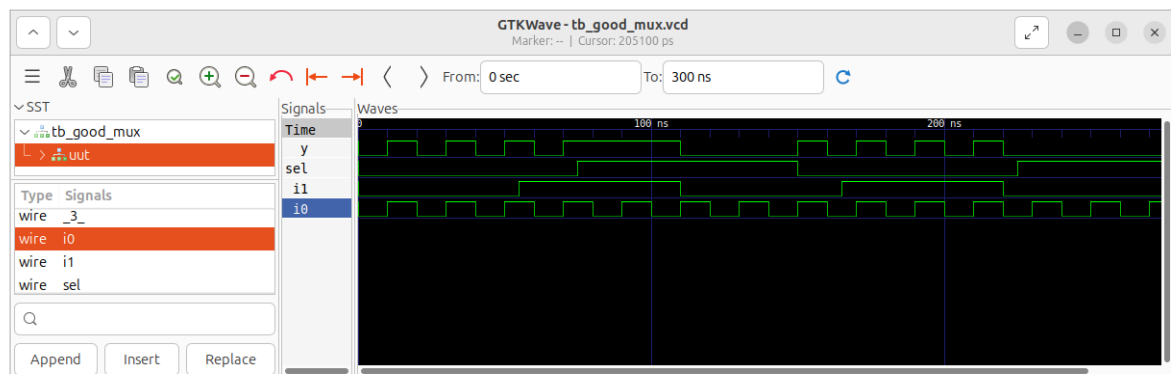
Week 1 Labs

d. Netlist generation Netlist code



```
1 Generated by Yosys 0.57+148 (git sha1 259bd6fb3, g++ 13.3.0-6ubuntu2~24.04 -fPIC -O3) */
2
3 module good_mux(i0, i1, sel, y);
4   input i0;
5   wire i0;
6   input i1;
7   wire i1;
8   input sel;
9   wire sel;
10  output y;
11  wire y;
12  wire _0_;
13  wire _1_;
14  wire _2_;
15  wire _3_;
16  sky130_fd_sc_hd_mux2_1_4 (
17    .A0(_0_),
18    .A1(_1_),
19    .S(_2_),
20    .X(_3_)
21  );
22  assign _0_ = i0;
23  assign _1_ = i1;
24  assign _2_ = sel;
25  assign y = _3_;
26 endmodule
```

e. GLS simulation and waveform



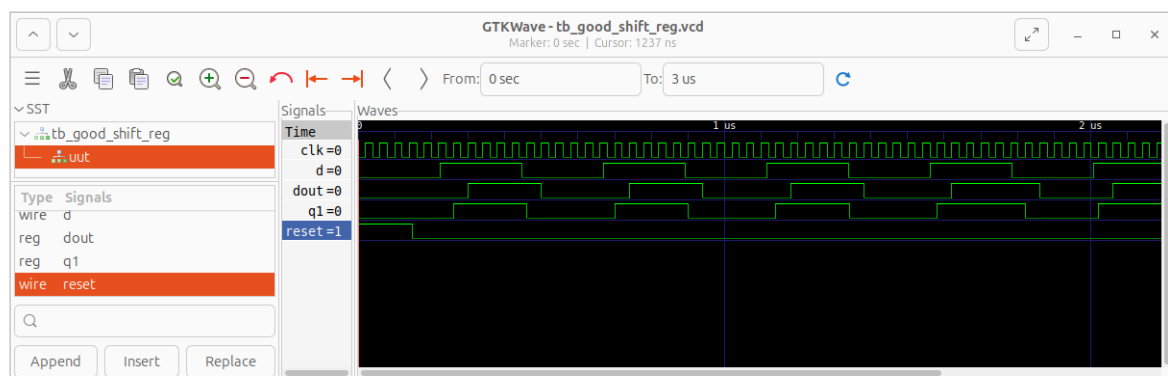
Week 1 Labs

20. Design Name - good_shift_reg

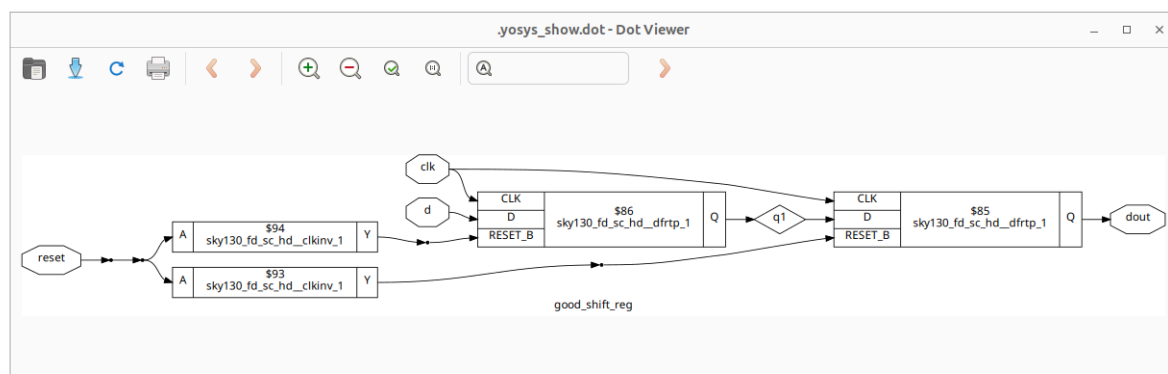
a. Design code

```
good_shift_reg.v (~vsd/VLSI/sky130RTLDesignAndSynthesisWorkshop/verilog_files) - GVIM
File Edit Tools Syntax Buffers Window Help
1 module good_shift_reg (input clk , input reset , input d , output reg dout);
2 reg q1;
3 always @ (posedge clk , posedge reset)
4 begin
5     if(reset)
6     begin
7         q1 <= 1'b0;
8         dout <= 1'b0;
9     end
10    else
11    begin
12        dout <= q1;
13        q1 <= d;
14    end
15 end
16 endmodule
```

b. RTL level simulation and waveform

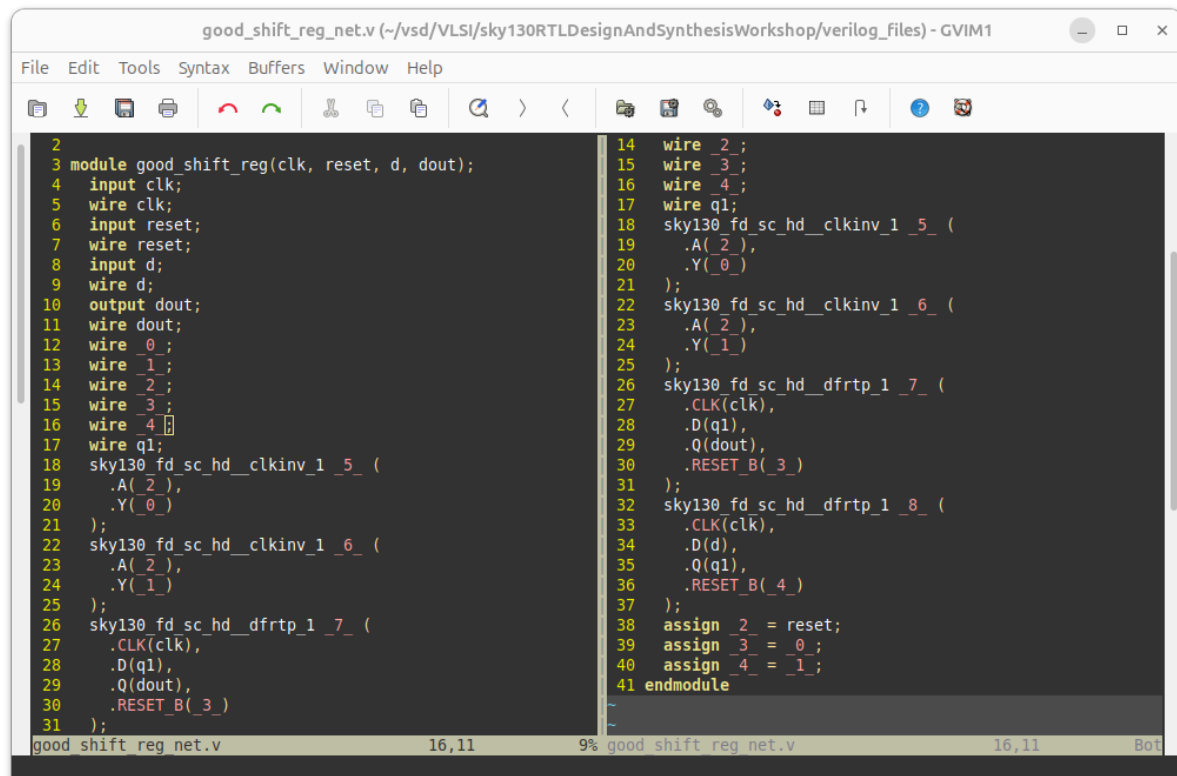


c. Synthesis Diagram



Week 1 Labs

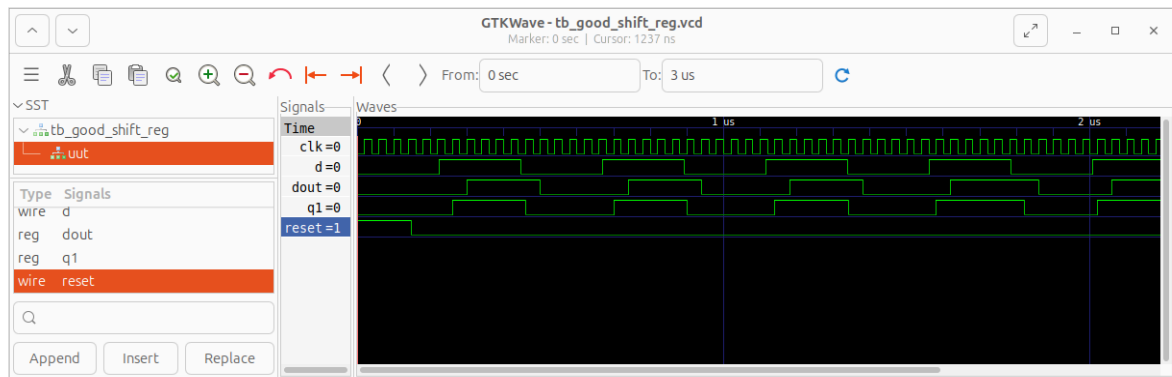
d. Netlist generation



The screenshot shows a Verilog code editor window titled "good_shift_reg_net.v (~/.vsd/VLSI/sky130RTLDesignAndSynthesisWorkshop/verilog_files) - GVIM1". The code defines a module "good_shift_reg" with inputs "clk", "reset", "d" and output "dout". It uses "sky130" primitives for shift register stages and includes logic for clock and reset control.

```
2 module good_shift_reg(clk, reset, d, dout);
3   input clk;
4   wire clk;
5   input reset;
6   wire reset;
7   input d;
8   wire d;
9   output dout;
10  wire _0;
11  wire _1;
12  wire _2;
13  wire _3;
14  wire _4;
15  wire q1;
16  sky130_fd_sc_hd_clkinv_1 _5 (
17    .A(_2),
18    .Y(_0)
19  );
20  sky130_fd_sc_hd_clkinv_1 _6 (
21    .A(_2),
22    .Y(_1)
23  );
24  sky130_fd_sc_hd_dfrtp_1 _7 (
25    .CLK(clk),
26    .D(q1),
27    .Q(dout),
28    .RESET_B(_3)
29  );
30  sky130_fd_sc_hd_dfrtp_1 _8 (
31    .CLK(clk),
32    .D(d),
33    .Q(q1),
34    .RESET_B(_4)
35  );
36  assign _2 = reset;
37  assign _3 = _0;
38  assign _4 = _1;
39 endmodule
```

e. GLS simulation and waveform



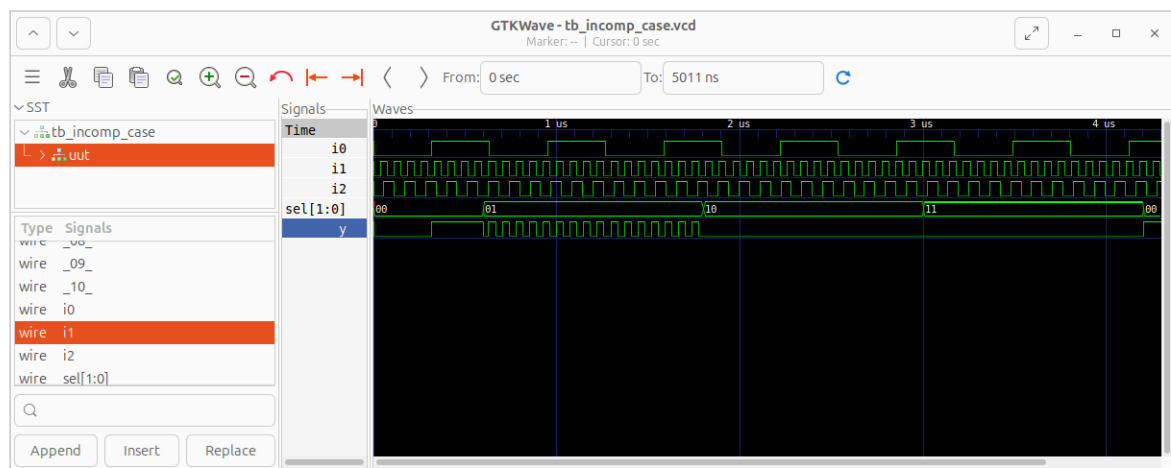
Week 1 Labs

21. Design Name - incomp_case

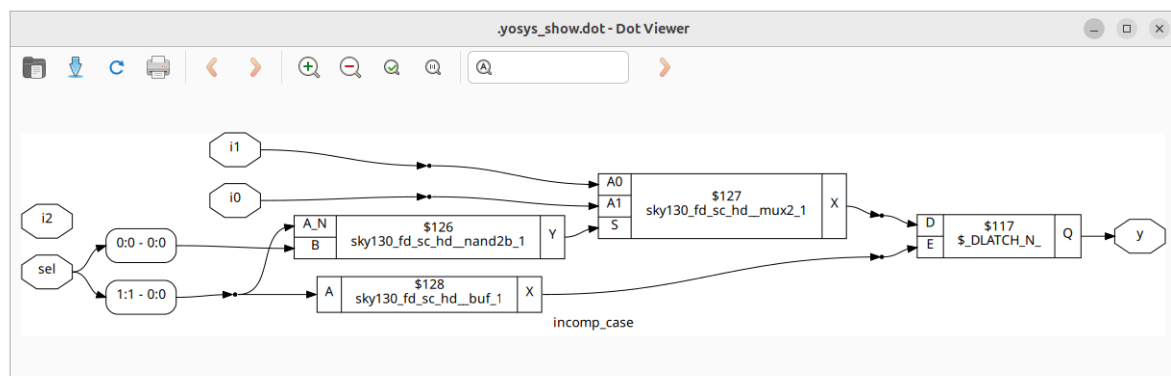
a. Design code

```
incomp_case.v (~vsd/VLSI/sky130RTLDesignAndSynthesisWorkshop/verilog_files) - GVIM
File Edit Tools Syntax Buffers Window Help
1
2 module incomp_case (input i0 , input i1 , input i2 , input [1:0] sel, output reg y);
3 always @ (*)
4 begin
5     case(sel)
6         2'b00 : y = i0;
7         2'b01 : y = i1;
8     endcase
9 end
10 endmodule
```

b. RTL level simulation and waveform

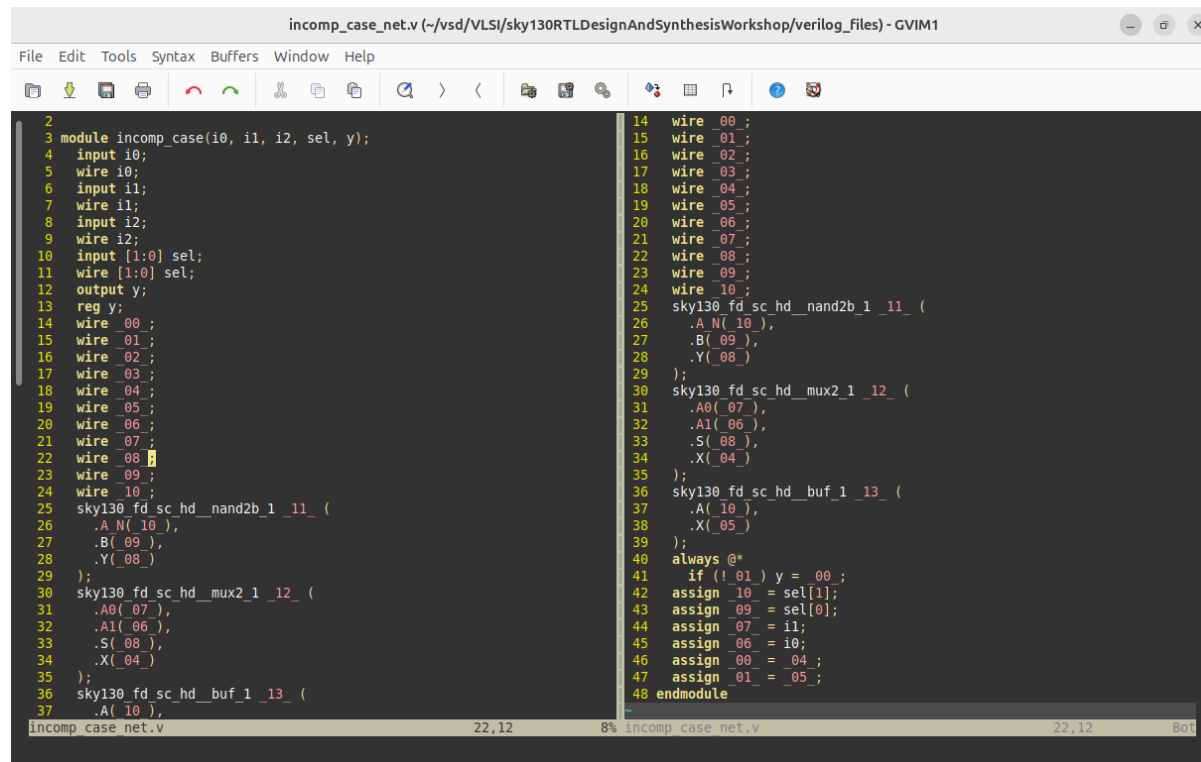


c. Synthesis Diagram



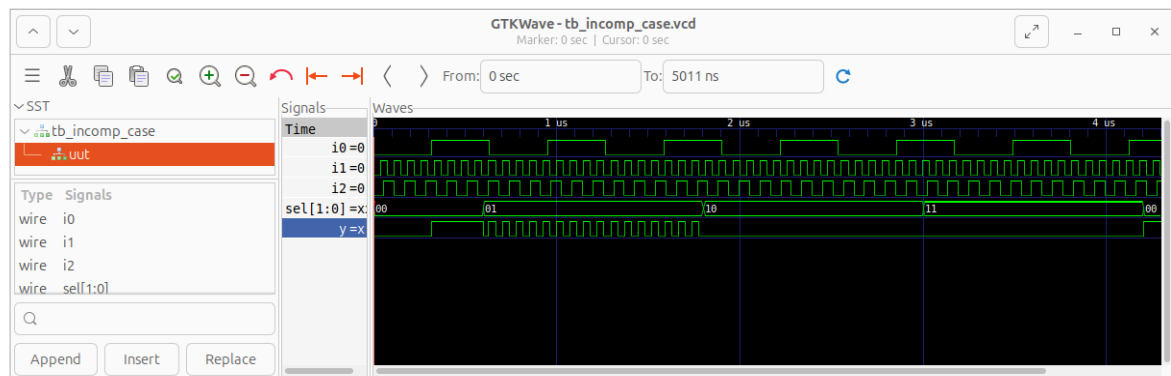
Week 1 Labs

d. Netlist generation



```
2 module incmp_case(i0, i1, i2, sel, y);
3   input i0;
4   wire i0;
5   input i1;
6   wire i1;
7   input i2;
8   wire i2;
9   input [1:0] sel;
10  wire [1:0] sel;
11  output y;
12  reg y;
13  wire _00_;
14  wire _01_;
15  wire _02_;
16  wire _03_;
17  wire _04_;
18  wire _05_;
19  wire _06_;
20  wire _07_;
21  wire _08_;
22  wire _09_;
23  wire _10_;
24  sky130_fd_sc_hd_nand2b_1_11_ (
25    .A(N(_00_)),
26    .B(_09_),
27    .Y(_08_)
28  );
29  sky130_fd_sc_hd_mux2_1_12_ (
30    .A0(_07_),
31    .A1(_06_),
32    .S(_08_),
33    .X(_04_)
34  );
35  sky130_fd_sc_hd_buf_1_13_ (
36    .A(_10_),
37    .X(_05_)
38  );
39  always @*
40  if (! _01_) y = _00_;
41  assign _10_ = sel[1];
42  assign _09_ = sel[0];
43  assign _07_ = i1;
44  assign _06_ = i0;
45  assign _00_ = _04_;
46  assign _01_ = _05_;
47  endmodule
```

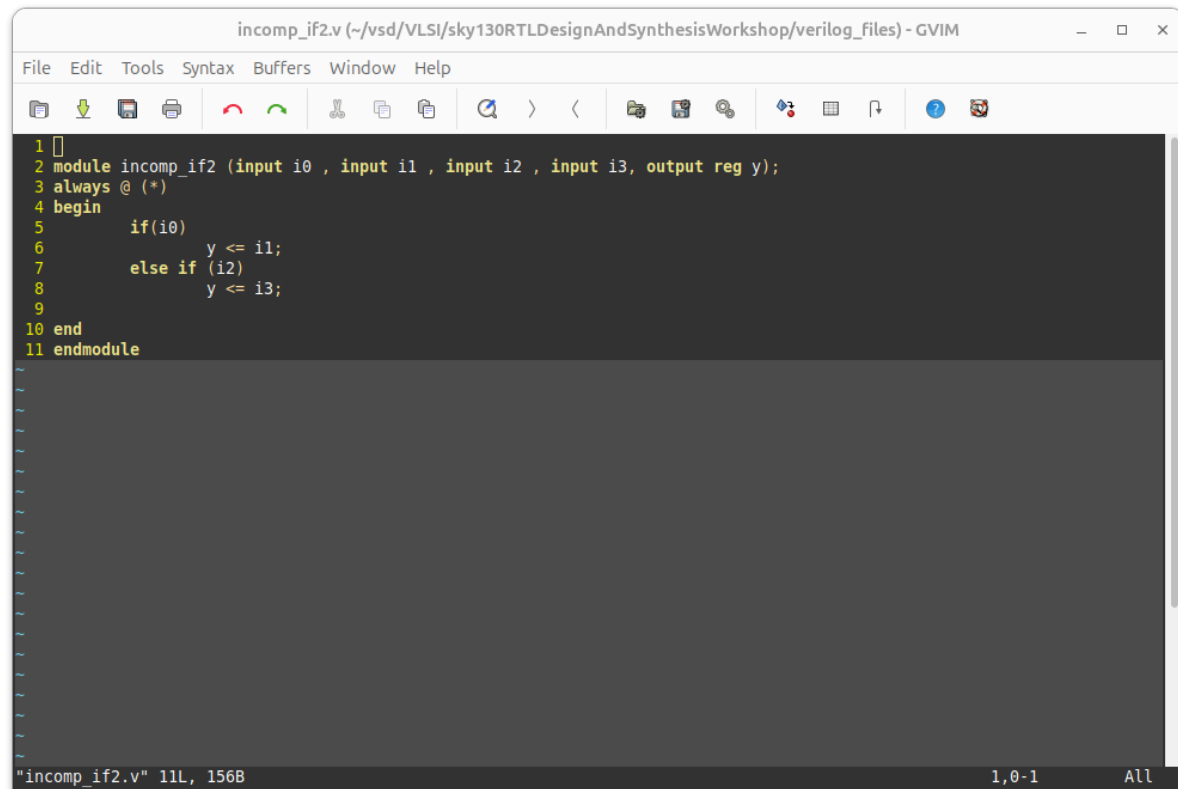
e. GLS simulation and waveform



Week 1 Labs

22. Design Name incomp_if2

a. Design code

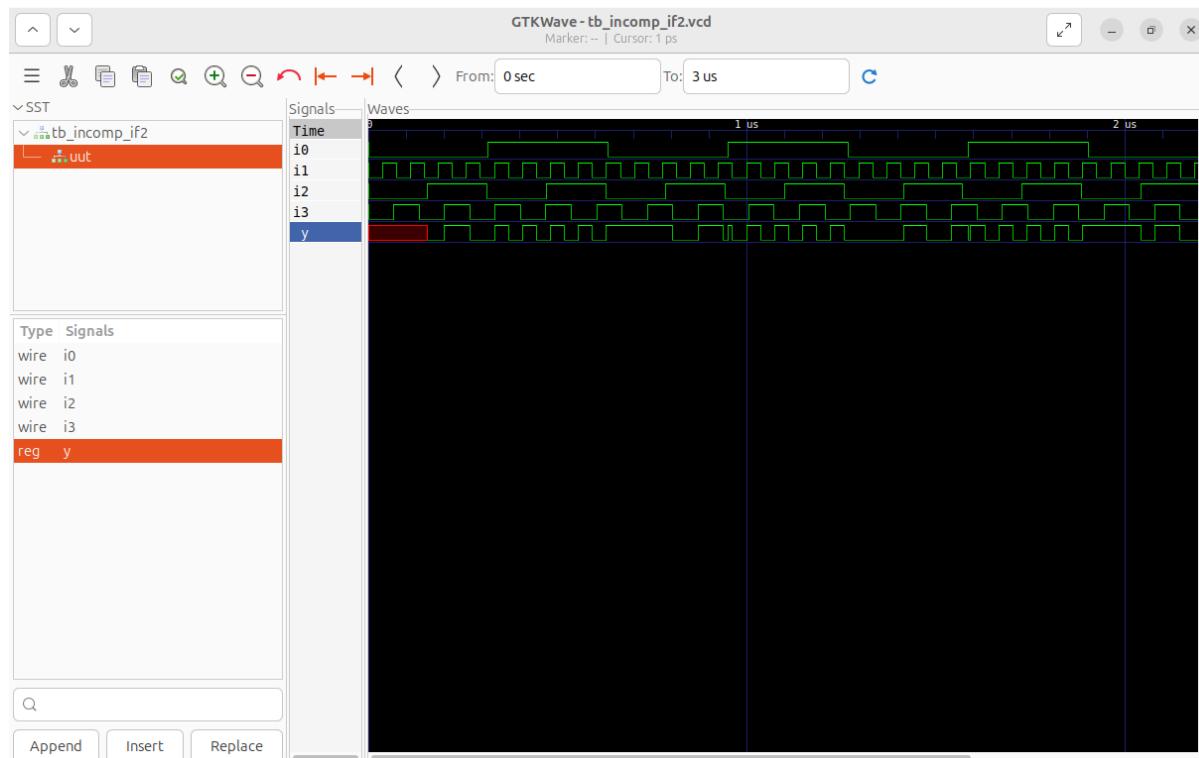


The screenshot shows a Verilog code editor window titled "incomp_if2.v (~\vsd\VLSI\sky130RTLDesignAndSynthesisWorkshop\verilog_files) - GVIM". The code is as follows:

```
1 []
2 module incomp_if2 (input i0 , input i1 , input i2 , input i3, output reg y);
3 always @ (*)
4 begin
5     if(i0)
6         y <= i1;
7     else if (i2)
8         y <= i3;
9
10 end
11 endmodule
```

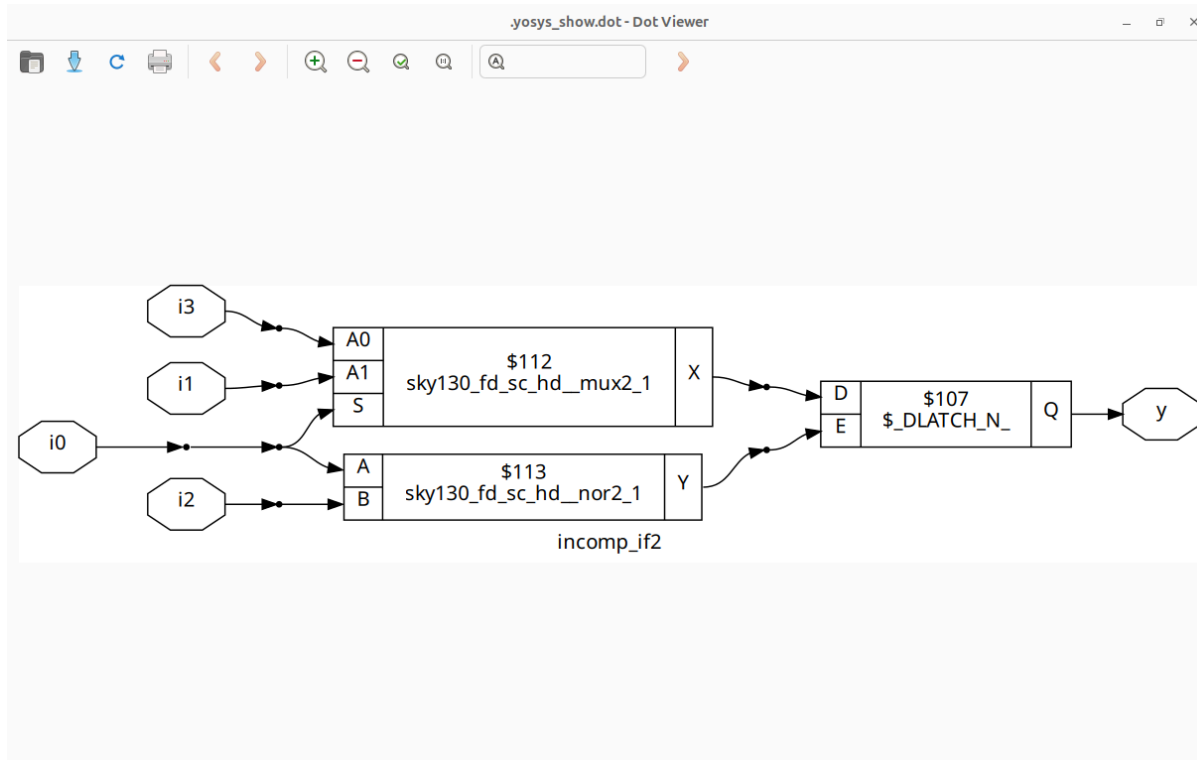
The status bar at the bottom indicates "incomp_if2.v" 11L, 156B, 1,0-1, and All.

b. RTL level simulation and waveform



Week 1 Labs

c. Synthesis Diagram

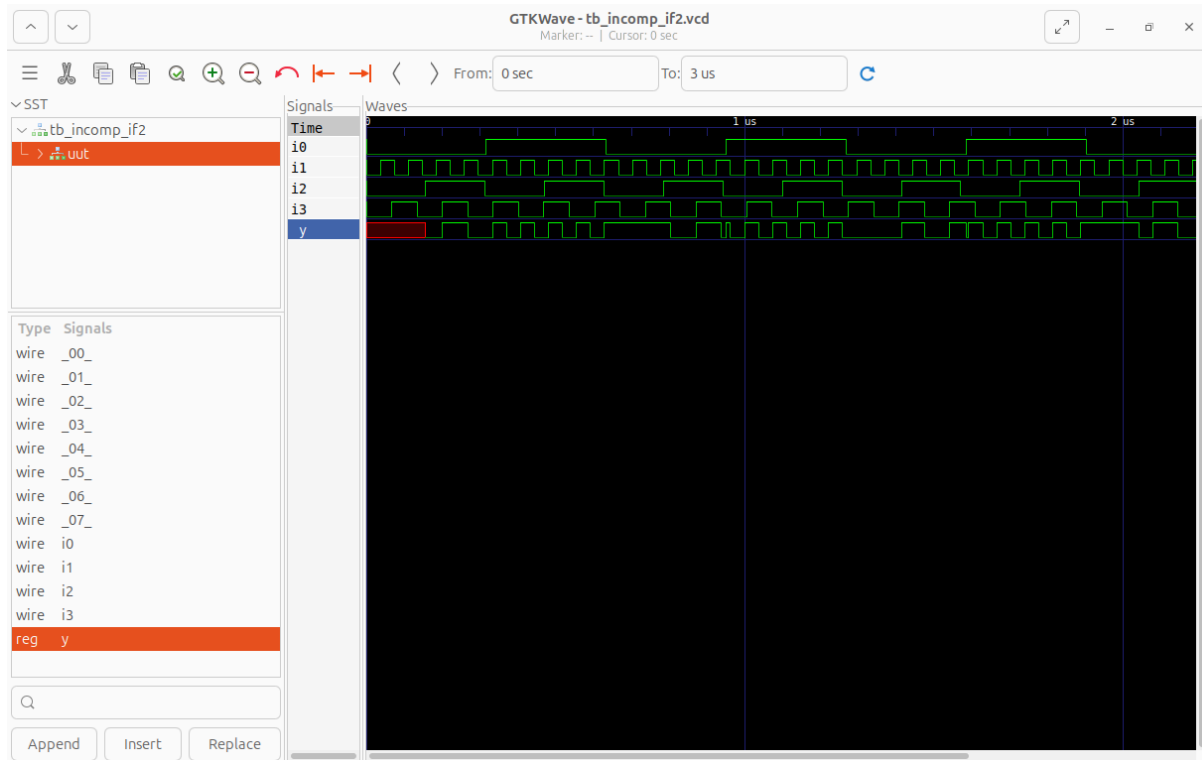


d. Netlist generation

```
incomp_if2_net.v (~:/vsd/VLSI/sky130RTLDesignAndSynthesisWorkshop/verilog_files) - GVIM1
File Edit Tools Syntax Buffers Window Help
1 /* Generated by Yosys 0.57+148 (git sha1 259bd6fb3, g++ 13.3.0-6ubuntu2~24.04 -fPIC -O3) */
2
3 module incomp_if2(i0, i1, i2, i3, y);
4   input i0;
5   wire i0;
6   input i1;
7   wire i1;
8   input i2;
9   wire i2;
10  input i3;
11  wire i3;
12  output y;
13  reg y;
14  wire _00_;
15  wire _01_;
16  wire _02_;
17  wire _03_;
18  wire _04_;
19  wire _05_;
20  wire _06_;
21  wire _07_;
22  sky130_fd_sc_hd_mux2_1 _08_ (
23    .A0(_07_),
24    .A1(_05_),
25    .S(_04_),
26    .X(_02_)
27  );
28  sky130_fd_sc_hd_nor2_1 _09_ (
29    .A(_04_),
30    .B(_06_),
31    .Y(_03_)
32  );
33  always @*
34    if (!_01_) y = _00_;
35    assign _07_ = i3;
36    assign _05_ = i1;
37    assign _04_ = i0;
38    assign _00_ = _02_;
39    assign _06_ = i2;
40    assign _01_ = _03_;
41 endmodule
incomp_if2_net.v 10,1 Top incomp_if2_net.v 10,1 Bot
```

Week 1 Labs

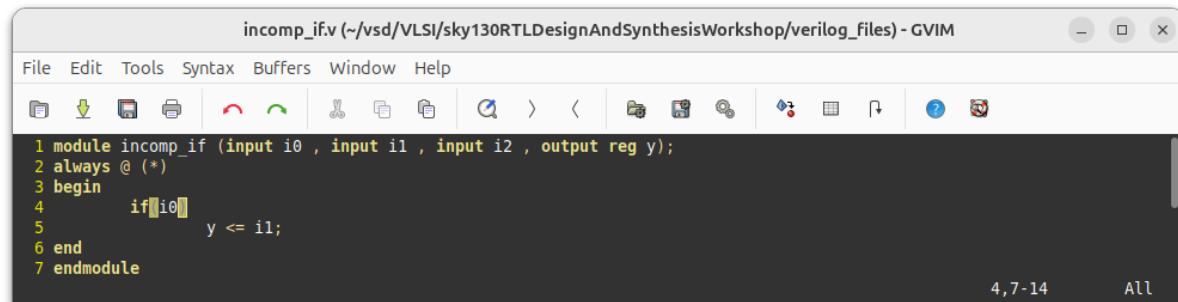
e. GLS simulation and waveform



Week 1 Labs

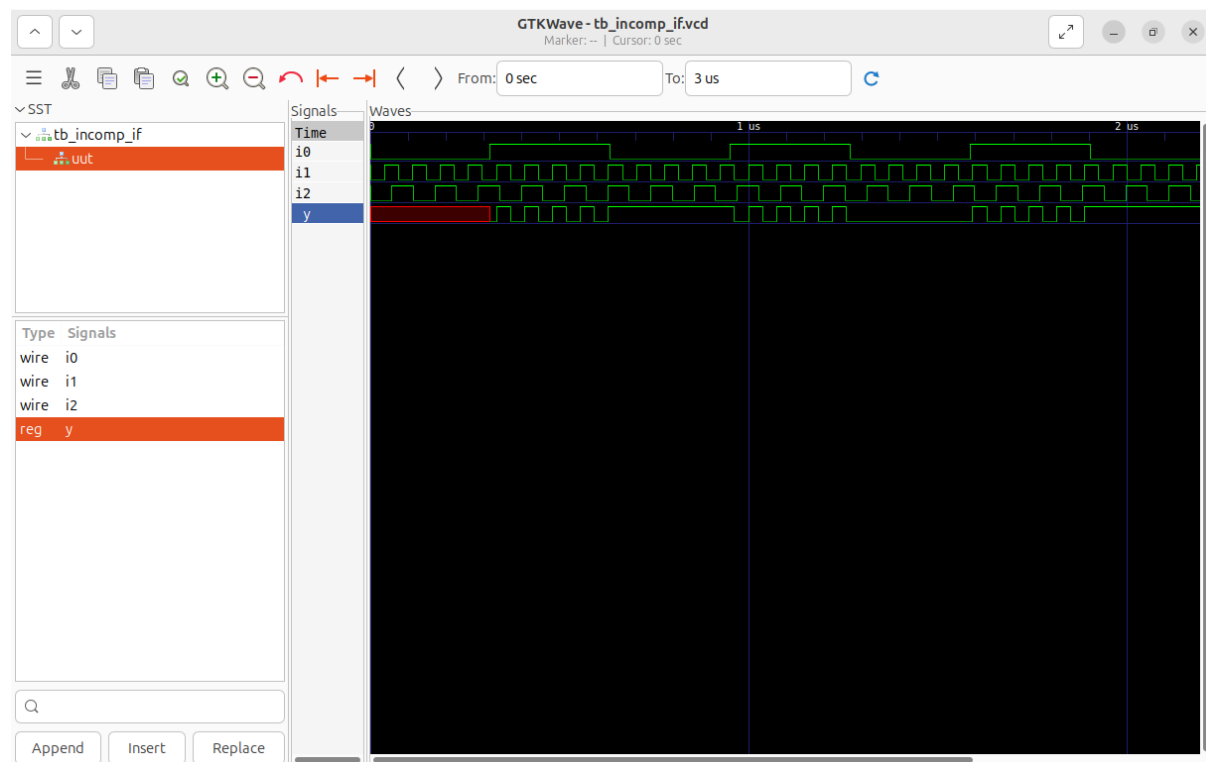
23. Design Name incomp_if

a. Design code



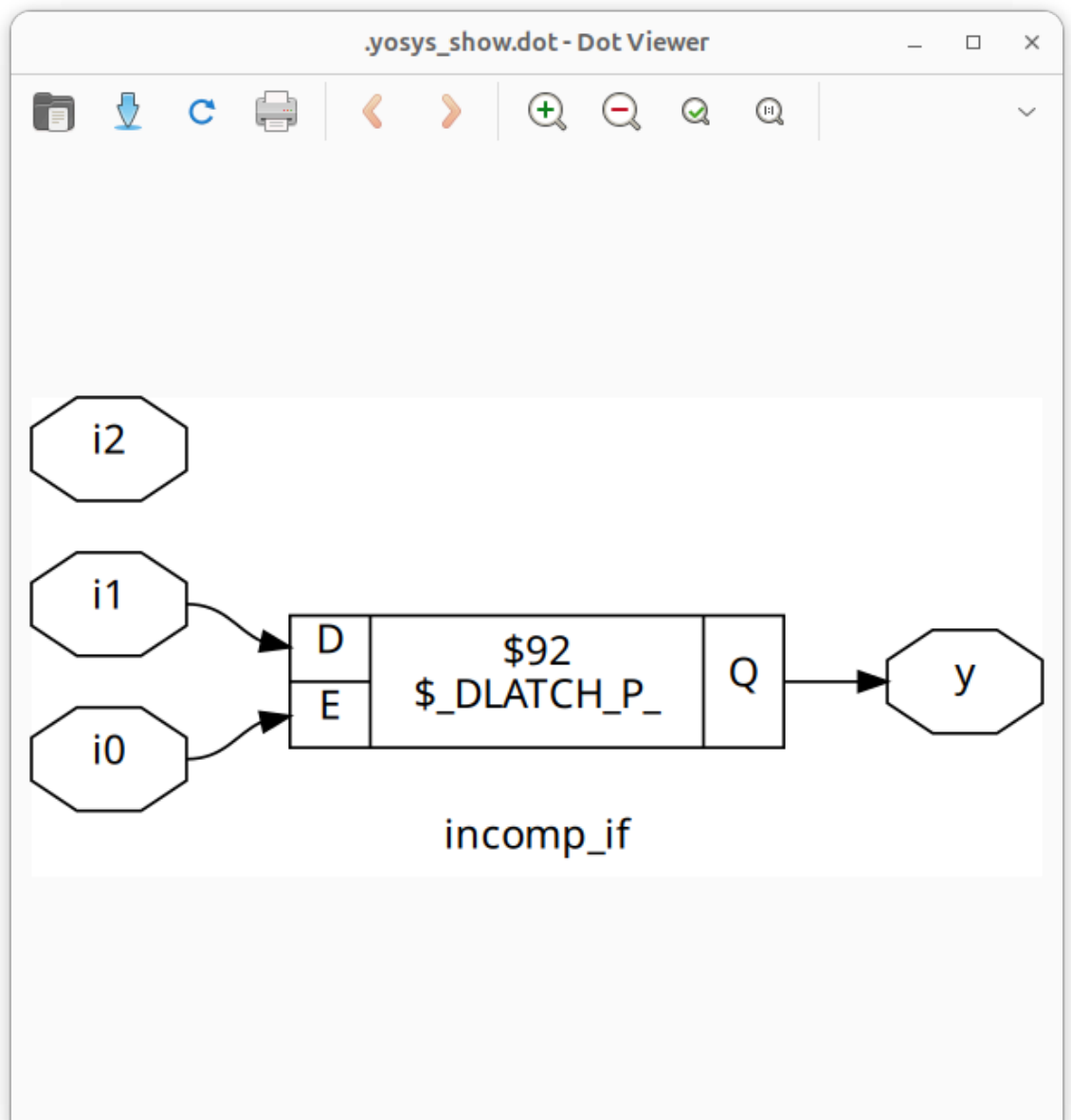
```
1 module incomp_if (input i0 , input i1 , input i2 , output reg y);
2 always @ (*)
3 begin
4     if(i0)
5         y <= i1;
6 end
7 endmodule
```

b. RTL level simulation and waveform



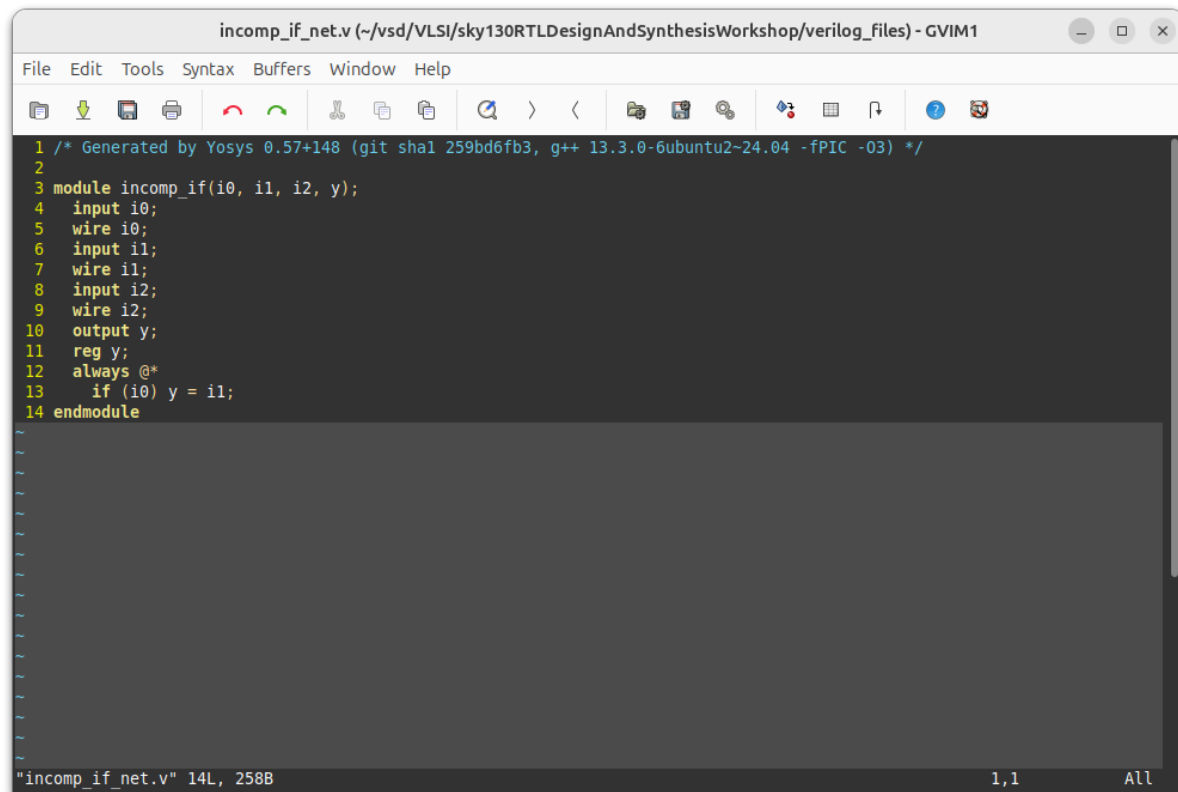
Week 1 Labs

c. Synthesis Diagram



Week 1 Labs

d. Netlist generation

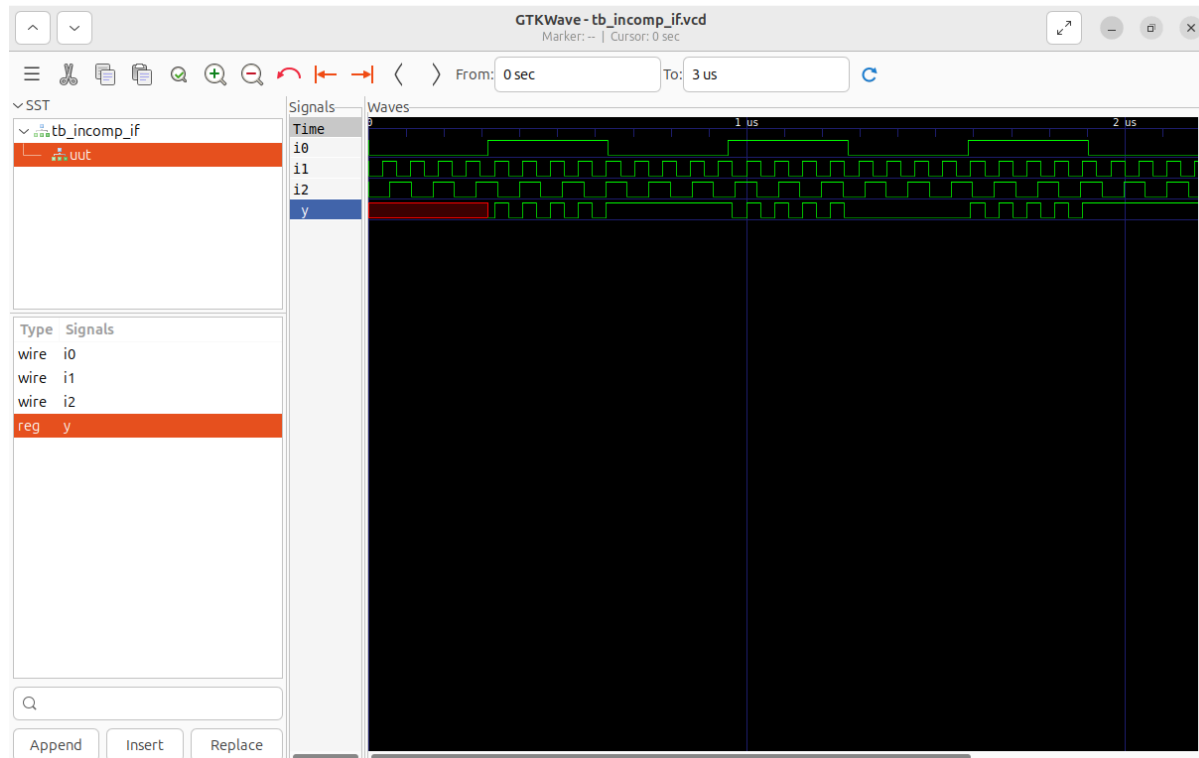


The screenshot shows a Verilog code editor window titled "incomp_if_net.v (~/.vsd/VLSI/sky130RTLDesignAndSynthesisWorkshop/verilog_files) - GVIM1". The code defines a module named `incomp_if` with three inputs (`i0`, `i1`, `i2`) and one output (`y`). The logic is as follows:

```
1 /* Generated by Yosys 0.57+148 (git sha1 259bd6fb3, g++ 13.3.0-6ubuntu2~24.04 -fPIC -O3) */
2
3 module incomp_if(i0, i1, i2, y);
4     input i0;
5     wire i0;
6     input i1;
7     wire i1;
8     input i2;
9     wire i2;
10    output y;
11    reg y;
12    always @*
13        if (i0) y = i1;
14 endmodule
```

The status bar at the bottom indicates the file is "incomp_if_net.v" with 14 lines and 258 bytes. The cursor is at line 1, column 1.

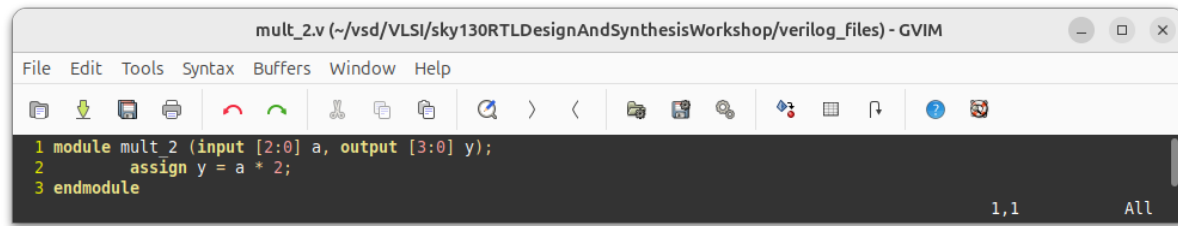
e. GLS simulation and waveform



Week 1 Labs

24. Design Name mult_2

a. Design code

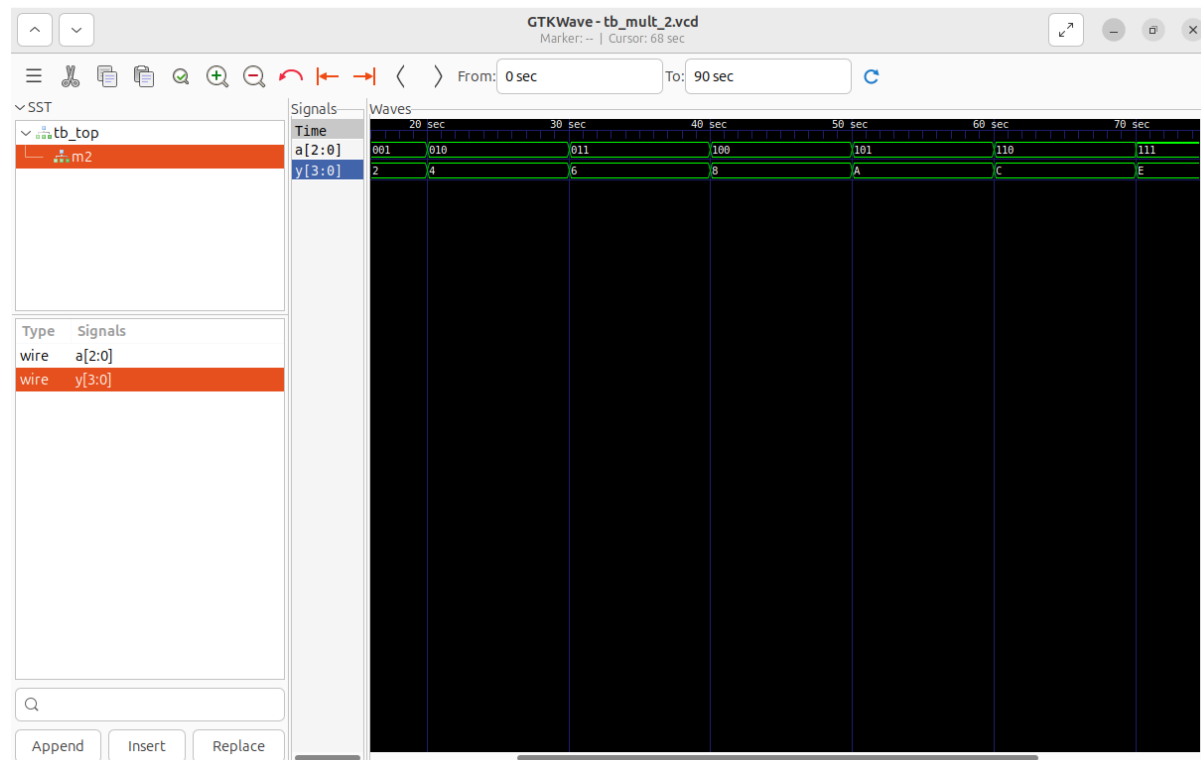


The screenshot shows a Verilog code editor window titled "mult_2.v (~vsd/VLSI/sky130RTLDesignAndSynthesisWorkshop/verilog_files) - GVIM". The code is as follows:

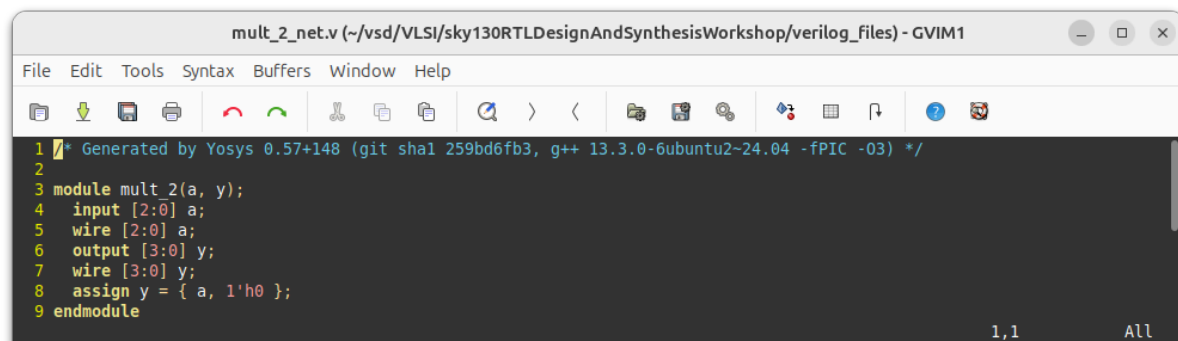
```
1 module mult_2 (input [2:0] a, output [3:0] y);
2   assign y = a * 2;
3 endmodule
```

The status bar at the bottom right indicates "1,1" and "All".

b. RTL level simulation and waveform



c. Synthesis Diagram



The screenshot shows a Verilog code editor window titled "mult_2_net.v (~vsd/VLSI/sky130RTLDesignAndSynthesisWorkshop/verilog_files) - GVIM1". The code is as follows:

```
1 /* Generated by Yosys 0.57+148 (git sha1 259bd6fb3, g++ 13.3.0-6ubuntu2-24.04 -fPIC -O3) */
2
3 module mult_2(a, y);
4   input [2:0] a;
5   wire [2:0] a;
6   output [3:0] y;
7   wire [3:0] y;
8   assign y = { a, 1'h0 };
9 endmodule
```

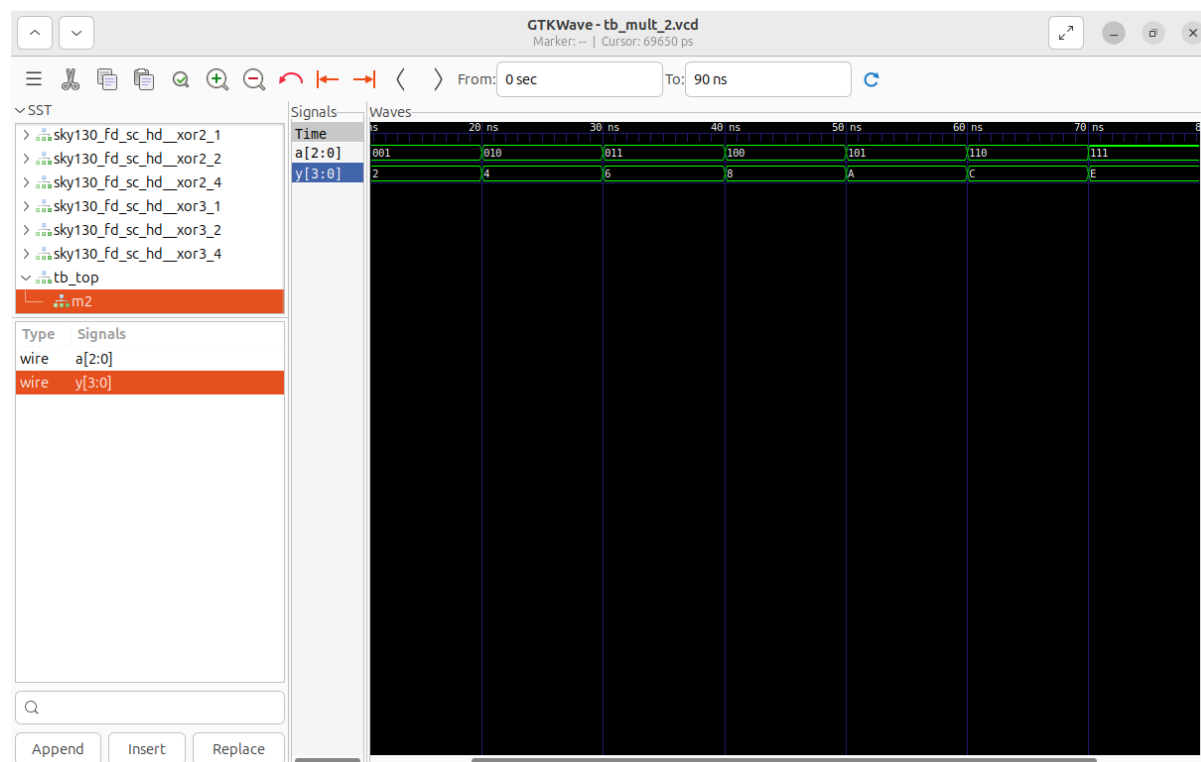
The status bar at the bottom right indicates "1,1" and "All".

Week 1 Labs

d. Netlist generation

```
1 /* Generated by Yosys 0.57+148 (git sha1 259bd6fb3, g++ 13.3.0-6ubuntu2-24.04 -fPIC -O3) */
2
3 module mult_2(a, y);
4 input [2:0] a;
5 wire [2:0] a;
6 output [3:0] y;
7 wire [3:0] y;
8 assign y = { a, 1'h0 };
9 endmodule
```

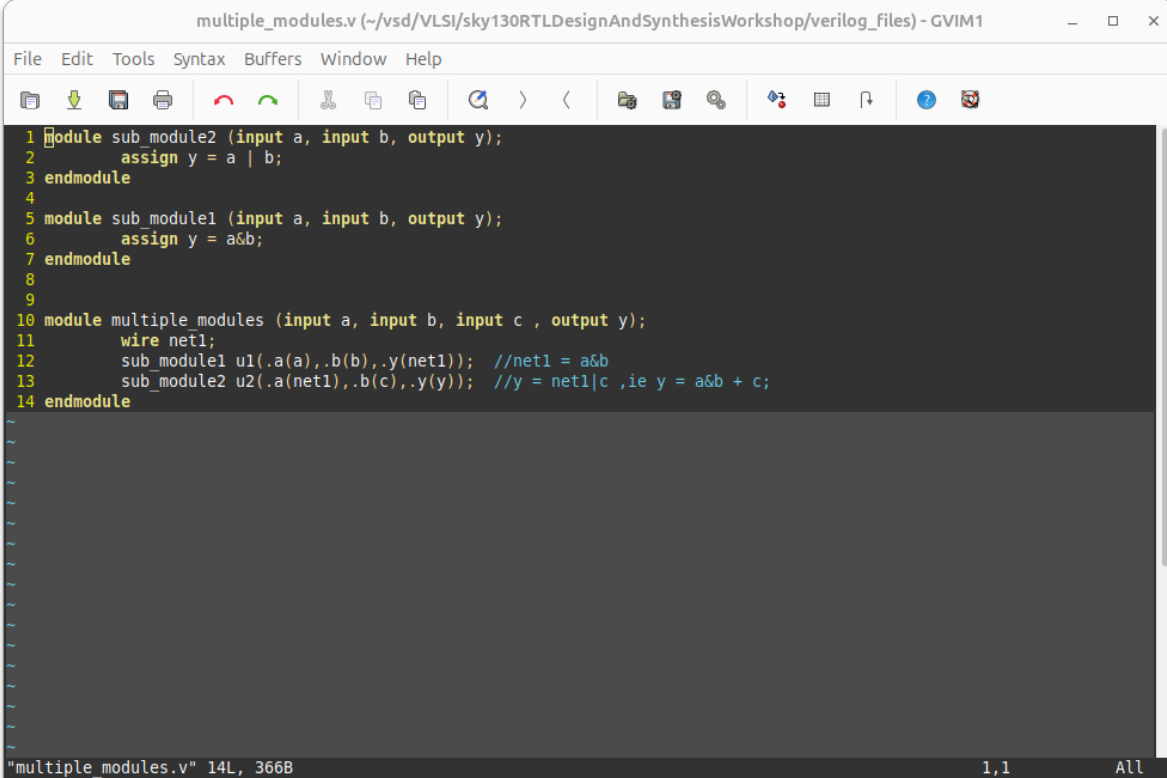
e. GLS simulation and waveform



Week 1 Labs

25. Design Name multiple_module

a. Design code



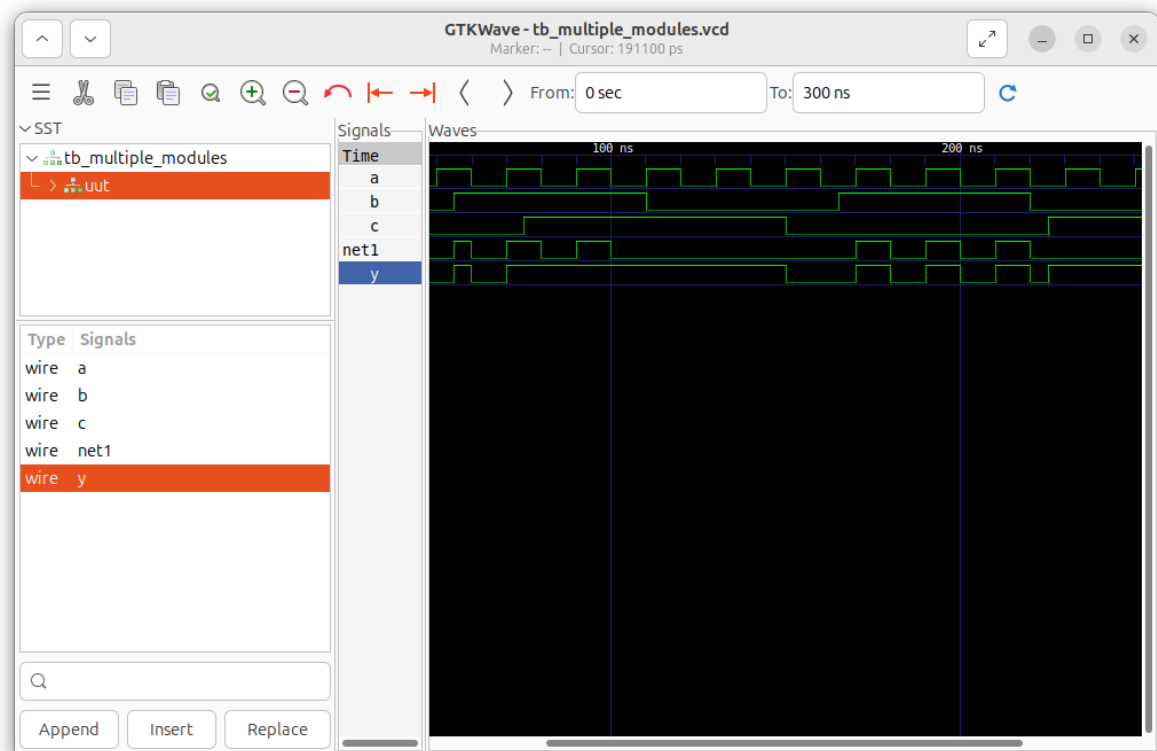
The screenshot shows a Verilog code editor window titled "multiple_modules.v (~\vsd\VLSI\sky130RTLDesignAndSynthesisWorkshop\verilog_files) - GVIM1". The editor has a menu bar (File, Edit, Tools, Syntax, Buffers, Window, Help) and a toolbar with various icons. The code is as follows:

```
1 module sub_module2 (input a, input b, output y);
2     assign y = a | b;
3 endmodule
4
5 module sub_module1 (input a, input b, output y);
6     assign y = a&b;
7 endmodule
8
9
10 module multiple_modules (input a, input b, input c , output y);
11     wire net1;
12     sub_module1 u1(.a(a),.b(b),.y(net1)); //net1 = a&b
13     sub_module2 u2(.a(net1),.b(c),.y(y)); //y = net1|c ,ie y = a&b + c;
14 endmodule
```

The status bar at the bottom indicates the file is "multiple_modules.v" with 14 lines and 366 bytes. The cursor is at line 1, column 1, and the search scope is set to "All".

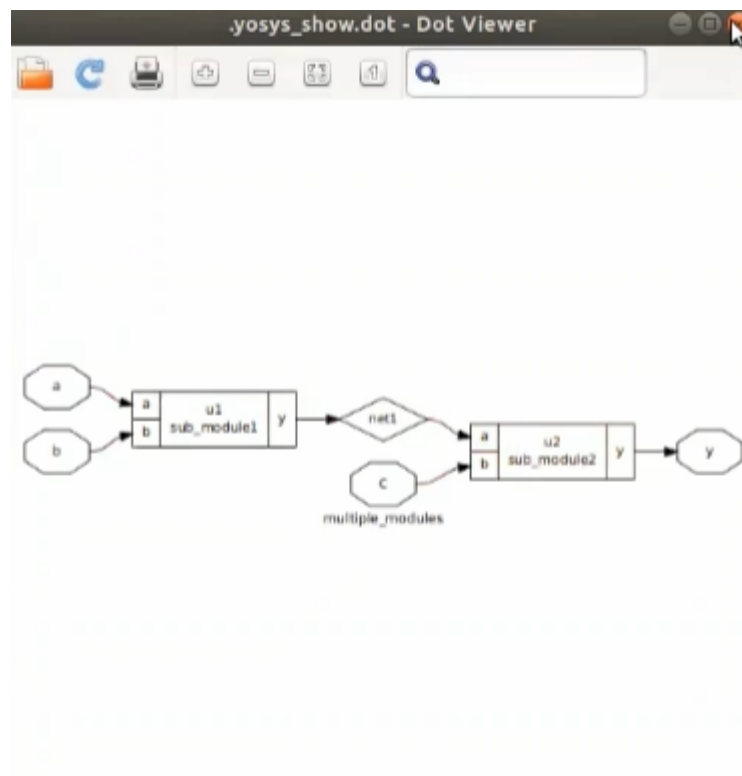
Week 1 Labs

b. RTL level simulation and waveform



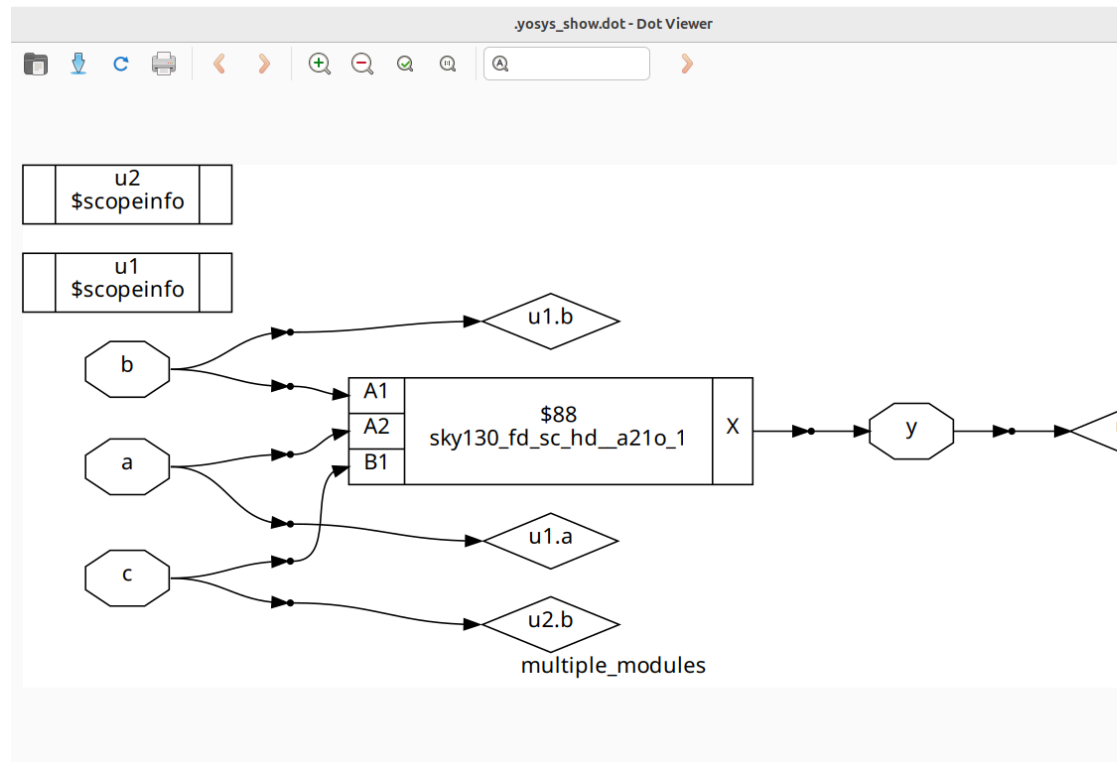
c. Synthesis Diagram

i. Hier



Week 1 Labs

ii. Flat



d. Netlist generation

i. Hier

```
mmh.v (~\vsd\VLSI/sky130RTLDesignAndSynthesisWorkshop/verilog_files) - GVIM3
File Edit Tools Syntax Buffers Window Help
[Icons]

2
3 module multiple_modules(a, b, c, y);
4   input a;
5   wire a;
6   input b;
7   wire b;
8   input c;
9   wire c;
10  output y;
11  wire y;
12  wire net1;
13  sub_module1 u1 (
14    .a(a),
15    .b(b),
16    .y(net1)
17  );
18  sub_module2 u2 (
19    .a(net1),
20    .b(c),
21    .y(y)
22  );
23 endmodule
24
25 module sub_module1(a, b, y);
26   input a;
27   wire a;
28   input b;
29   wire b;
30   output y;
31   wire y;
32   assign y = b & a;
33 endmodule
34
35 module sub_module2(a, b, y);
36   input a;
37   wire a;
38   input b;
39   wire b;
40   output y;
41   wire y;
42   assign y = b | a;
43 endmodule
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```

Week 1 Labs

ii. Flat

```
multiple_modules_flat.v (~vsd/VLSI/sky130RTLDesignAndSynthesisWorkshop/verilog_files) - GVIM3
File Edit Tools Syntax Buffers Window Help
[Icons]

2
3 module multiple_modules(a, b, c, y);
4   wire _00_;
5   wire _01_;
6   wire _02_;
7   wire _03_;
8   wire _04_;
9   wire _05_;
10  wire _06_;
11  wire _07_;
12  input a;
13  input b;
14  input c;
15  wire net1;
16  wire \u1.a ;
17  wire \u1.b ;
18  wire \u1.y ;
19  wire \u2.a ;
20  wire \u2.b ;
21  wire \u2.y ;
22  output y;
23  sky130_fd_sc_hd_and2_2 _08_ (
24    .A( _00_ ),
25    .B( _01_ ),
26    .X( _02_ )
27  );
28  sky130_fd_sc_hd_clkinv_1 _09_ (
29    .A( _03_ ),
30    .Y( _06_ )
31  );
32  sky130_fd_sc_hd_clkinv_1 _10_ (
33    .A( _04_ ),
34    .Y( _07_ )
35  );
36  sky130_fd_sc_hd_nand2_1 _11_ (
37    .A( _06_ ),
38    .B( _07_ ),
39    .Y( _05_ )
40  );
41  assign \u1.a = a;
42  assign \u1.b = b;
43  assign net1 = \u1.y ;
44  assign _00_ = \u1.b ;
45  assign _01_ = \u1.a ;
46  assign \u1.y = _02_ ;
47  assign \u2.a = net1;
48  assign \u2.b = c;
49  assign y = \u2.y ;
50  assign _03_ = \u2.b ;
51  assign _04_ = \u2.a ;
52  assign \u2.y = _05_ ;

```